

Proof of Concept Technical Solution for the Marconi Law Firm, LLC. (WordPress Website)

Orlando, Florida

Robert Potter

Office Networks and Security Wizards LLC.

Table of Contents

Inventory.....	6
Custom Network.....	6
IDs and Passwords.....	6
Preface.....	7
Network Topology Diagram.....	8
SSH Install & Access.....	9
OpenSSH Install (Ubuntu)	9
Node.js Application (Ghost) on Docker.....	10
Update Rocky8-Docker Host Name	10
Update Rocky8-Docker	14
Install EPEL Packages	15
Install Nano Editor.....	16
Docker CE.....	17
Set up stable repository	17
Install Docker CE	18
Verify Docker version.....	19
Initialize Docker	20
Start Docker.....	20
Enable Docker.....	21
Test Docker (hello-world)	22
Disable SELinux (/etc/selinux.config)	23
Reboot VM.....	25
Confirm SELinux Status	26
Install Ghost Docker Container	27
Check for Ghost Container (docker ps)	28
Ghost Container ID 03159d327ed1	28
NginX Reverse Proxy.....	29
Update Rocky8-Nginx Host Name	29
Update Rocky8-Nginx	34
Install EPEL Packages	35
Install Nano Editor.....	36
Disable SELinux	37

Reboot VM.....	39
Confirm SELinux Status	40
Rocky Firewall	41
Stop Firewall	41
Disable Firewall	42
NginX.....	43
Install NginX	43
Start NginX	45
Enable NginX.....	45
Confirm NginX Status	46
Reverse Proxy for Ghost Site.....	47
Edit NginX configuration file	47
Reload NginX service.....	47
Terminate Docker	47
Delete First Ghost Container.....	48
Create New Ghost Container	48
Browse to Ghost from Firefox on Ubuntu (10.10.229.10/blog).....	49
WordPress on Ubuntu - LAMP Stack.....	50
Update Ubuntu Host Name	50
Update Ubuntu	51
Upgrade Ubuntu	52
Install Nano Editor.....	52
Install Git	53
Install Apache2.....	54
Open Firewall Ports 80 and 443	55
Browse to Apache2 Ubuntu Default Page	55
Install MySQL.....	56
Alter root user password (root@localhost)	57
Flush Privileges.....	58
Exit MySQL	58
Install PHP.....	59
Install Required PHP Libraries	59
Install Required MySQL Libraries	60
Enable URL Rewrites (clean URLs)	61
Restart Apache Service	61
Create a test.php Web Page	62
Test the test.php Web Page.....	63
Database Configuration in MySQL Log into MySQL	64
Create MySQL WordPress Database (WordPressDB)	65
Create MySQL WordPress User (WordPressUser)	66
Grant Privileges to the WordPress Database (WordPressDB) to WordPress User (WordPressUser).....	67

Flush Privileges.....	68
Exit MySQL	69
Install WordPress	70
Grant Permission to /var/www/html/ Directory to WordPress User	70
Delete Files from /var/www/html/ Directory	70
Verify /var/www/html/ Directory is Empty	71
Clone WordPress to /var/www/html/ Directory	72
Verify /var/www/html/ Directory Contains WordPress Files	72
Verify Permissions on /var/www/html/ Directory	73
Edit Ownership	74
Edit Ownership of and the contents of /var/www/html/ Directory	74
Edit the apache2.conf File.....	74
Override All Default Apache Directives.....	76
Create a .htaccess File in the /var/www/html/.git/ Directory	77
Restart the Apache Service	78
WordPress Configuration.....	79
Configure WordPress.....	79
WordPress Configuration Selections	79
Run Installation	83
Create an Admin WordPress User	84
WordPress Site Selections	86
Test WordPress Website.....	89
WordPress Security Settings and Configurations	94
File Permissions	95
Vulnerability.....	95
Configuration	97
Validation.....	98
Securing wp-config.php	99
Vulnerability.....	99
Configuration	99
Validation.....	99
Firewall (Shield)	100
Vulnerability.....	100
Configuration	101
Validation.....	107
Appendix A.....	108
NginX Config File	108
Appendix B.....	109
NginX Access Log File.....	109

Inventory

EQUIPMENT	OPERATING SYSTEM	ADDITIONAL INFO	IP ADDRESS
Router/Custom Network	- (Firewall VM)	-Firewall VM	10.10.229.1
Docker	Rocky 8 (-Docker)	Ghost Container	10.10.229.11
NginX Reverse Proxy	Rocky 8 (-Nginx)	Reverse Proxy	10.10.229.10
WordPress	Ubuntu	LAMP Stack running WordPress	10.10.229.12

Custom Network

NETWORK NAME	SUBNET IP	SUBNET MASK	DNS	GATEWAY
ITE229	10.10.229.0	255.255.255.0	10.10.229.1	10.10.229.1

IDs and Passwords

ACCOUNT	USER ID	PASSWORD
Rocky8-Docker Root User	root	Fullsail1!
Rocky8-Nginx Root User	root	Fullsail1!
Ubuntu Root User	Root	Fullsail1!
MySQL Root User	root@localhost	Fullsail1!
MySQL WordPress User	WordPressUser	Fullsail2!
WordPress Admin	admin	Fullsail1!

Preface

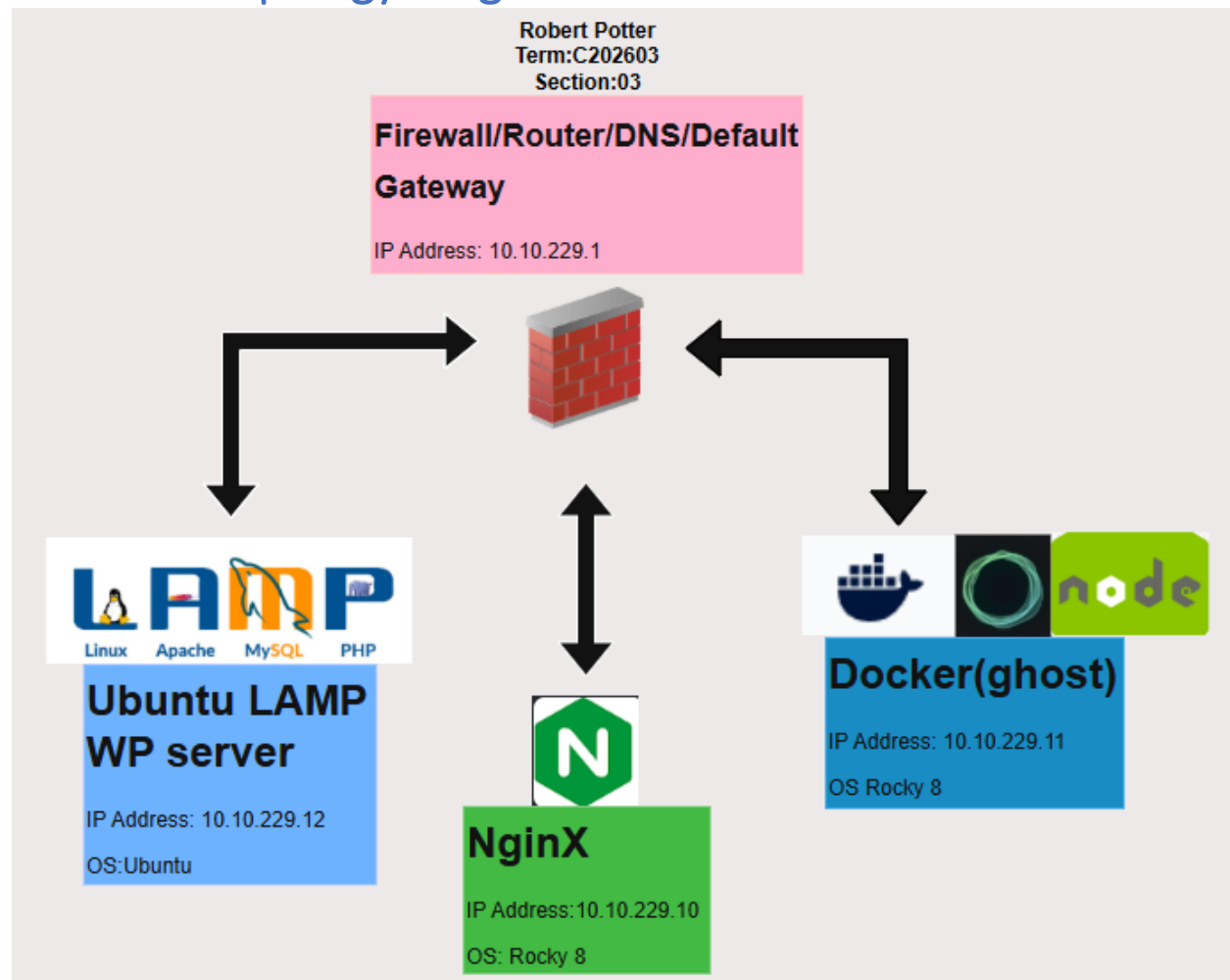
This document will serve as proof of concept to Mr. Marconi for creating his WordPress website for his law firm and as audit documentation.

The purpose of audit documentation is to provide a comprehensive record of the organization's information technology infrastructure and security controls and processes. It plays a crucial role in providing transparency, accountability, and QA/QC regarding an organization's cybersecurity controls and practices. It enables organizations to demonstrate compliance, identify areas for improvement, and make informed decisions to strengthen their overall organizational cybersecurity.

Audit documentation serves several important purposes:

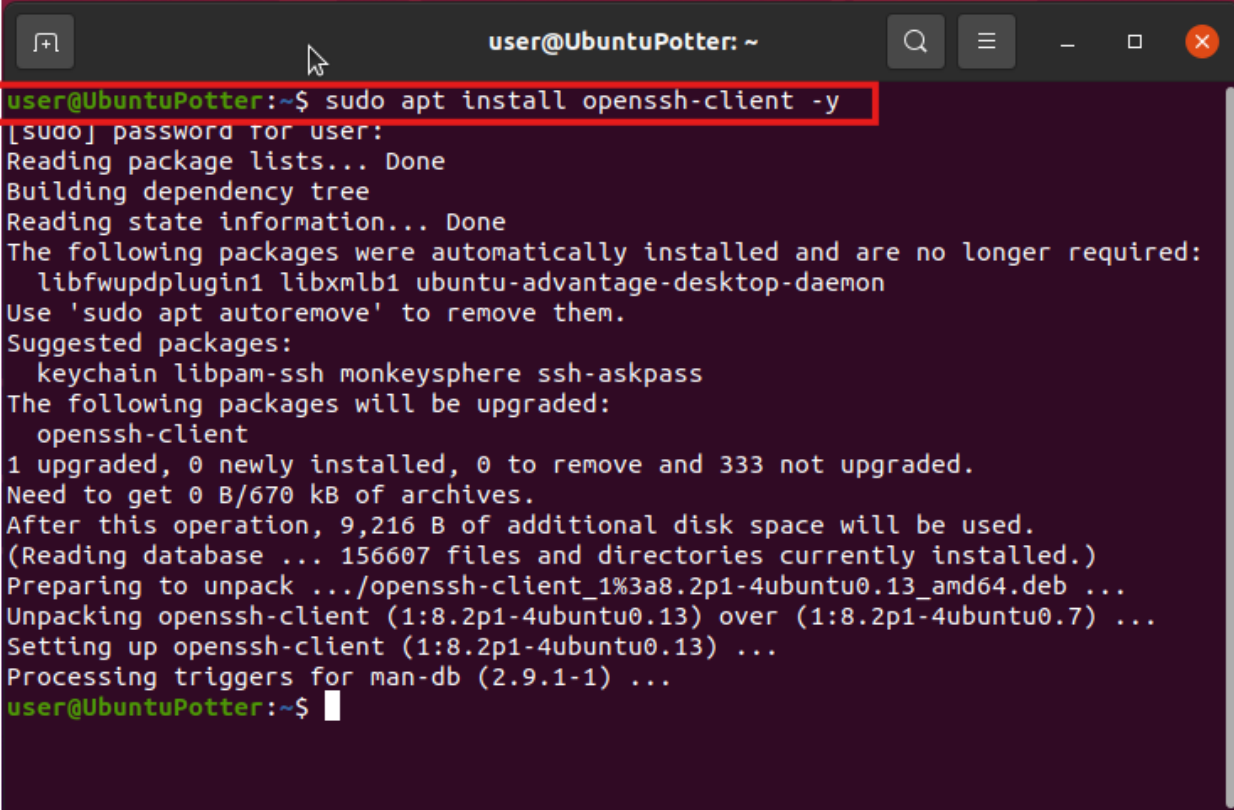
- **Compliance:** Evidence that an organization has undergone a thorough examination of its systems. It helps validate that the organization has implemented appropriate controls to protect its information systems and sensitive data.
- **Validation:** Verification of the effectiveness and adequacy of cybersecurity controls. It provides detailed information about the design, implementation, and operation of these controls, enabling reviewers to assess their reliability and identify any gaps or weaknesses.
- **Records Maintenance:** Historical record of cybersecurity audits conducted over time. It enables organizations to track their progress, identify trends, and evaluate the effectiveness actions taken. It also serves as reference for future audits and allows auditors to understand the current cybersecurity implemented and facilitates a more targeted approach to future cybersecurity updates and audits.
- **Decision-making Support:** Valuable insights and information that can support decision-making processes. It allows management to make informed decisions about allocating resources, prioritizing cybersecurity investments, and addressing identified risks and vulnerabilities.

Network Topology Diagram



SSH Install & Access

OpenSSH Install (Ubuntu)

A terminal window titled 'user@UbuntuPotter: ~' with standard Ubuntu window controls. The command 'sudo apt install openssh-client -y' is entered and highlighted with a red box. The terminal output shows the installation process, including package list reading, dependency tree building, and the successful installation of openssh-client. The prompt returns to 'user@UbuntuPotter:~\$' at the end.

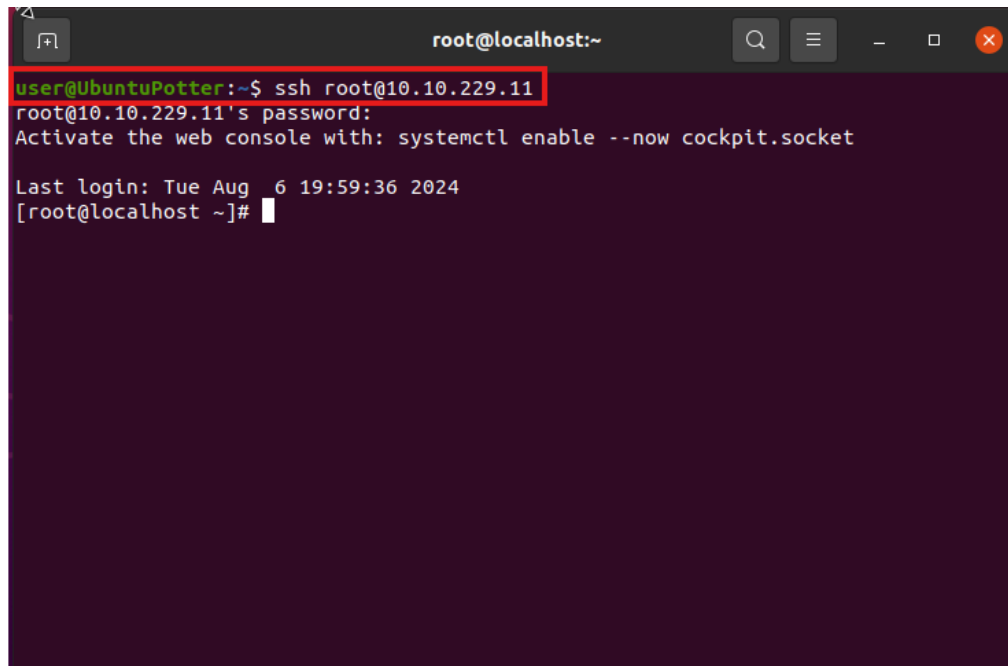
```
user@UbuntuPotter:~$ sudo apt install openssh-client -y
[sudo] password for user:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwupdplugin1 libxmlb1 ubuntu-advantage-desktop-daemon
Use 'sudo apt autoremove' to remove them.
Suggested packages:
  keychain libpam-ssh monkeysphere ssh-askpass
The following packages will be upgraded:
  openssh-client
1 upgraded, 0 newly installed, 0 to remove and 333 not upgraded.
Need to get 0 B/670 kB of archives.
After this operation, 9,216 B of additional disk space will be used.
(Reading database ... 156607 files and directories currently installed.)
Preparing to unpack .../openssh-client_1%3a8.2p1-4ubuntu0.13_amd64.deb ...
Unpacking openssh-client (1:8.2p1-4ubuntu0.13) over (1:8.2p1-4ubuntu0.7) ...
Setting up openssh-client (1:8.2p1-4ubuntu0.13) ...
Processing triggers for man-db (2.9.1-1) ...
user@UbuntuPotter:~$
```

To Install open SSH on Your Ubuntu Machine with Logged into our user Account. We then went to Terminal in the “Activities” Button in the upper right hand side of the Desktop. Once in the Terminal we entered the Command “**sudo apt install openssh-client -y**” you are then prompted to enter your password. Once entered correctly the system will install the package or confirm that the package is already installed.

Node.js Application (Ghost) on Docker

Update Rocky8-Docker Host Name

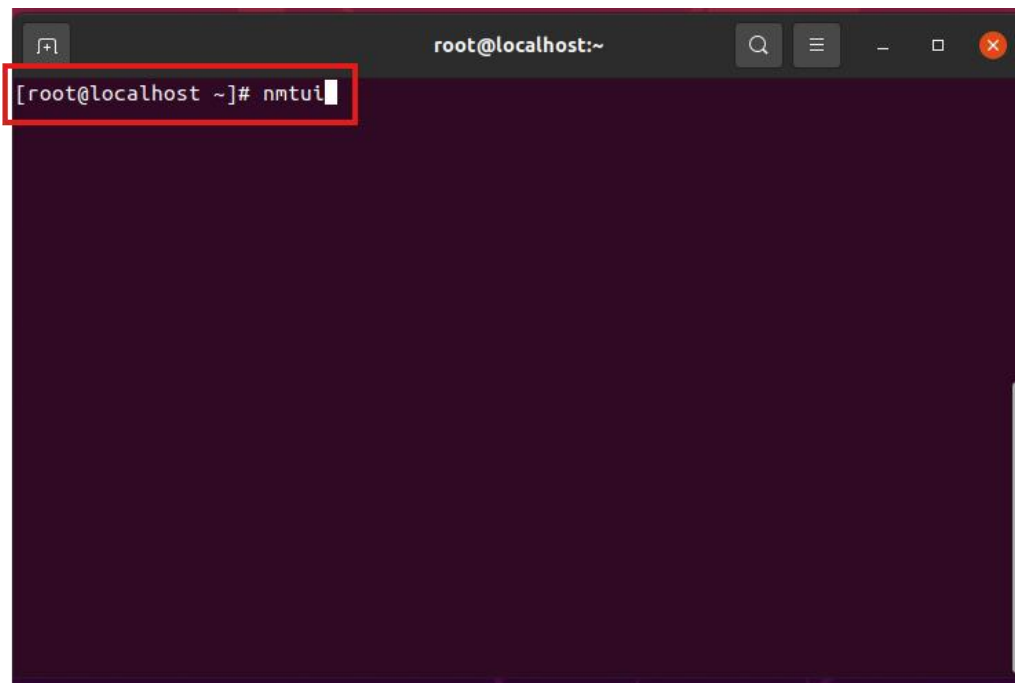
First We will SSH into The Rocky 8 Docker Host using the command “ssh root@10.10.229.11” Followed by the password:

A terminal window titled 'root@localhost:~' with search, menu, and window control icons. The prompt is 'user@UbuntuPotter:~\$'. The command 'ssh root@10.10.229.11' is entered and highlighted with a red box. The output shows the password prompt, a message to activate the web console, the last login time, and the root shell prompt '[root@localhost ~]#'.

```
user@UbuntuPotter:~$ ssh root@10.10.229.11
root@10.10.229.11's password:
Activate the web console with: systemctl enable --now cockpit.socket

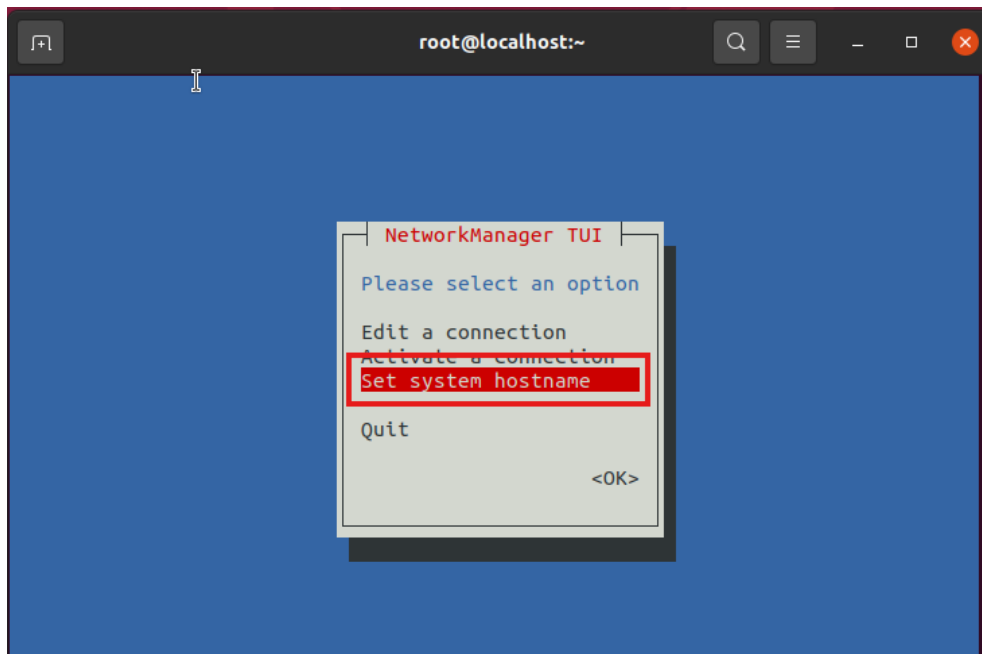
Last login: Tue Aug  6 19:59:36 2024
[root@localhost ~]#
```

We will then use the “nmtui” command to set the host name.

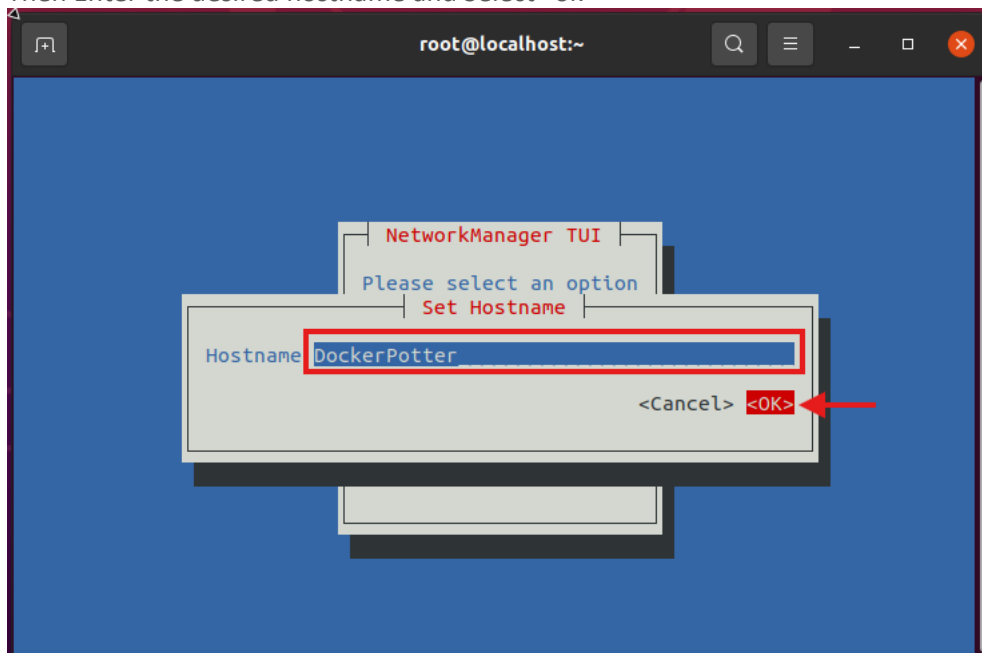
A terminal window titled 'root@localhost:~' with search, menu, and window control icons. The prompt is '[root@localhost ~]#'. The command 'nmtui' is entered and highlighted with a red box.

```
[root@localhost ~]# nmtui
```

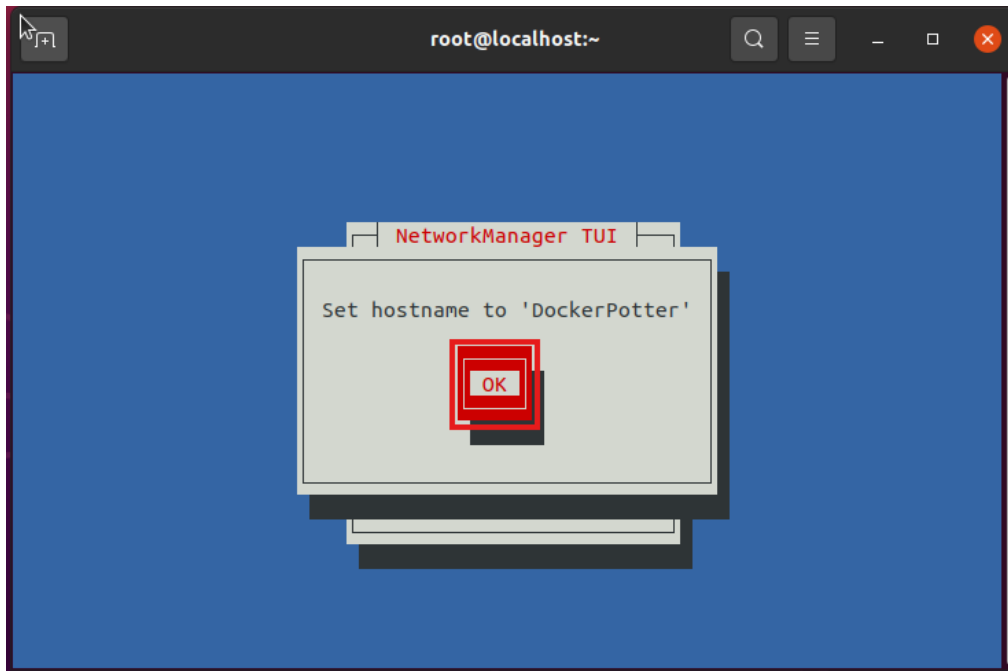
Select "Set system hostname"



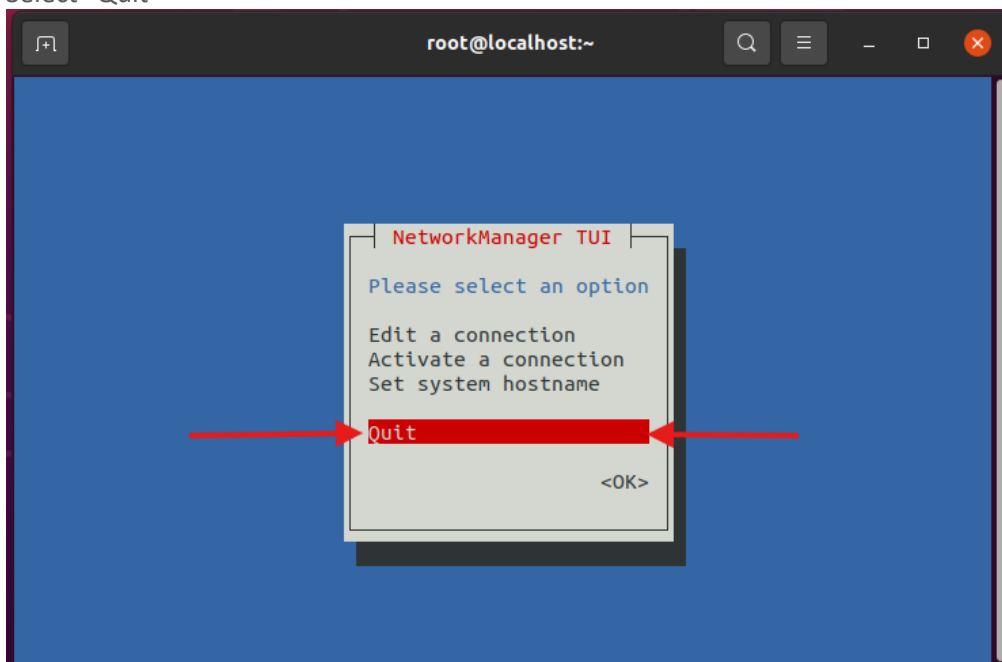
Then Enter the desired hostname and Select "ok"



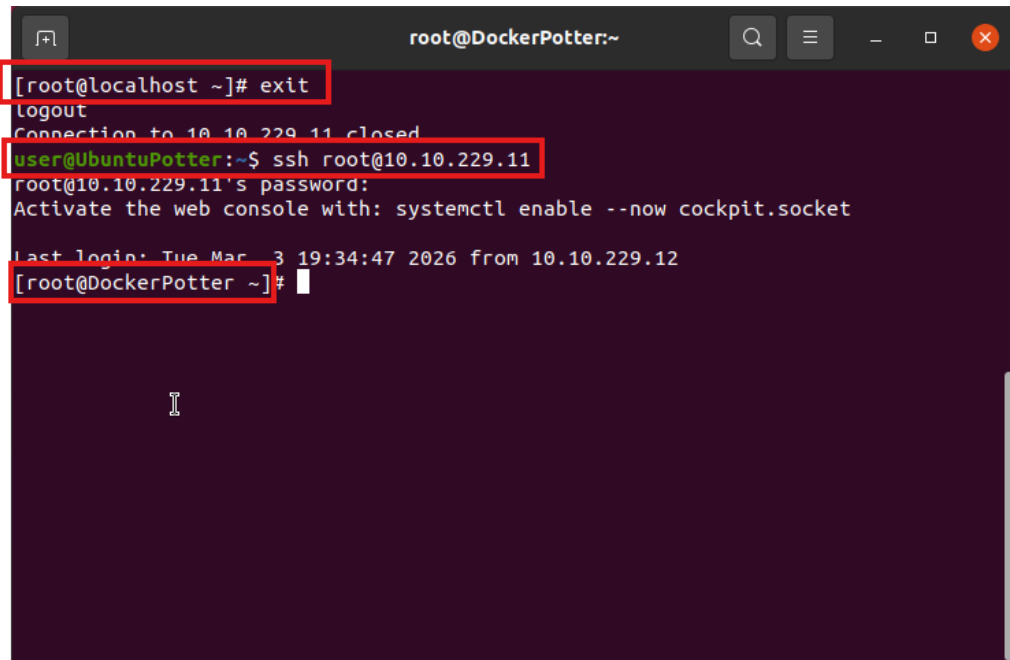
Select "OK"



Select "Quit"



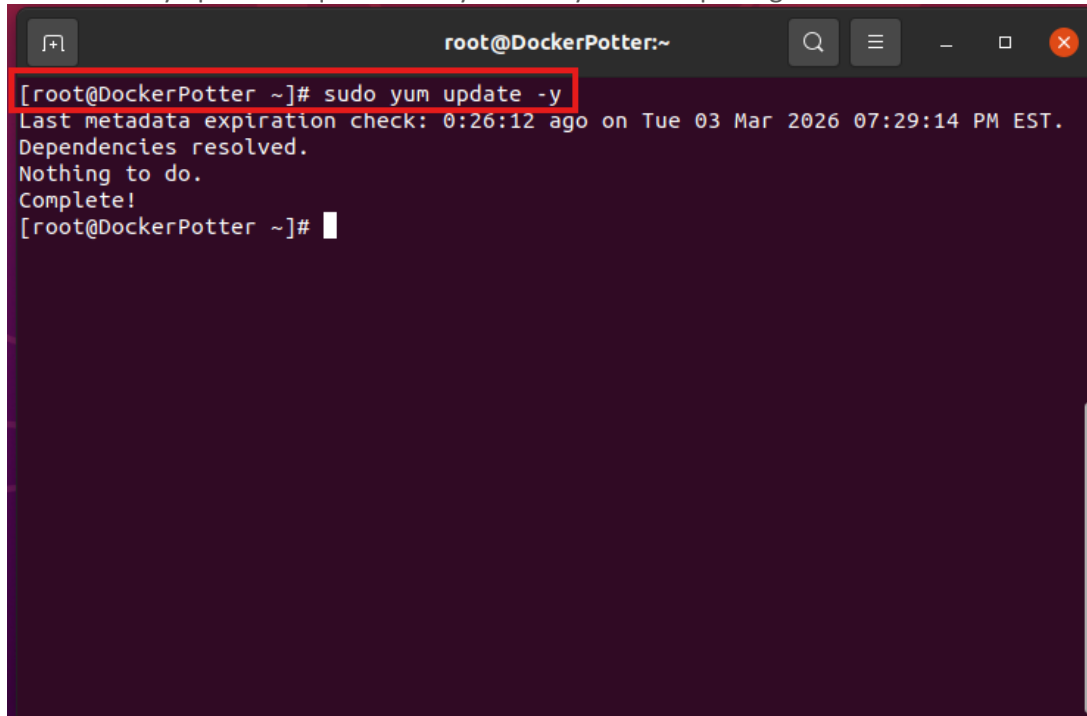
Use the command “exit” and Reconnect to see the new Host name.

A terminal window titled 'root@DockerPotter:~' with standard window controls. The terminal shows a sequence of commands and outputs: 1. '[root@localhost ~]# exit' is entered and highlighted with a red box. 2. The output 'logout' is shown. 3. 'Connection to 10.10.229.11 closed.' is shown. 4. '[user@UbuntuPotter:~\$ ssh root@10.10.229.11]' is entered and highlighted with a red box. 5. The prompt 'root@10.10.229.11's password:' is shown. 6. The instruction 'Activate the web console with: systemctl enable --now cockpit.socket' is shown. 7. The message 'Last login: Tue Mar 3 19:34:47 2026 from 10.10.229.12' is shown. 8. '[root@DockerPotter ~]#' is entered and highlighted with a red box, with a cursor following it. The terminal has a dark purple background and a vertical scrollbar on the right.

```
root@DockerPotter:~  
[root@localhost ~]# exit  
logout  
Connection to 10.10.229.11 closed.  
[user@UbuntuPotter:~$ ssh root@10.10.229.11  
root@10.10.229.11's password:  
Activate the web console with: systemctl enable --now cockpit.socket  
Last login: Tue Mar 3 19:34:47 2026 from 10.10.229.12  
[root@DockerPotter ~]#
```

Update Rocky8-Docker

To run the updates on our Rocky8-Docker System we simply input the command “**sudo yum update -y**” This will install any updates required for any currently installed packages:

A terminal window titled 'root@DockerPotter:~' with a dark background. The command '[root@DockerPotter ~]# sudo yum update -y' is entered and highlighted with a red box. The output shows the system is up to date.

```
[root@DockerPotter ~]# sudo yum update -y
Last metadata expiration check: 0:26:12 ago on Tue 03 Mar 2026 07:29:14 PM EST.
Dependencies resolved.
Nothing to do.
Complete!
[root@DockerPotter ~]#
```

Install EPEL Packages

We now want to install EPEL packages on the docker system so to do this we will use the command “**sudo yum install epel-release -y**” this will install EPEL on the system.

```
root@DockerPotter:~  
[root@DockerPotter ~]# sudo yum install epel-release -y  
Last metadata expiration check: 0:37:10 ago on Tue 03 Mar 2026 07:29:14 PM EST.  
Dependencies resolved.  
=====
```

Package	Architecture	Version	Repository	Size
Installing: epel-release	noarch	8-22.el8	extras	25 k

```
=====
```

Transaction Summary

=====

Install 1 Package

Total download size: 25 k
Installed size: 34 k
Downloading Packages:

Package	Download Size	Progress	Time
epel-release-8-22.el8.noarch.rpm	25 kB	174 kB/s	00:00

Total	Download Size	Progress	Time
	25 kB	85 kB/s	00:00

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction

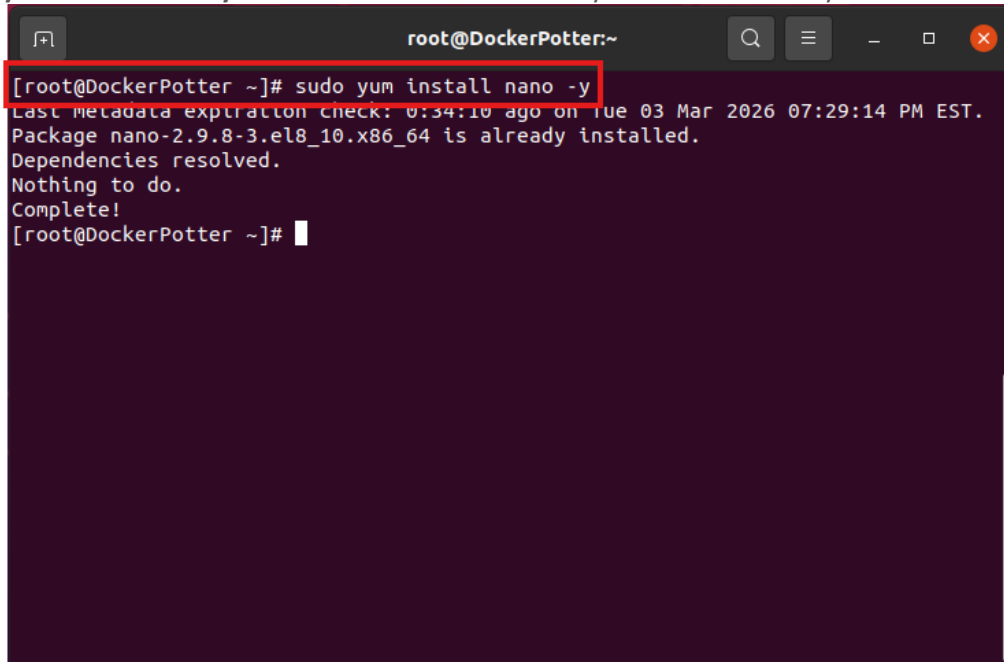
Step	Package	Progress
Preparing	:	1/1
Installing	: epel-release-8-22.el8.noarch	1/1
Running scriptlet:	epel-release-8-22.el8.noarch	1/1

Many EPEL packages require the CodeReady Builder (CRB) repository.
It is recommended that you run /usr/bin/crb enable to enable the CRB repository.

Step	Package	Progress
Verifying	: epel-release-8-22.el8.noarch	1/1

Install Nano Editor

We now want to ensure nano is installed on the docker system so to do this we will use the command “**sudo yum install nano -y**” this will install nano on the system. If it is already installed it will show like this:

A terminal window titled 'root@DockerPotter:~' with standard window controls. The command '[root@DockerPotter ~]# sudo yum install nano -y' is entered and highlighted with a red box. The output shows that the package is already installed.

```
[root@DockerPotter ~]# sudo yum install nano -y
Last metadata expiration check: 0:34:10 ago on Tue 03 Mar 2026 07:29:14 PM EST.
Package nano-2.9.8-3.el8_10.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@DockerPotter ~]#
```

Docker CE

Set up stable repository

We now want to setup the stable repository for Docker. In order to do this we first need to install the yum-utils package manager using the command “**sudo yum install -y yum-utils**”:

```
root@DockerPotter:~  
[root@DockerPotter ~]# sudo yum install -y yum-utils  
Extra Packages for Enterprise Linux 8 - x86_64      13 MB/s | 14 MB    00:01  
Last metadata expiration check: 0:00:06 ago on Tue 03 Mar 2026 08:14:39 PM EST.  
Dependencies resolved.  
=====
```

Package	Architecture	Version	Repository	Size
Installing: yum-utils	noarch	4.0.21-25.el8	baseos	75 k

```
=====
```

Transaction Summary
=====

Install 1 Package

Total download size: 75 k
Installed size: 23 k
Downloading Packages:
yum-utils-4.0.21-25.el8.noarch.rpm 288 kB/s | 75 kB 00:00

Total 165 kB/s | 75 kB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
 Preparing : 1/1
 Installing : yum-utils-4.0.21-25.el8.noarch 1/1
 Running scriptlet: yum-utils-4.0.21-25.el8.noarch 1/1
 Verifying : yum-utils-4.0.21-25.el8.noarch 1/1
Installed:

Once this is installed, we want to add the Repo using this command “**sudo yum-config-manager --add-repo <https://download.docker.com/linux/centos/docker-ce.repo>**” you should get an output like below:

```
root@DockerPotter:~  
[root@DockerPotter ~]# sudo yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo  
Adding repo from: https://download.docker.com/linux/centos/docker-ce.repo  
[root@DockerPotter ~]#
```

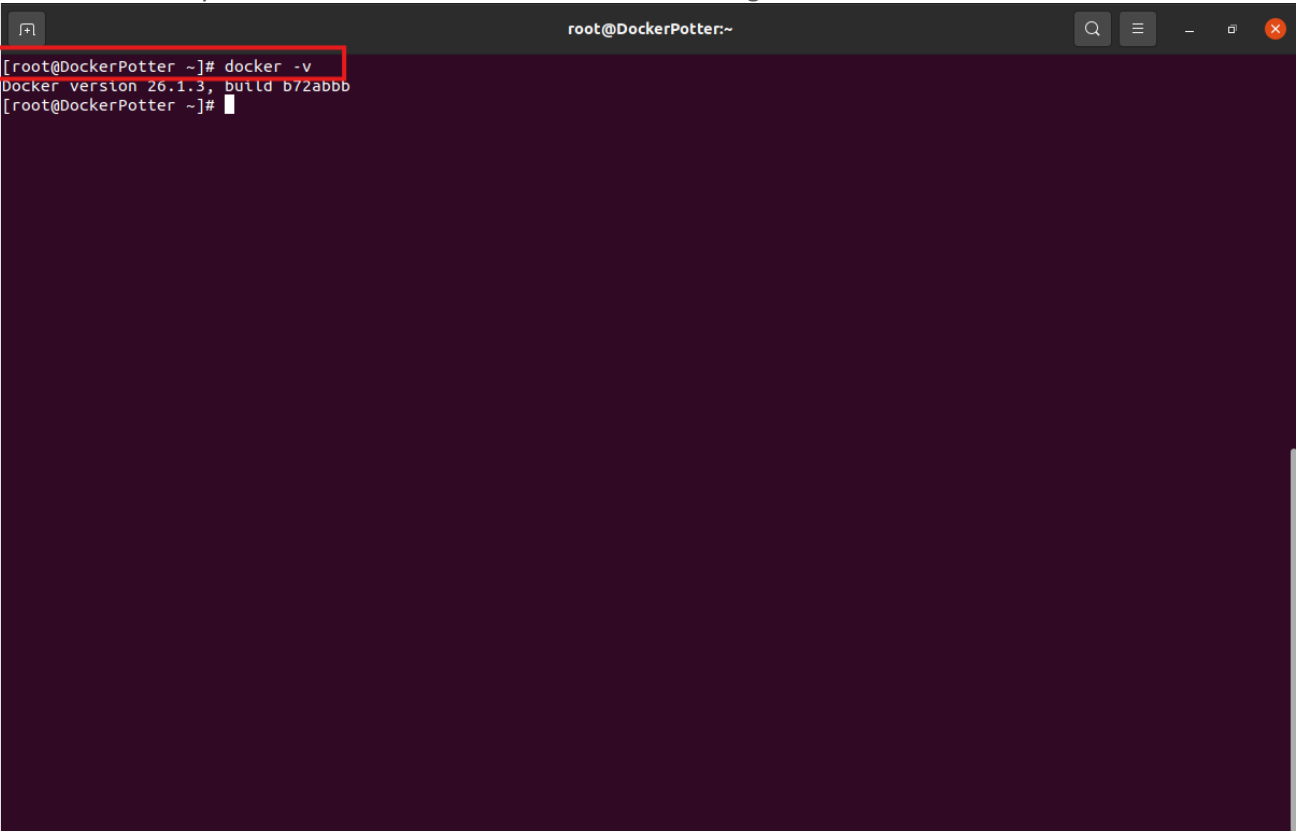
Install Docker CE

Now we will install Docker CE, We will use the following command to download docker and some of its dependencies “**sudo yum install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin**”:

```
root@DockerPotter:~# sudo yum install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
docker-ce-stable | x86_64 | 301 kB/s | 80 kB | 00:00
Dependencies resolved.
=====
Package Arch Version Repository Size
=====
Installing:
containerd.io x86_64 1.6.32-3.1.el8 docker-ce-stable 35 M
docker-buildx-plugin x86_64 0.14.0-1.el8 docker-ce-stable 14 M
docker-ce x86_64 3:26.1.3-1.el8 docker-ce-stable 27 M
docker-ce-cli x86_64 1:26.1.3-1.el8 docker-ce-stable 7.8 M
docker-compose-plugin x86_64 2.27.0-1.el8 docker-ce-stable 13 M
Installing dependencies:
container-selinux noarch 2:2.229.0-2.module+el8.10.0+40047+0c0a73f4 appstream 70 k
fuse-common x86_64 3.3.0-19.el8 baseos 21 k
fuse-overlayfs x86_64 1.13-1.module+el8.10.0+40047+0c0a73f4 appstream 69 k
fuse3 x86_64 3.3.0-19.el8 baseos 54 k
fuse3-libs x86_64 3.3.0-19.el8 baseos 95 k
libcgroup x86_64 0.41-19.el8 baseos 69 k
libslirp x86_64 4.4.0-2.module+el8.10.0+40047+0c0a73f4 appstream 69 k
slirp4netns x86_64 1.2.3-1.module+el8.10.0+40047+0c0a73f4 appstream 55 k
Installing weak dependencies:
docker-ce-rootless-extras x86_64 26.1.3-1.el8 docker-ce-stable 5.0 M
Enabling module streams:
container-tools rhel8
Transaction Summary
=====
Install 14 Packages
Total download size: 103 M
Installed size: 390 M
Downloading Packages:
(1/14): container-selinux-2.229.0-2.module+el8.10.0+40047+ 131 kB/s | 70 kB 00:00
```

Verify Docker version

We can then verify the version of Docker that was installed using the “**docker -v**” Command:

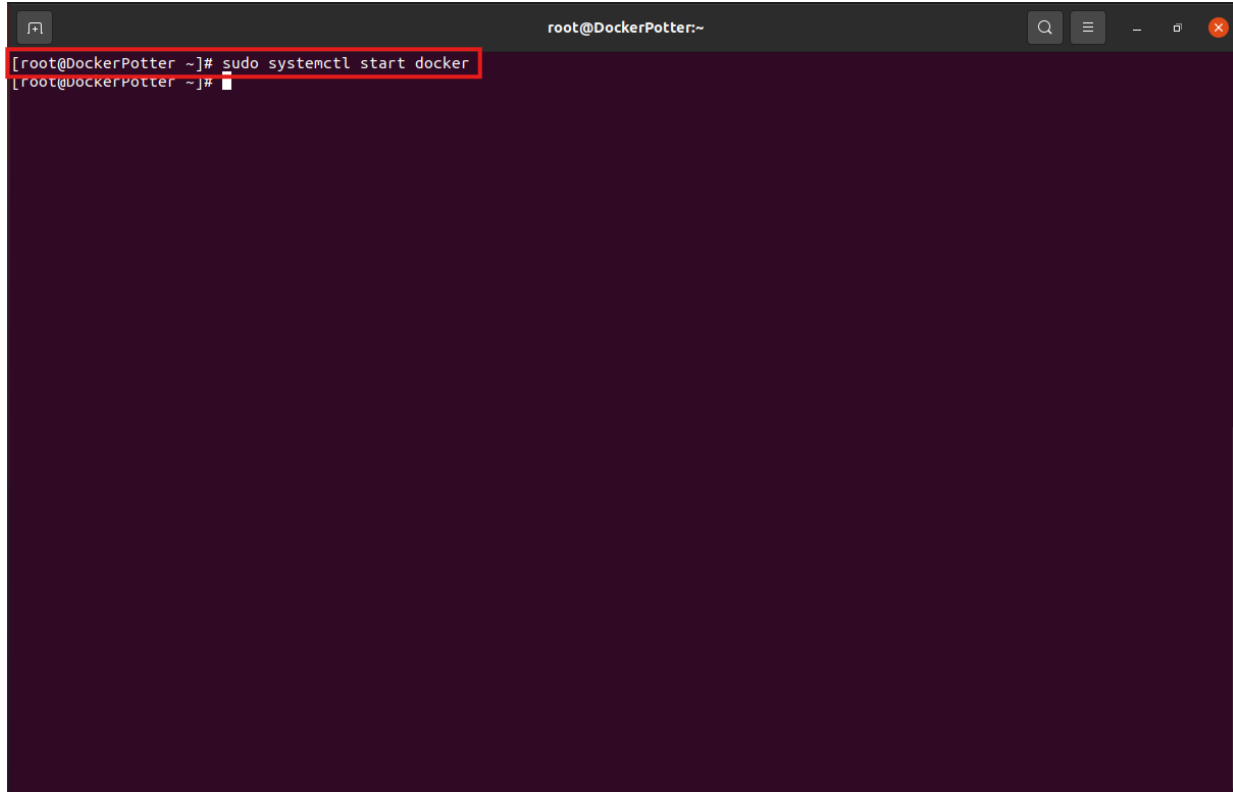
A terminal window titled 'root@DockerPotter:~' with search, menu, and window control icons in the title bar. The terminal shows the command 'docker -v' being executed, which returns 'Docker version 26.1.3, build b72abbb'. The prompt returns to '[root@DockerPotter ~]#'.

```
[root@DockerPotter ~]# docker -v
Docker version 26.1.3, build b72abbb
[root@DockerPotter ~]#
```

Initialize Docker

Start Docker

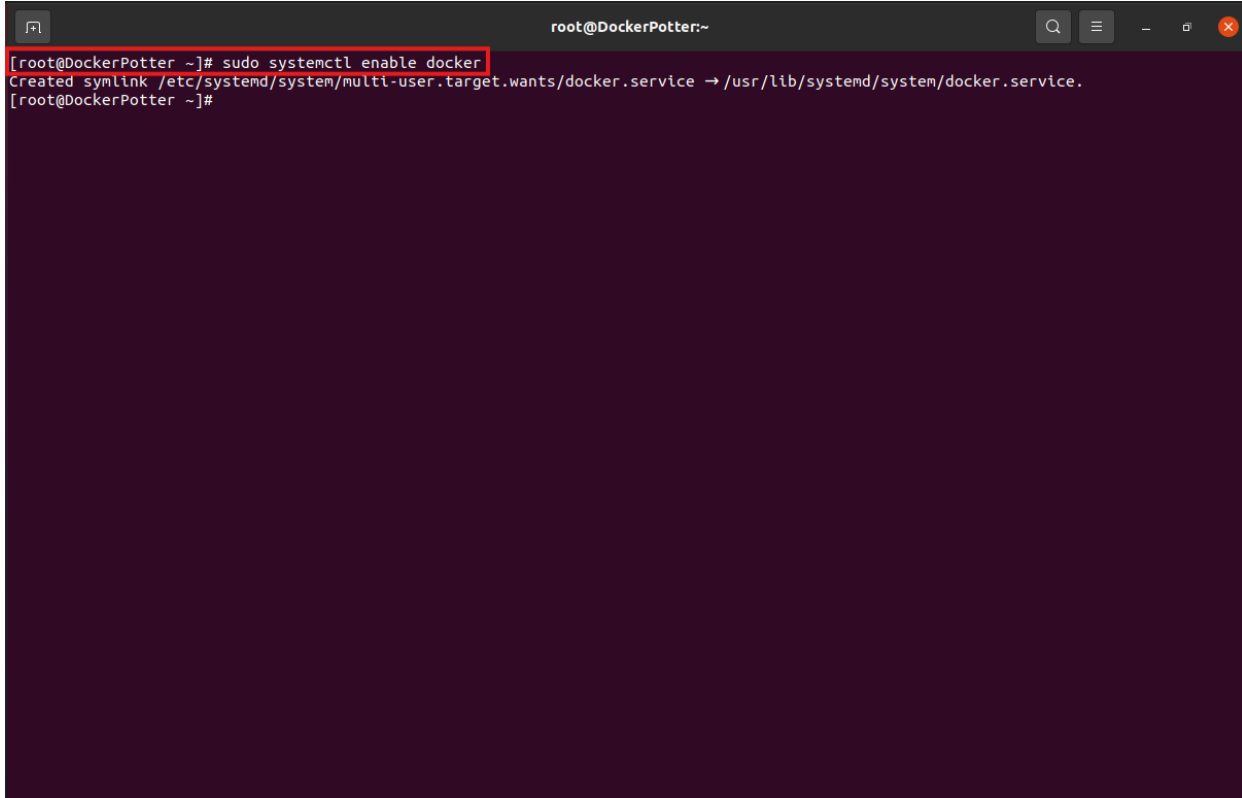
We now need to start the Docker service using the command “**sudo systemctl start docker**” Note: There is no output from this command.

A terminal window titled 'root@DockerPotter:~' with a dark background. The command '[root@DockerPotter ~]# sudo systemctl start docker' is entered and highlighted with a red box. The prompt changes to '[root@DockerPotter ~]#' on the next line, indicating the command has been executed. The rest of the terminal is empty.

```
root@DockerPotter:~  
[root@DockerPotter ~]# sudo systemctl start docker  
[root@DockerPotter ~]#
```


Enable Docker

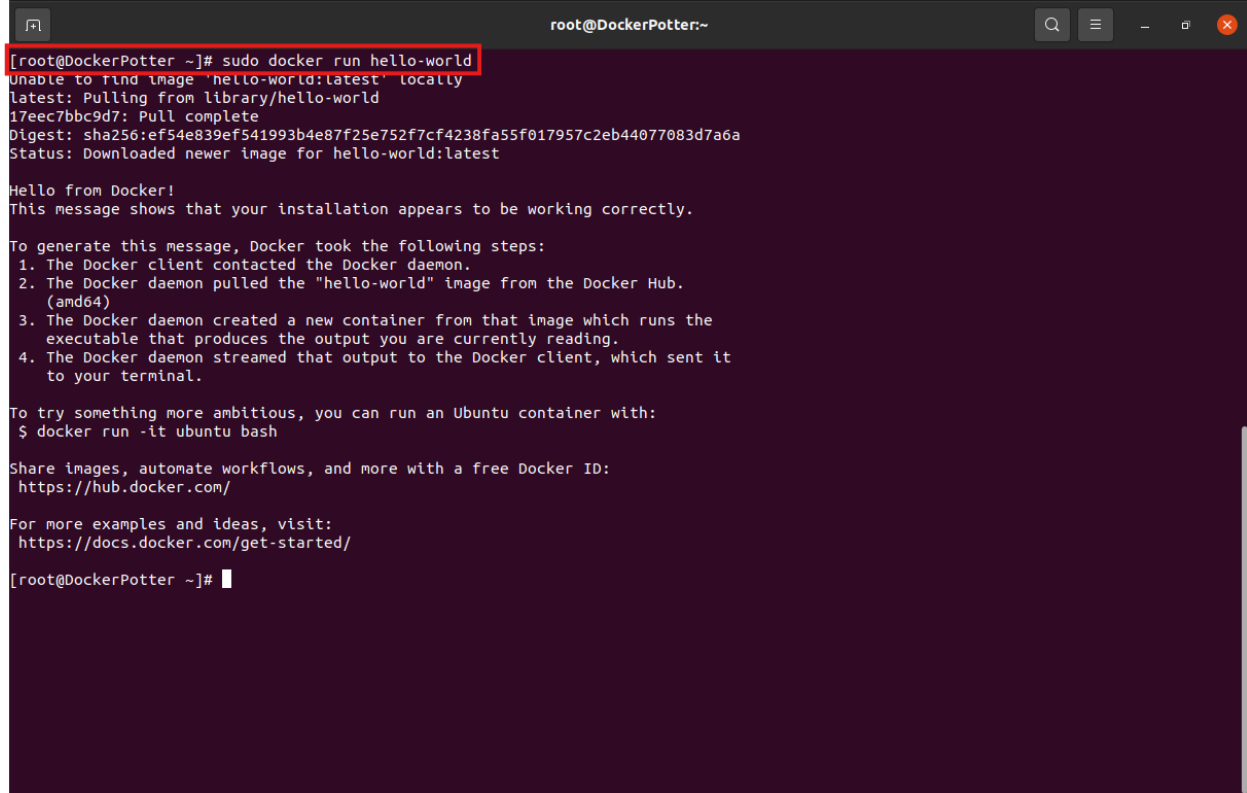
After that, we need to enable Docker, This will automatically start docker when the system boots up, the command for this is “**sudo systemctl enable docker**”



```
root@DockerPotter:~  
[root@DockerPotter ~]# sudo systemctl enable docker  
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service -> /usr/lib/systemd/system/docker.service.  
[root@DockerPotter ~]#
```

Test Docker (hello-world)

We will now test that our Docker install is working correctly by using the command “**sudo docker run hello-world**”. This will run a test container it will automatically retrieve from dock and display the below message:

A terminal window titled 'root@DockerPotter:~' with standard window controls. The command '[root@DockerPotter ~]# sudo docker run hello-world' is entered and highlighted with a red box. The output shows the Docker client pulling the 'hello-world:latest' image from the Docker Hub, displaying the image ID '17eec7bbc9d7', the digest 'sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a', and the status 'Downloaded newer image for hello-world:latest'. The container then prints 'Hello from Docker!' and a message confirming the installation. It lists four steps: 1. Docker client contacted the daemon, 2. Docker daemon pulled the 'hello-world' image, 3. Docker daemon created a new container, and 4. Docker daemon streamed output to the client. It also provides a link to run an Ubuntu container and a link to Docker Hub for more examples.

```
[root@DockerPotter ~]# sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

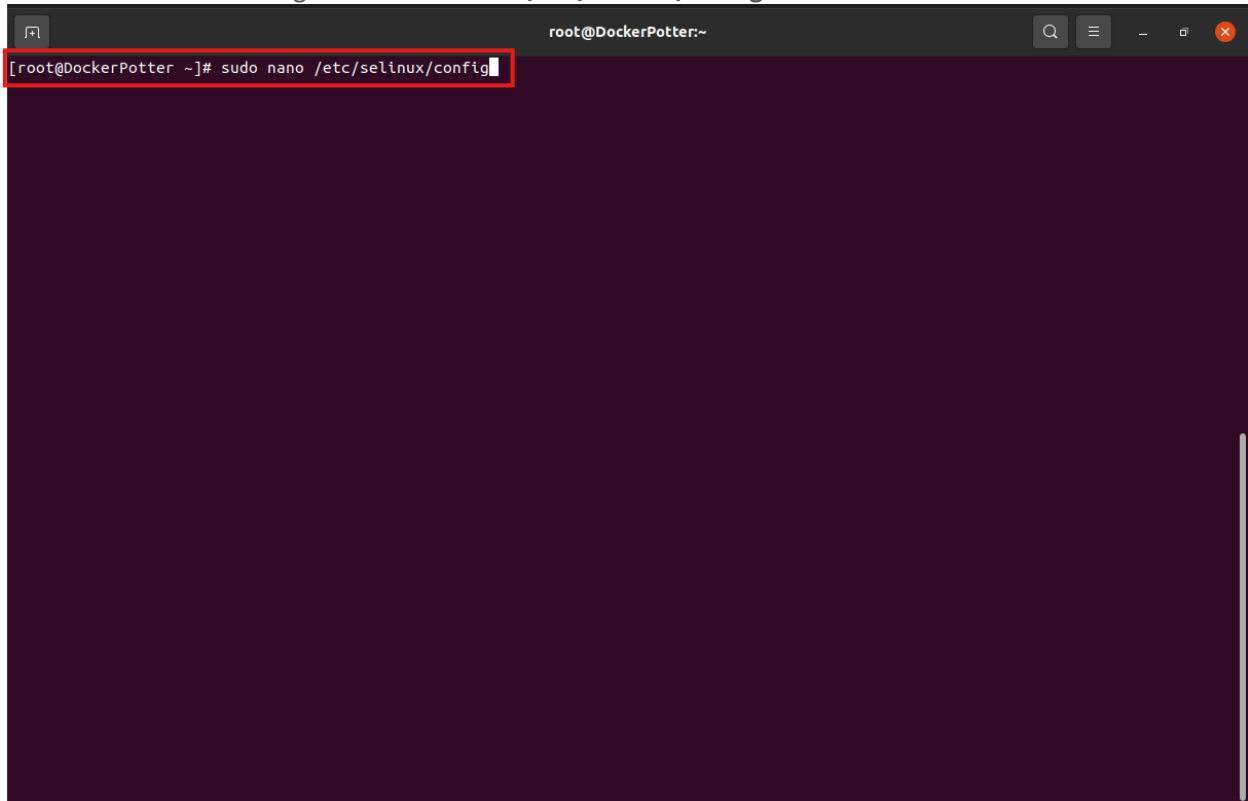
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

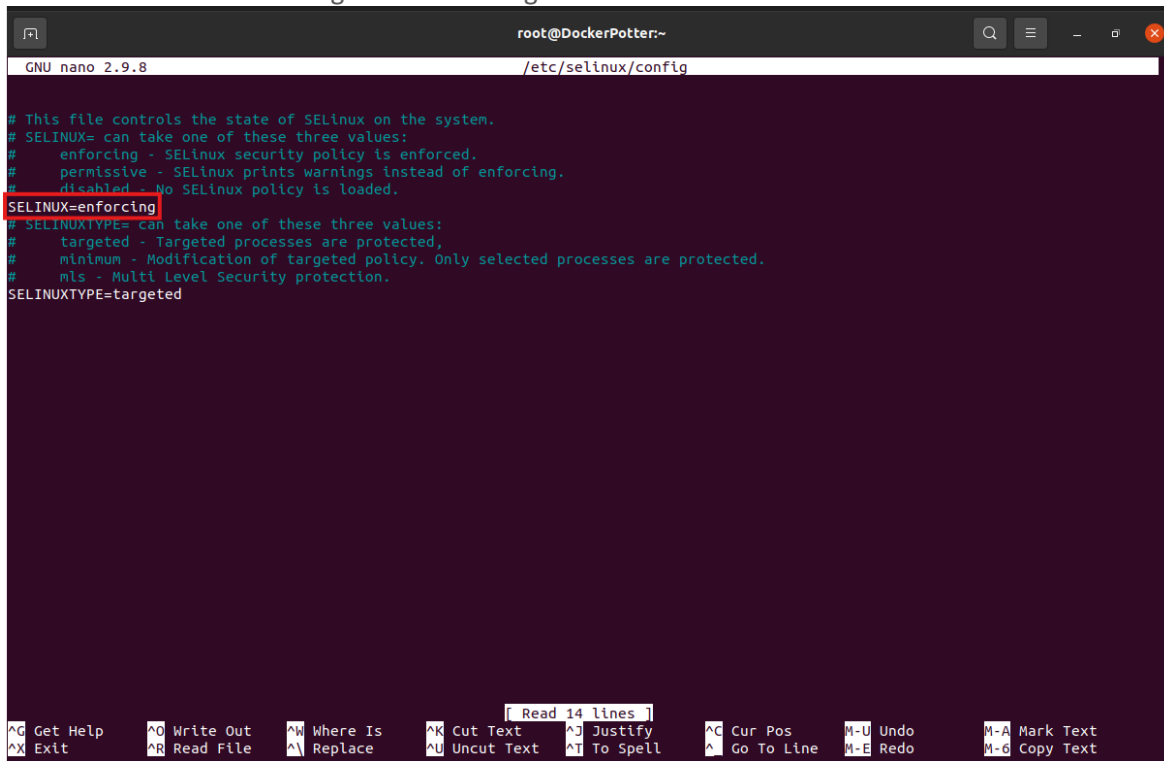
[root@DockerPotter ~]#
```

Disable SELinux (/etc/selinux/config)

We will now be disabling SELinux in this environment so that we don't need to login and disable it every time we want to make a change in the future. I can only recommend this in a closed environment. The command to edit the SELinux Config file is “**sudo nano /etc/selinux/config**”

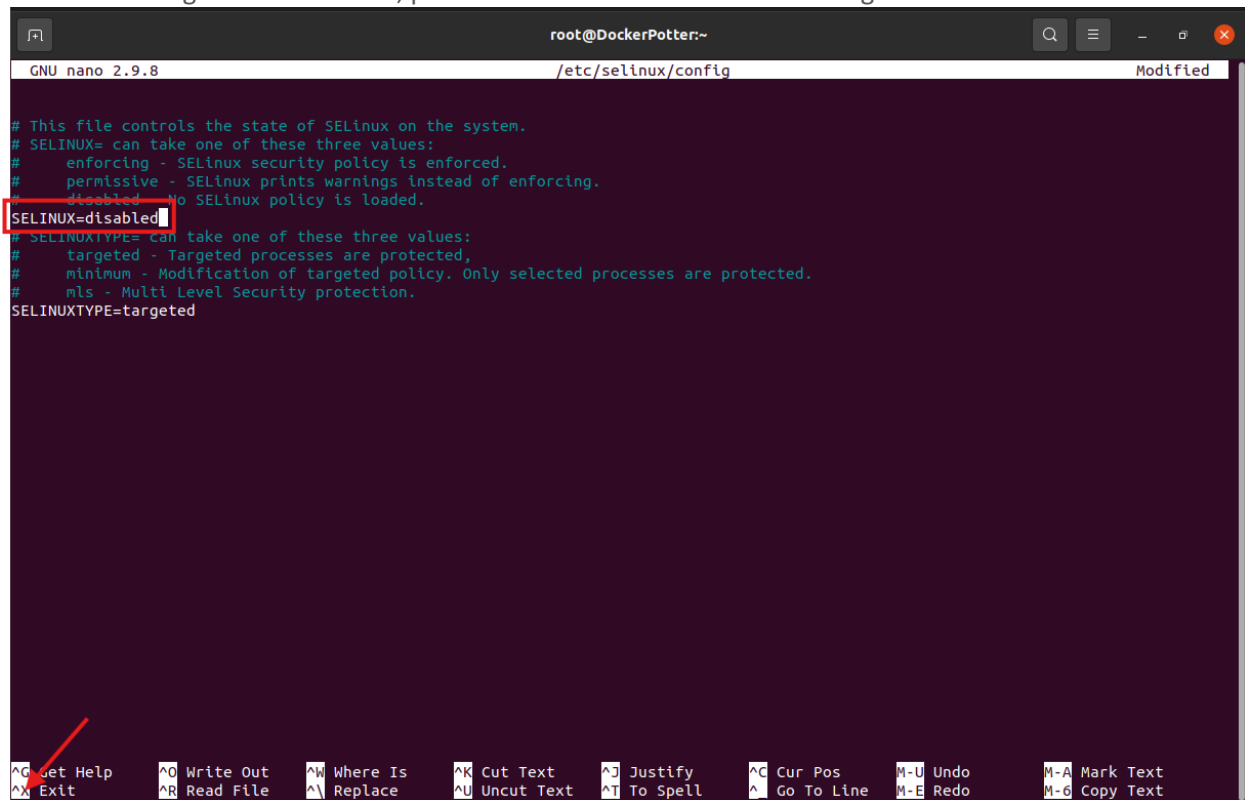
A terminal window titled 'root@DockerPotter:~' with a dark background. The command '[root@DockerPotter ~]# sudo nano /etc/selinux/config' is entered at the prompt and is highlighted with a red rectangular box. The rest of the terminal area is empty and dark.

From here we want to change the “enforcing” to “disabled”



```
root@DockerPotter:~  
GNU nano 2.9.8 /etc/selinux/config  
  
# This file controls the state of SELinux on the system.  
# SELINUX= can take one of these three values:  
#   enforcing - SELinux security policy is enforced.  
#   permissive - SELinux prints warnings instead of enforcing.  
#   disabled - No SELinux policy is loaded.  
SELINUX=enforcing  
# SELINUXTYPE= can take one of these three values:  
#   targeted - Targeted processes are protected,  
#   minimum - Modification of targeted policy. Only selected processes are protected.  
#   mls - Multi Level Security protection.  
SELINUXTYPE=targeted  
  
^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify      ^C Cur Pos      M-U Undo        M-A Mark Text  
^X Exit          ^R Read File    ^\ Replace      ^U Uncut Text   ^T To Spell     ^_ Go To Line    M-E Redo        M-6 Copy Text
```

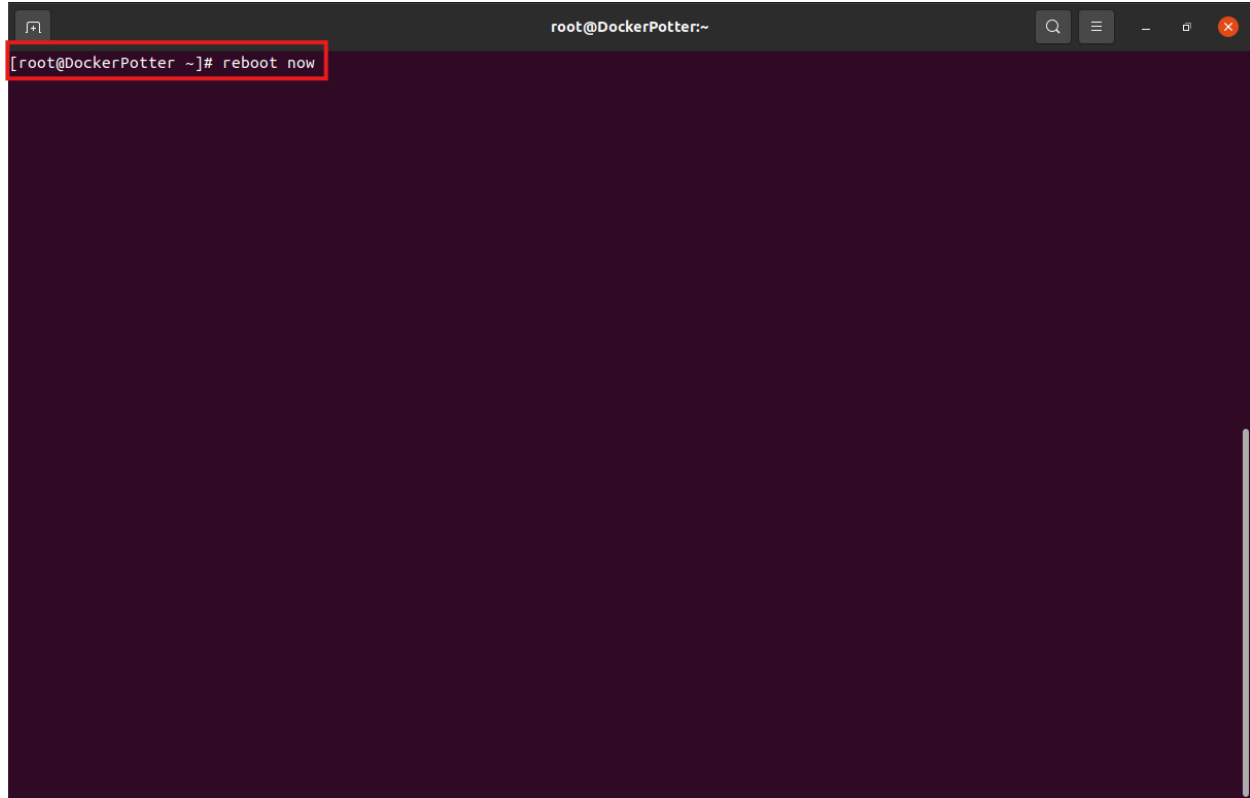
Make the Change and Press CtrlX, press Y and Hit Enter to save the changes.



```
root@DockerPotter:~  
GNU nano 2.9.8 /etc/selinux/config Modified  
  
# This file controls the state of SELinux on the system.  
# SELINUX= can take one of these three values:  
#   enforcing - SELinux security policy is enforced.  
#   permissive - SELinux prints warnings instead of enforcing.  
#   disabled - No SELinux policy is loaded.  
SELINUX=disabled  
# SELINUXTYPE= can take one of these three values:  
#   targeted - Targeted processes are protected,  
#   minimum - Modification of targeted policy. Only selected processes are protected.  
#   mls - Multi Level Security protection.  
SELINUXTYPE=targeted  
  
^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify      ^C Cur Pos      M-U Undo        M-A Mark Text  
^X Exit          ^R Read File    ^\ Replace      ^U Uncut Text   ^T To Spell     ^_ Go To Line    M-E Redo        M-6 Copy Text
```

Reboot VM

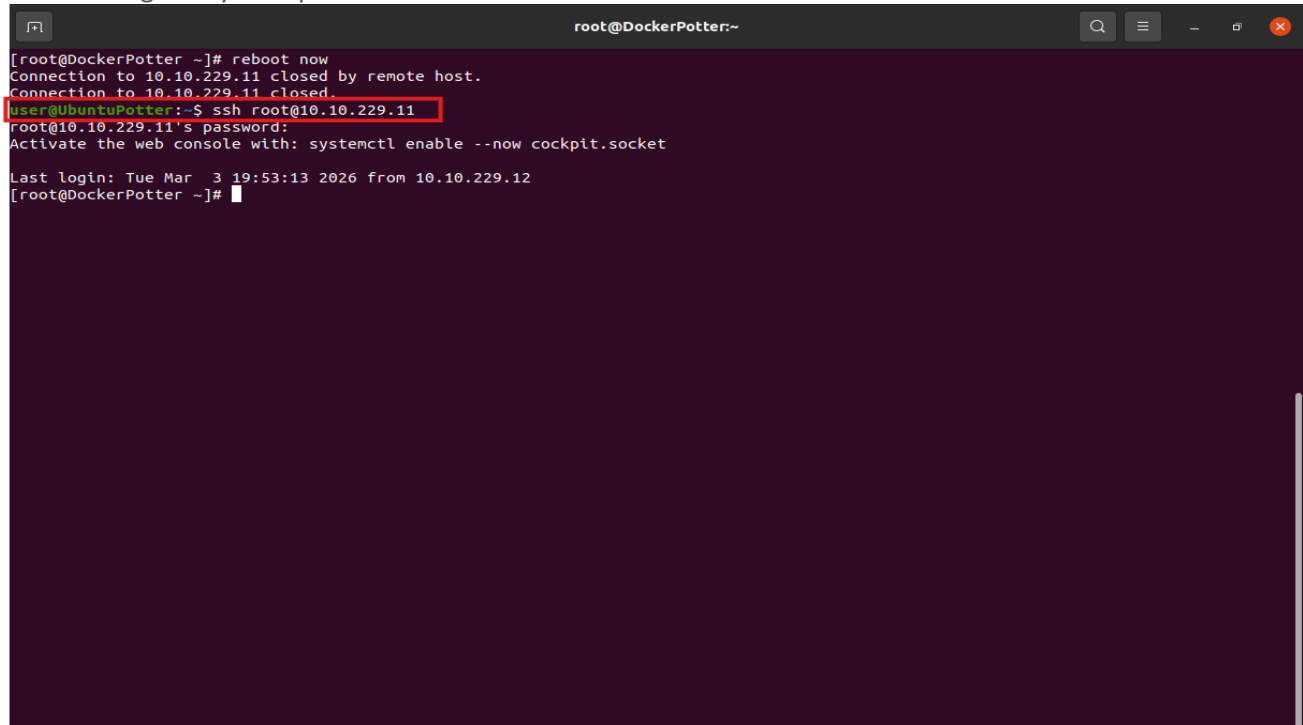
We now want to reboot to make sure the SELinux changes save correctly, to do this we will use the “**Reboot now**” command:

A terminal window titled 'root@DockerPotter:~' with a dark background. The command '[root@DockerPotter ~]# reboot now' is entered at the prompt and is highlighted with a red rectangular box. The terminal window includes standard window controls (search, menu, zoom, close) in the top right corner.

```
[root@DockerPotter ~]# reboot now
```

Confirm SELinux Status

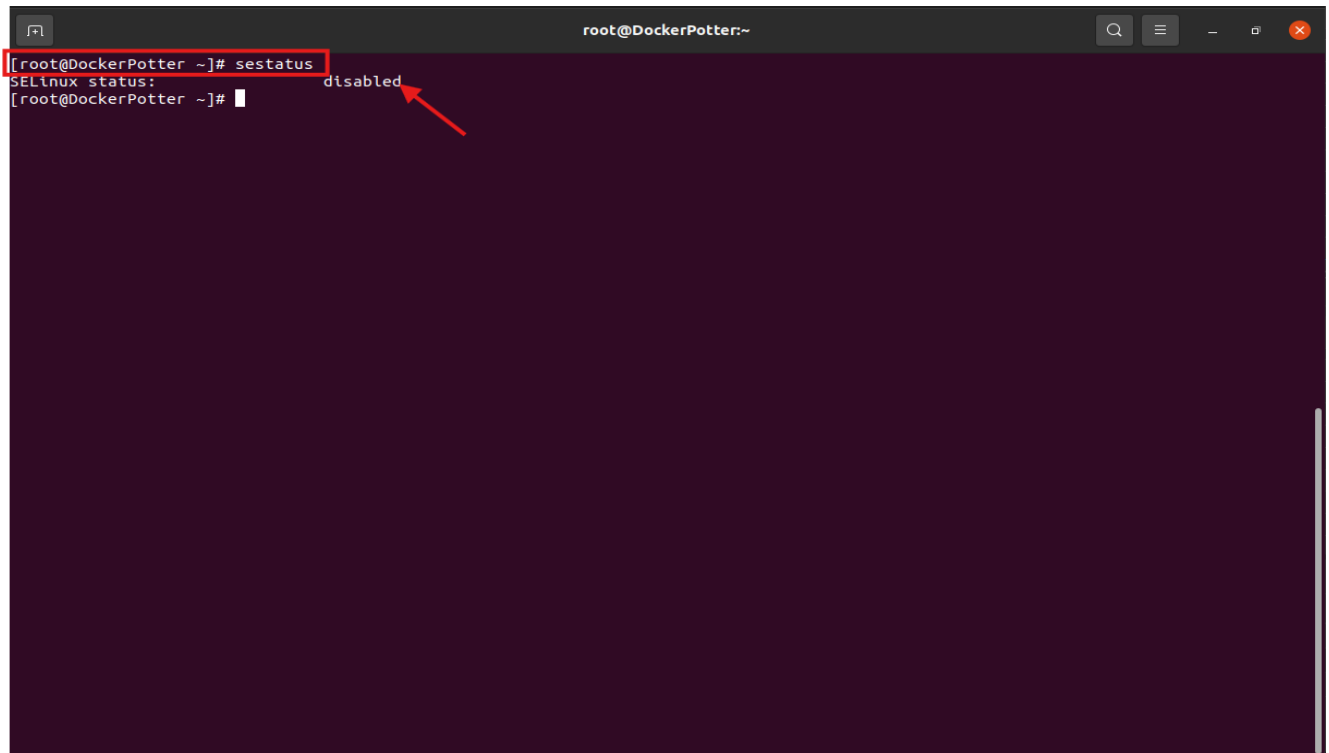
After Waiting a moment for the system to reboot with will reconnect using the “ssh root@10.10.229.11” and entering the system password to reconnect:

A terminal window titled 'root@DockerPotter:~' showing a sequence of commands and outputs. The user enters 'reboot now', followed by two lines indicating the connection to 10.10.229.11 was closed by the remote host. Then, the user enters 'ssh root@10.10.229.11', which is highlighted with a red box. The terminal shows the password prompt and the activation of Cockpit. Finally, it shows the last login information and returns to the root prompt.

```
[root@DockerPotter ~]# reboot now
Connection to 10.10.229.11 closed by remote host.
Connection to 10.10.229.11 closed.
user@UbuntuPotter:~$ ssh root@10.10.229.11
root@10.10.229.11's password:
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Tue Mar  3 19:53:13 2026 from 10.10.229.12
[root@DockerPotter ~]#
```

And to confirm That the SELinux is disable we will use the command “sestatus” the output should look like this:

A terminal window titled 'root@DockerPotter:~' showing the command 'sestatus' being executed, which is highlighted with a red box. The output shows 'SELinux status: disabled', with a red arrow pointing to the word 'disabled'. The prompt returns to the root user.

```
[root@DockerPotter ~]# sestatus
SELinux status: disabled
[root@DockerPotter ~]#
```

Install Ghost Docker Container

We will now create, start and name a new ghost container to our docker setup. The command for this is “**docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.11:3001 ghost**” you should get an output that looks like this:

```
root@DockerPotter:~  
[root@DockerPotter ~]# docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.11:3001 ghost  
Unable to find image 'ghost:latest' locally  
latest: Pulling from library/ghost  
84a2afebaf4d: Pull complete  
ef9116903129: Pull complete  
5aad6122b53c: Pull complete  
136d84623d33: Pull complete  
f7c136321f37: Pull complete  
8c8da35162bf: Pull complete  
320e33304965: Pull complete  
1fd7318e92be: Pull complete  
4f4fb700ef54: Pull complete  
22513f68cb30: Pull complete  
Digest: sha256:5bd610ad07ad35ce39247c7055c9d6c47ff25f40631453fc8f7311042f362c36  
Status: Downloaded newer image for ghost:latest  
03159d327ed1f7915cbaeab319f82a56f7838fcd9b119f14241789c5ef0eb53  
[root@DockerPotter ~]#
```

Check for Ghost Container (docker ps)

Ghost Container ID 03159d327ed1

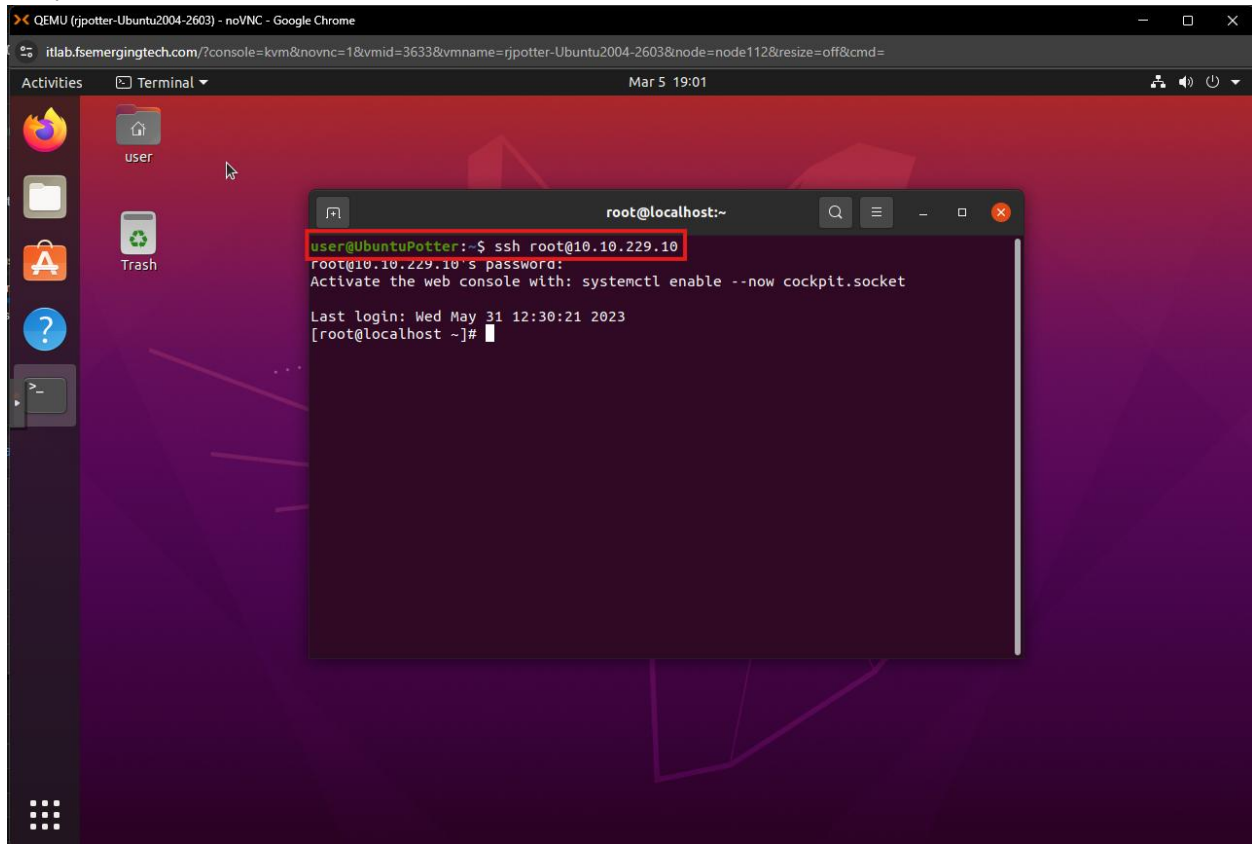
We now want to see our new container and confirm the id of it, in order to do this we will use the following command “**docker ps -a**” this command lists all containers.

```
root@DockerPotter:~  
[root@DockerPotter ~]# docker ps -a  
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS      PORTS      NAMES  
03159d327ed1   ghost    "docker-entrypoint.s..." 2 minutes ago  Exited (2) 2 minutes ago           ghost  
34d73731a0b1   hello-world  "/hello"                 33 minutes ago  Exited (0) 33 minutes ago           kind_agness1  
[root@DockerPotter ~]#
```

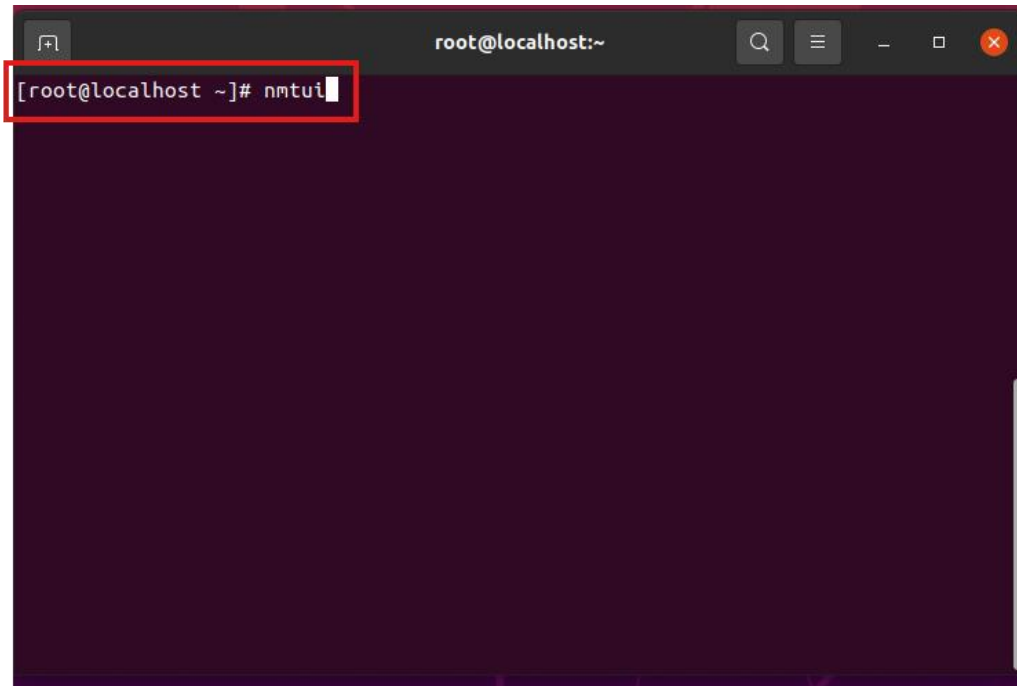

NginX Reverse Proxy

Update Rocky8-Nginx Host Name

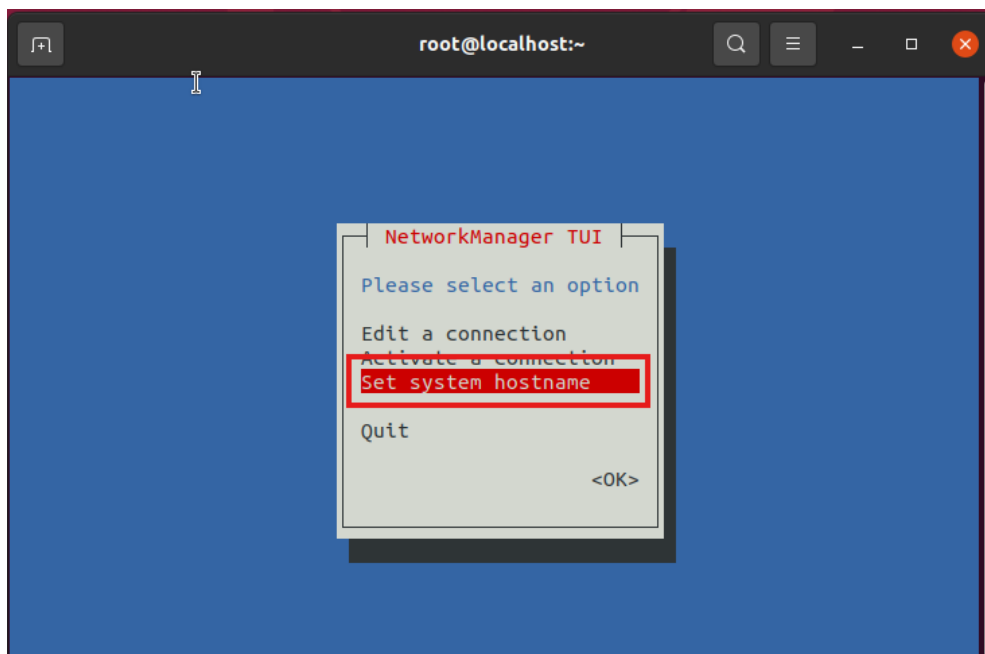
First We will SSH into The Rocky 8 NginX Host using the command “**ssh root@10.10.229.10**” Followed by the password:



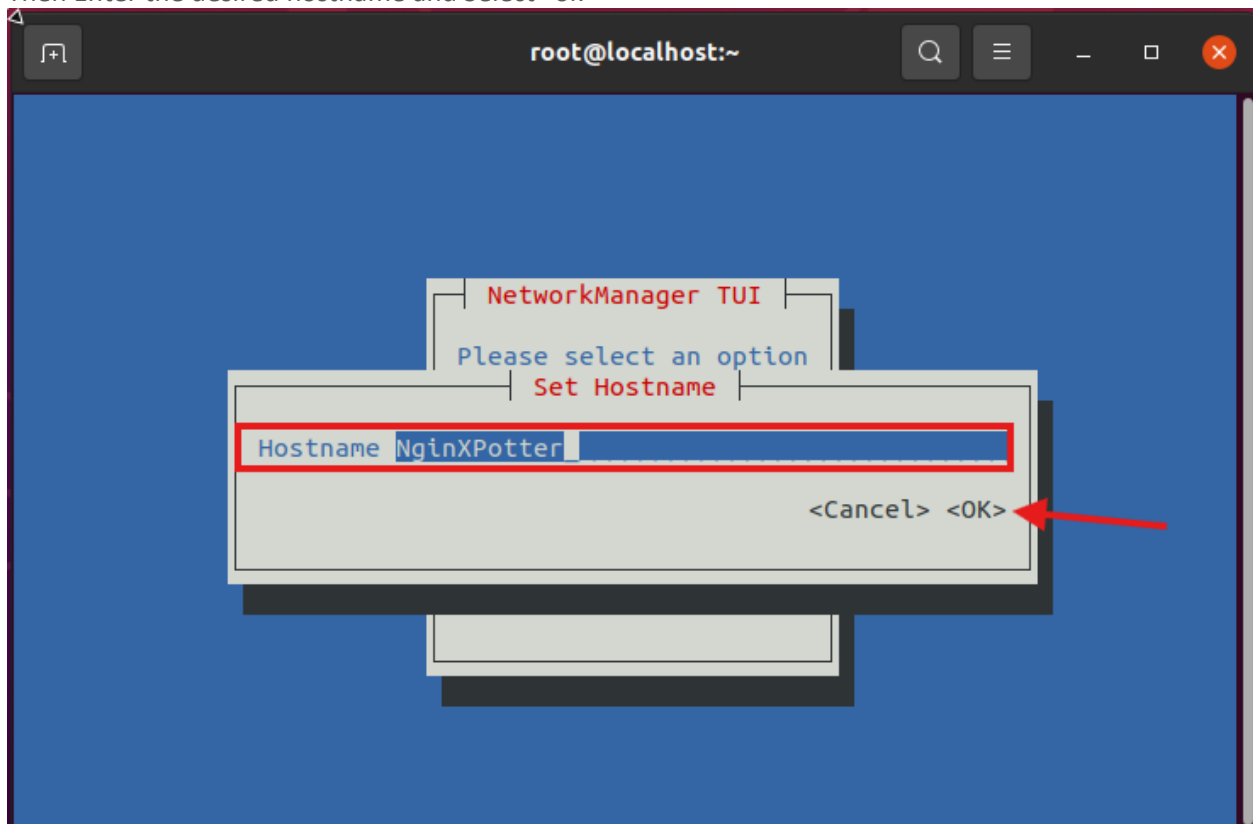
We will then use the “**nmtui**” command to set the host name.



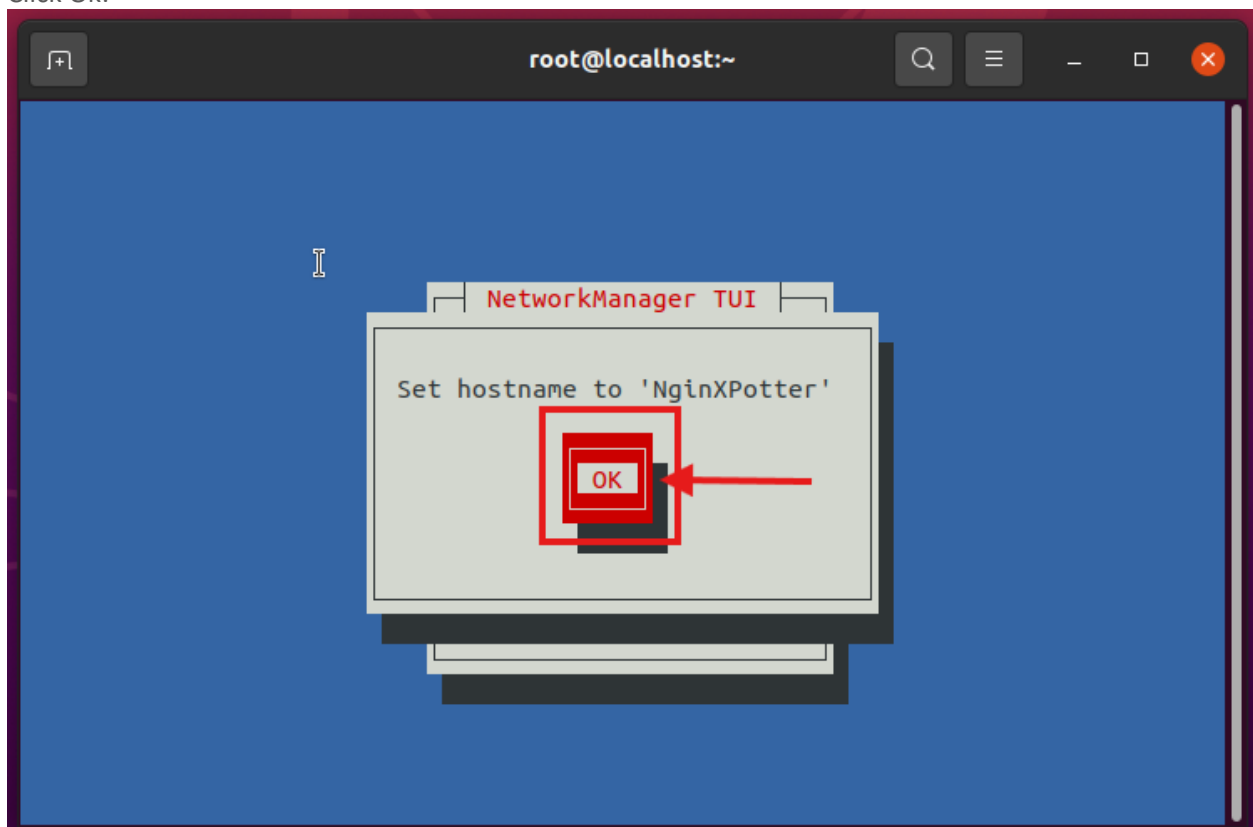
Select “Set system hostname”



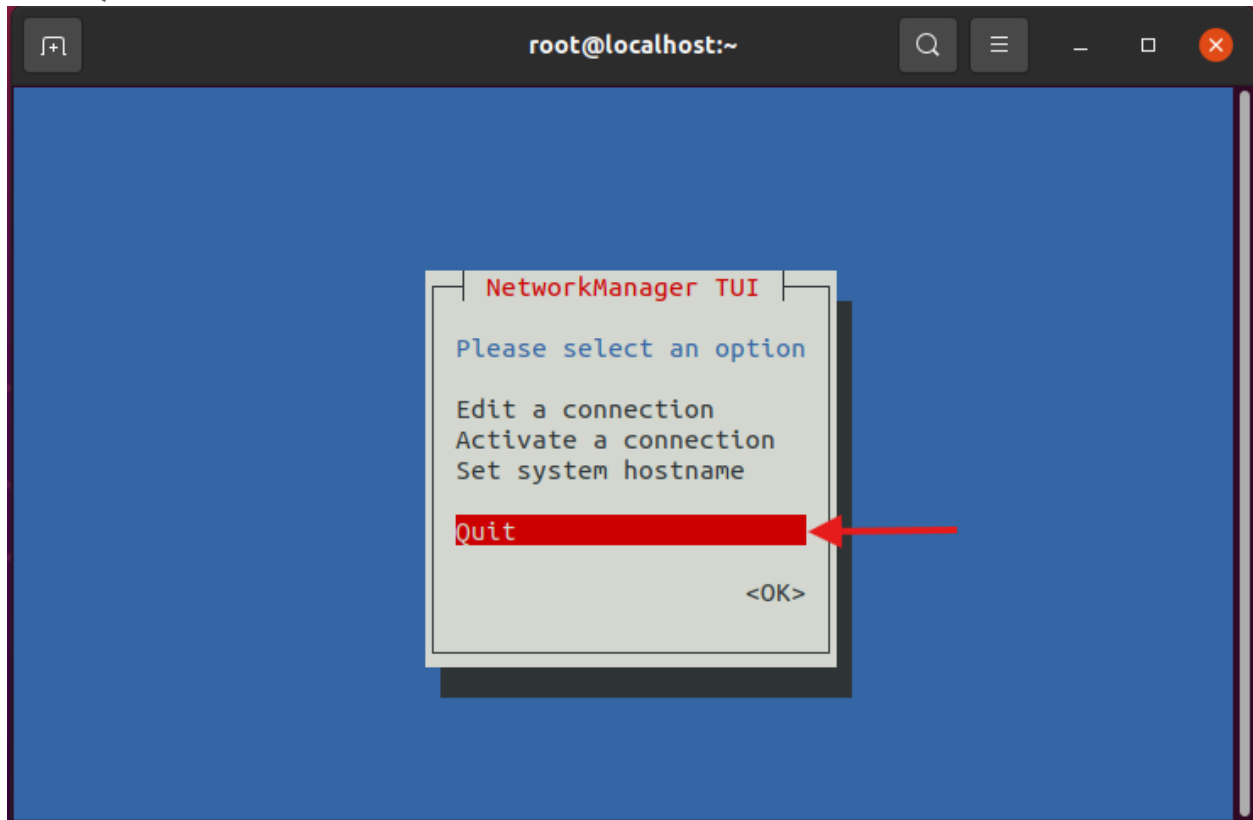
Then Enter the desired hostname and Select "ok"



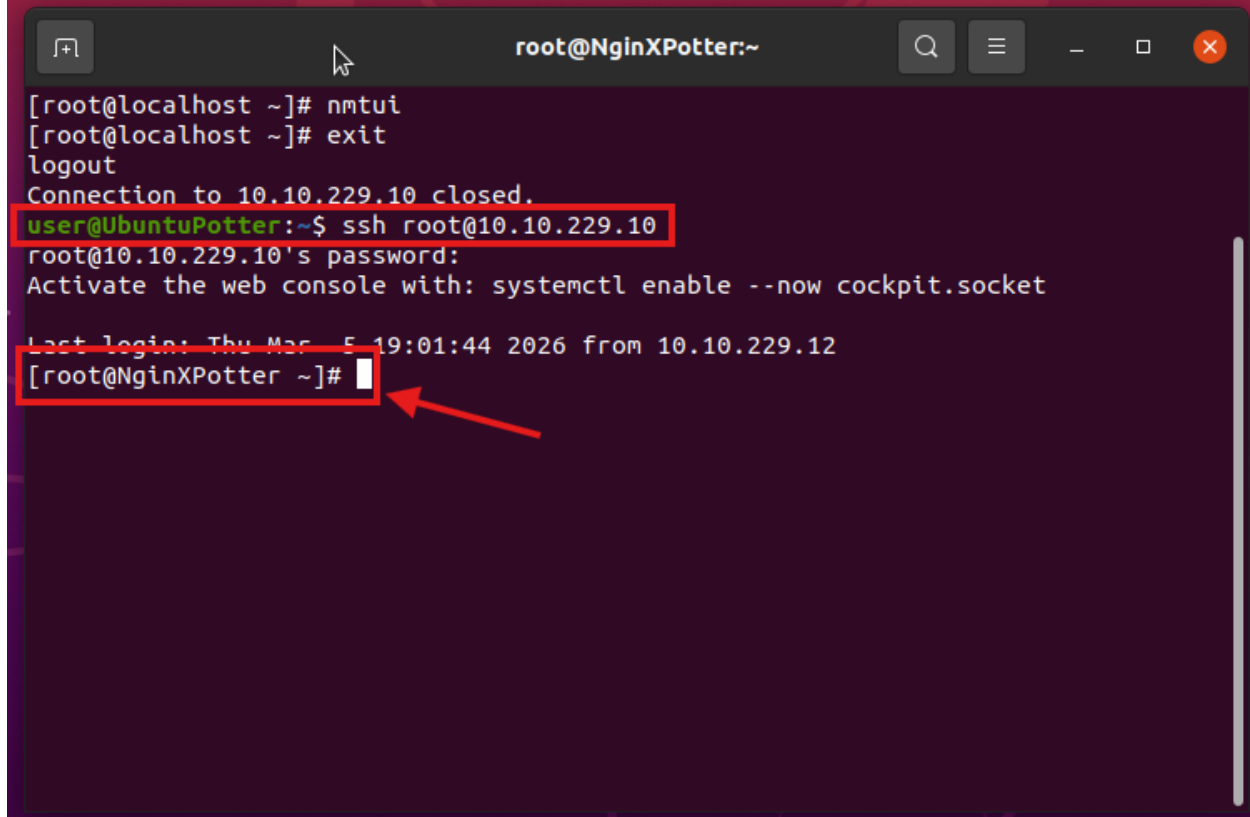
Click Ok:



Select Quit:



Then type "Exit" and relogin Using the "ssh 10.10.229.10" and Reenter the password to see the new Hostname Like Below:

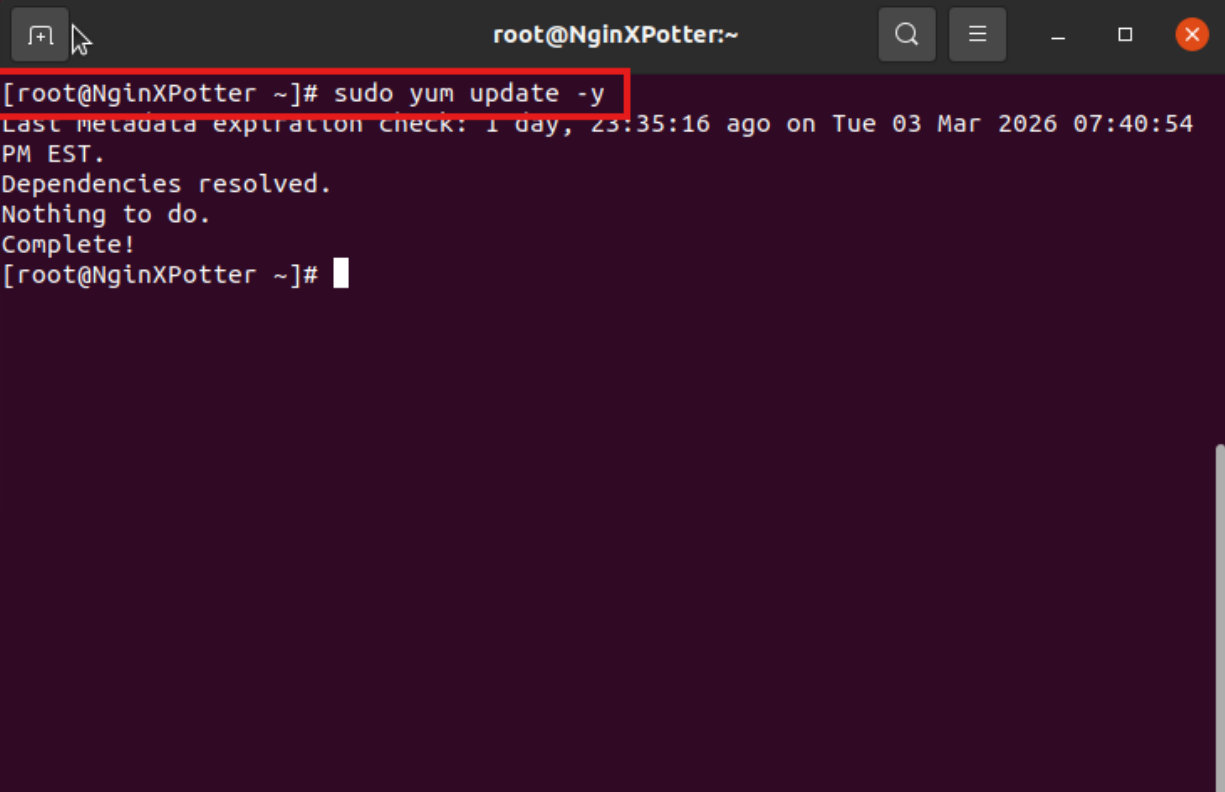


```
root@NginXPotter:~  
[root@localhost ~]# nmtui  
[root@localhost ~]# exit  
logout  
Connection to 10.10.229.10 closed.  
user@UbuntuPotter:~$ ssh root@10.10.229.10  
root@10.10.229.10's password:  
Activate the web console with: systemctl enable --now cockpit.socket  
  
Last login: Thu Mar 5 19:01:44 2026 from 10.10.229.12  
[root@NginXPotter ~]#
```

The terminal window shows a sequence of commands and their outputs. The user starts at a root prompt on a host named 'localhost'. They run 'nmtui' and 'exit', then 'logout'. A message indicates the connection to 10.10.229.10 is closed. The user then runs 'ssh root@10.10.229.10' from a prompt 'user@UbuntuPotter:~\$'. The SSH session prompts for a password, displays a message about enabling the cockpit socket, and shows the last login information. Finally, the user is at a root prompt on the remote host, which is now named 'NginXPotter'. A red box highlights the 'user@UbuntuPotter:~\$ ssh root@10.10.229.10' command, and another red box highlights the '[root@NginXPotter ~]#' prompt, with a red arrow pointing to it.

Update Rocky8-Nginx

To run the updates on our Rocky8-NginX System we simply input the command “**sudo yum update -y**” This will install any updates required for any currently installed packages:



```
root@NginXPotter:~  
[root@NginXPotter ~]# sudo yum update -y  
Last metadata expiration check: 1 day, 23:35:16 ago on Tue 03 Mar 2026 07:40:54 PM EST.  
Dependencies resolved.  
Nothing to do.  
Complete!  
[root@NginXPotter ~]#
```

Install EPEL Packages

We now want to install EPEL packages on the NginX system so to do this we will use the command “**sudo yum install epel-release -y**” this will install EPEL on the system.

```
root@NginXPotter:~  
[root@NginXPotter ~]# sudo yum install epel-release -y  
Last metadata expiration check: 1 day, 23:38:55 ago on Tue 03 Mar 2026 07:40:54 PM EST.  
Dependencies resolved.  
=====
```

Package	Architecture	Version	Repository	Size
---------	--------------	---------	------------	------

```
=====
```

Installing:

epel-release	noarch	8-22.el8	extras	25 k
--------------	--------	----------	--------	------

Transaction Summary

```
=====
```

Install 1 Package

Total download size: 25 k
Installed size: 34 k
Downloading Packages:

epel-release-8-22.el8.noarch.rpm	191 kB/s 25 kB	00:00
----------------------------------	------------------	-------

```
-----
```

Total	72 kB/s 25 kB	00:00
-------	-----------------	-------

```
-----
```

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction

Preparing	:	1/1
Installing	: epel-release-8-22.el8.noarch	1/1
Running scriptlet: epel-release-8-22.el8.noarch		1/1

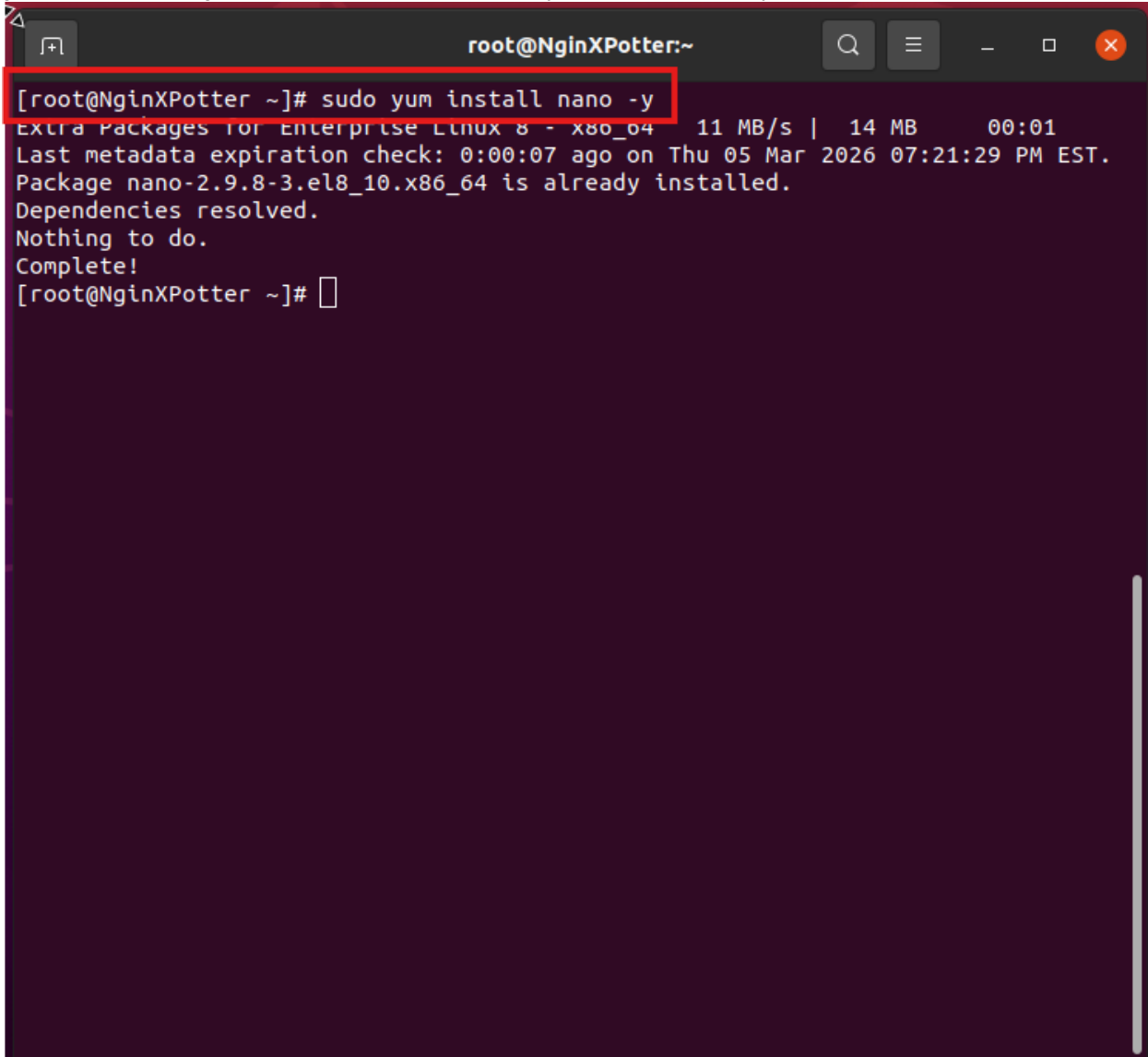
Many EPEL packages require the CodeReady Builder (CRB) repository.
It is recommended that you run /usr/bin/crb enable to enable the CRB repository.

Verifying	: epel-release-8-22.el8.noarch	1/1
-----------	--------------------------------	-----

Installed:

Install Nano Editor

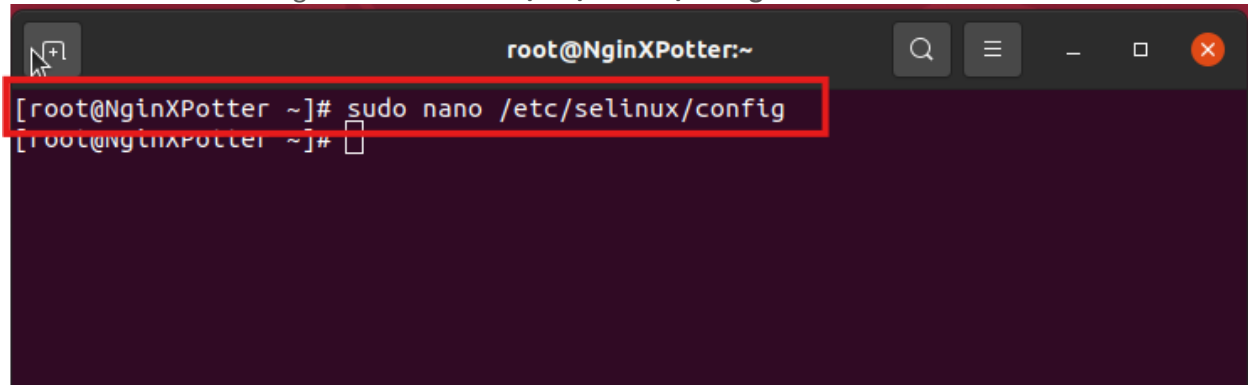
We now want to ensure nano is installed on the NginX system so to do this we will use the command “**sudo yum install nano -y**” this will install nano on the system. If it is already installed it will show like this:

A terminal window titled 'root@NginXPotter:~' with a dark purple background. The command '[root@NginXPotter ~]# sudo yum install nano -y' is entered and highlighted with a red box. The output shows progress for 'Extra Packages for Enterprise Linux 8 - x86_64' at 11 MB/s, a metadata expiration check from 07:21:29 PM EST on March 5, 2026, and a confirmation that 'Package nano-2.9.8-3.el8_10.x86_64 is already installed'. The terminal concludes with 'Dependencies resolved.', 'Nothing to do.', and 'Complete!'.

```
[root@NginXPotter ~]# sudo yum install nano -y
Extra Packages for Enterprise Linux 8 - x86_64 11 MB/s | 14 MB 00:01
Last metadata expiration check: 0:00:07 ago on Thu 05 Mar 2026 07:21:29 PM EST.
Package nano-2.9.8-3.el8_10.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@NginXPotter ~]#
```

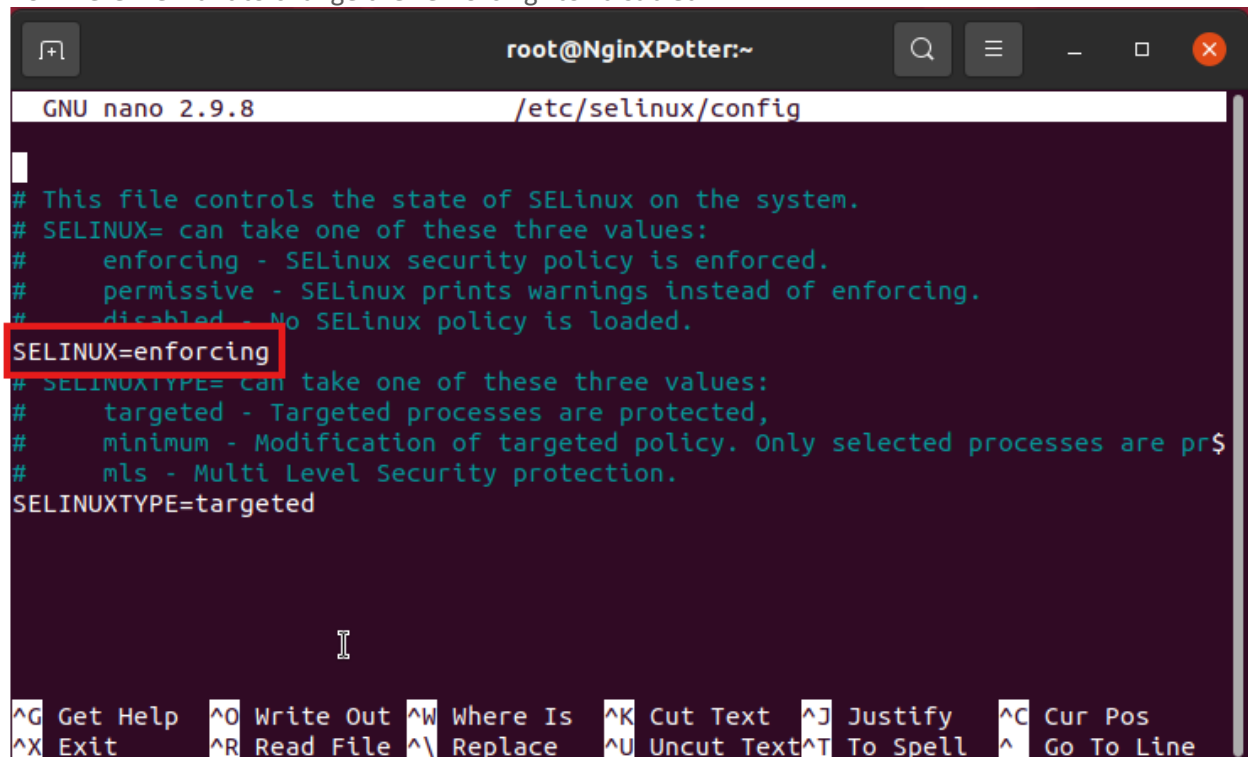

Disable SELinux

We will now be disabling SELinux in this environment so that we don't need to login and disable it every time we want to make a change in the future. I can only recommend this in a closed environment. The command to edit the SELinux Config file is "**sudo nano /etc/selinux/config**"



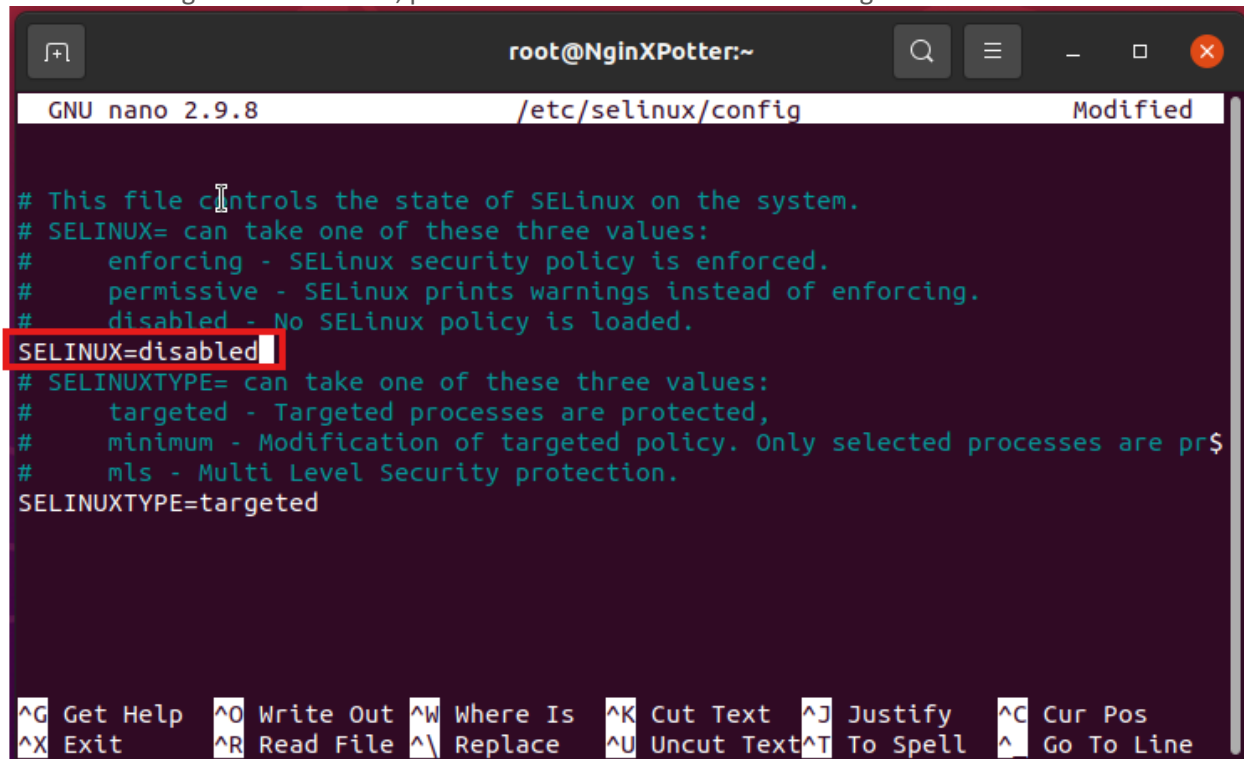
```
root@NginXPotter:~  
[root@NginXPotter ~]# sudo nano /etc/selinux/config  
[root@NginXPotter ~]#
```

From here we want to change the "enforcing" to "disabled"



```
GNU nano 2.9.8 /etc/selinux/config  
  
# This file controls the state of SELinux on the system.  
# SELINUX= can take one of these three values:  
#   enforcing - SELinux security policy is enforced.  
#   permissive - SELinux prints warnings instead of enforcing.  
#   disabled - No SELinux policy is loaded.  
SELINUX=enforcing  
# SELINUXTYPE= can take one of these three values:  
#   targeted - Targeted processes are protected,  
#   minimum - Modification of targeted policy. Only selected processes are protected.  
#   mls - Multi Level Security protection.  
SELINUXTYPE=targeted  
  
^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos  
^X Exit      ^R Read File  ^\ Replace   ^U Uncut Text ^T To Spell  ^_ Go To Line
```

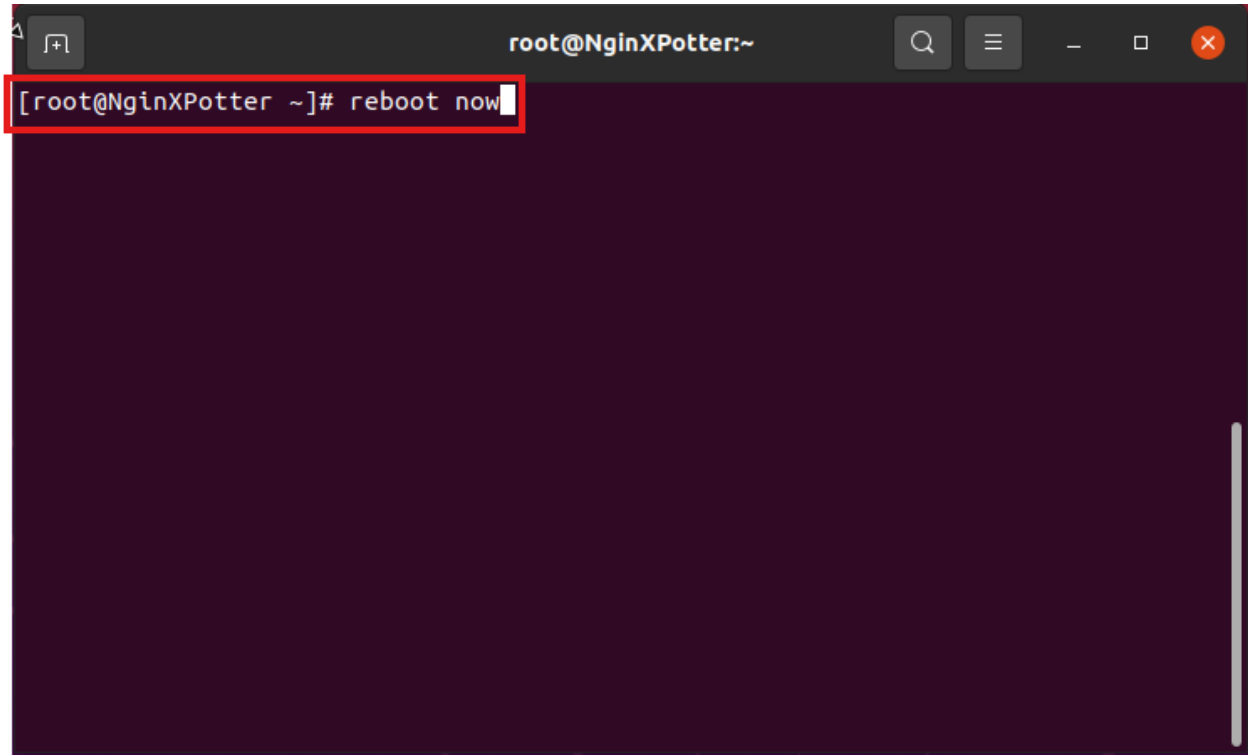
Make the Change and Press CtrlX, press Y and Hit Enter to save the changes.



```
root@NginXPotter:~  
GNU nano 2.9.8 /etc/selinux/config Modified  
  
# This file controls the state of SELinux on the system.  
# SELINUX= can take one of these three values:  
#     enforcing - SELinux security policy is enforced.  
#     permissive - SELinux prints warnings instead of enforcing.  
#     disabled - No SELinux policy is loaded.  
SELINUX=disabled  
# SELINUXTYPE= can take one of these three values:  
#     targeted - Targeted processes are protected,  
#     minimum - Modification of targeted policy. Only selected processes are protected.  
#     mls - Multi Level Security protection.  
SELINUXTYPE=targeted  
  
^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos  
^X Exit      ^R Read File  ^\ Replace   ^U Uncut Text ^T To Spell  ^_ Go To Line
```

Reboot VM

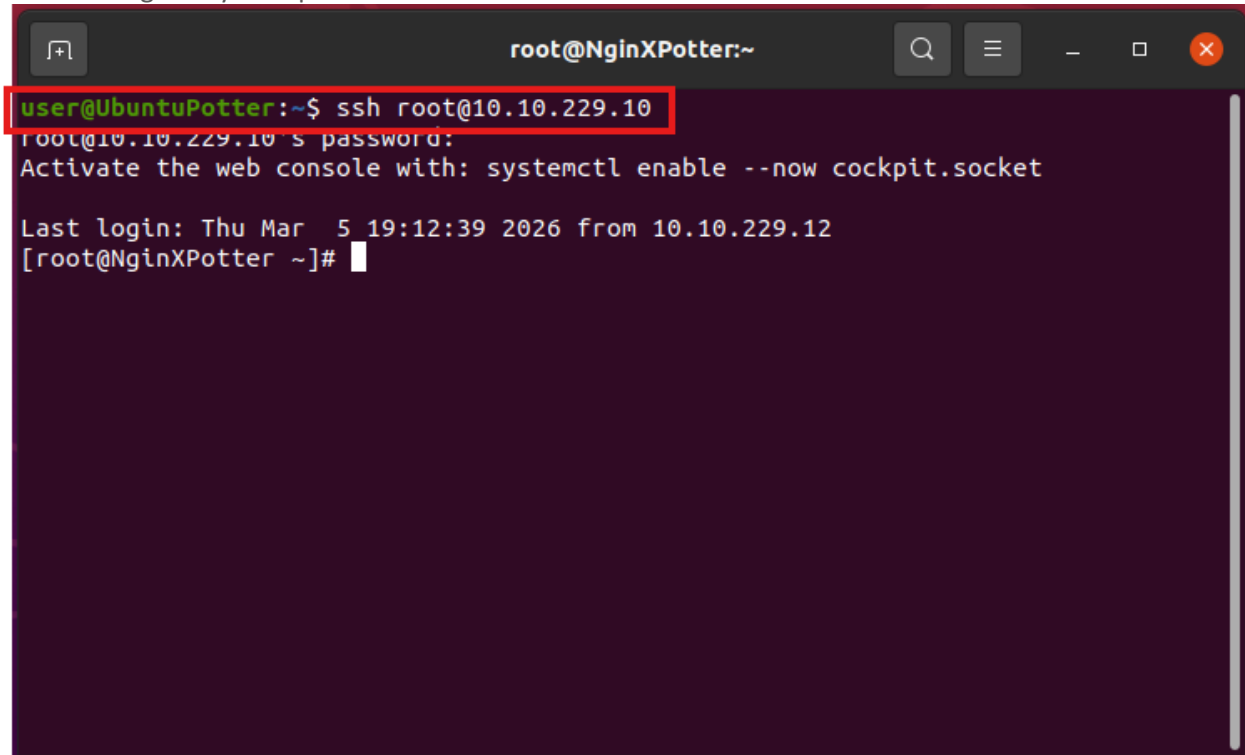
We now want to reboot to make sure the SELinux changes save correctly, to do this we will use the “**Reboot now**” command:

A terminal window titled 'root@NginXPotter:~' with standard window controls. The command '[root@NginXPotter ~]# reboot now' is entered and highlighted with a red rectangular box. The terminal background is dark purple.

```
[root@NginXPotter ~]# reboot now
```

Confirm SELinux Status

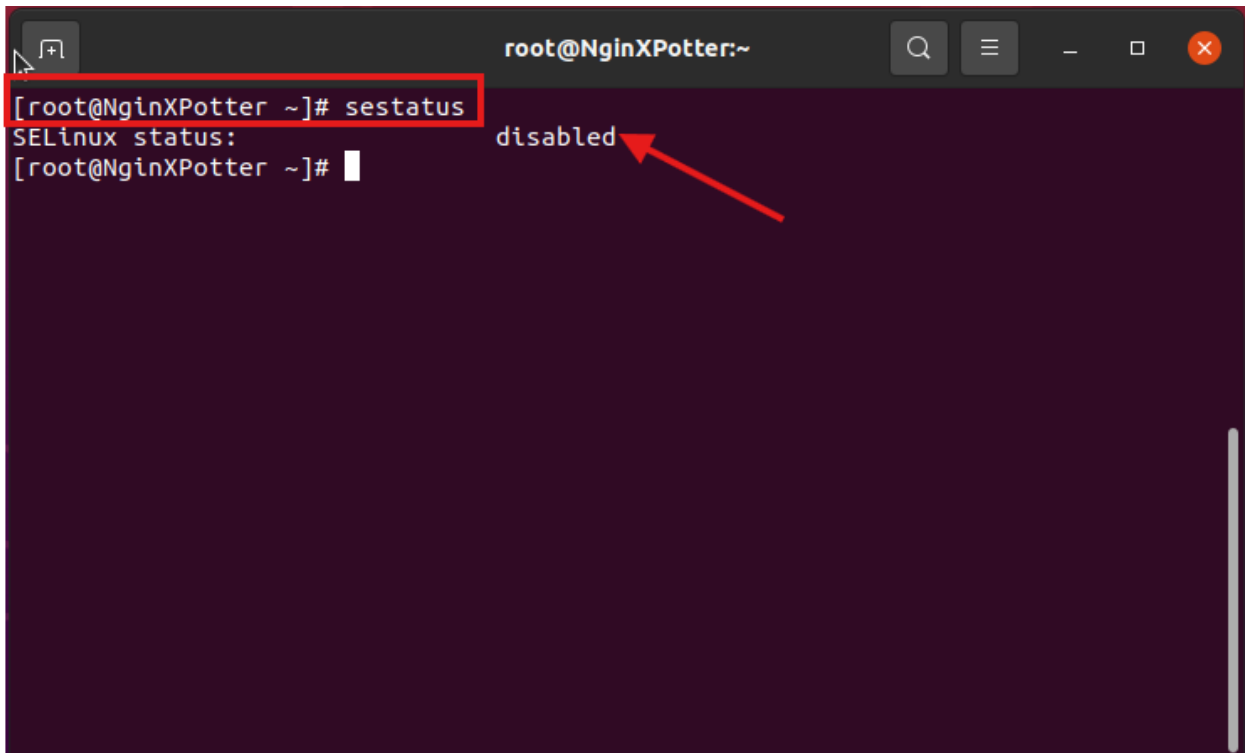
After Waiting a moment for the system to reboot with will reconnect using the “ssh root@10.10.229.10” and entering the system password to reconnect:

A terminal window titled 'root@NginXPotter:~' with standard window controls. The prompt is 'user@UbuntuPotter:~\$'. The command 'ssh root@10.10.229.10' is entered and highlighted with a red box. The output shows the password prompt, a system message to activate Cockpit, and a successful login for root at 10.10.229.12. The prompt changes to '[root@NginXPotter ~]#'.

```
user@UbuntuPotter:~$ ssh root@10.10.229.10
root@10.10.229.10's password:
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Thu Mar  5 19:12:39 2026 from 10.10.229.12
[root@NginXPotter ~]#
```

And to confirm That the SELinux is disable we will use the command “sestatus” the output should look like this:

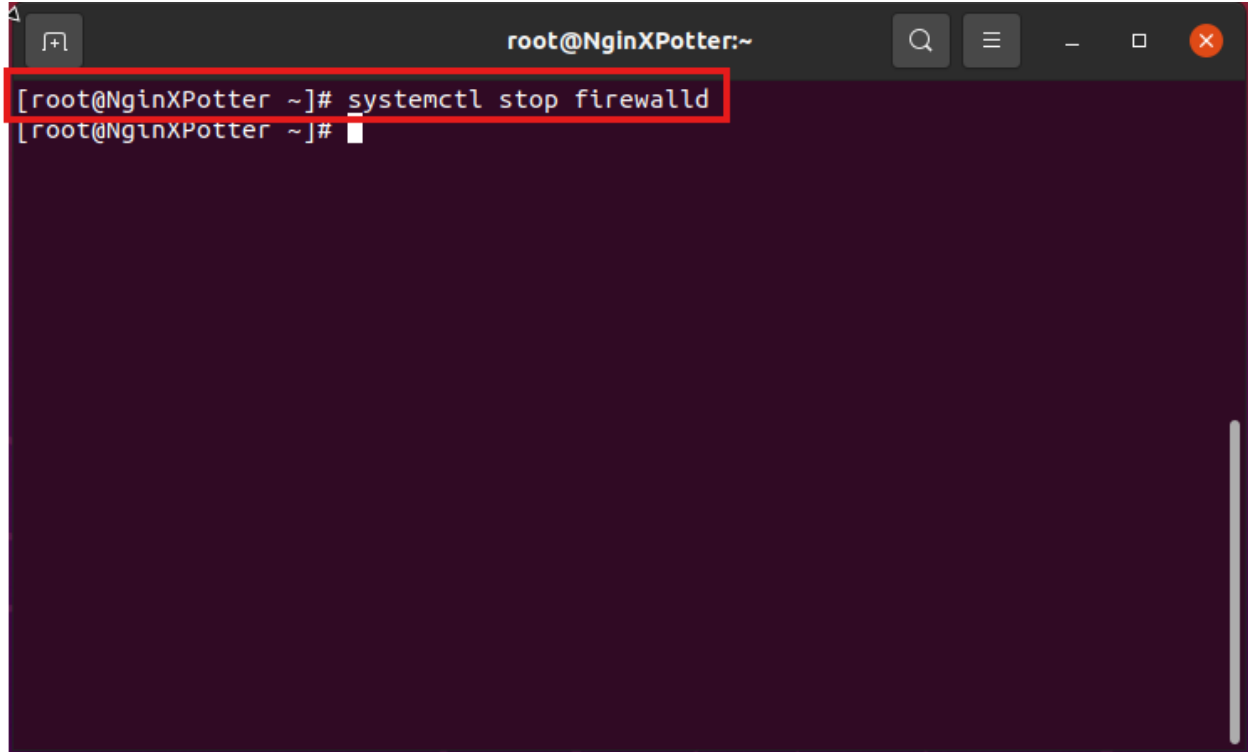
A terminal window titled 'root@NginXPotter:~' with standard window controls. The prompt is '[root@NginXPotter ~]#'. The command 'sestatus' is entered and highlighted with a red box. The output is 'SELinux status: disabled', with a red arrow pointing to the word 'disabled'. The prompt remains '[root@NginXPotter ~]#'.

```
[root@NginXPotter ~]# sestatus
SELinux status: disabled
[root@NginXPotter ~]#
```

Rocky Firewall

Stop Firewall

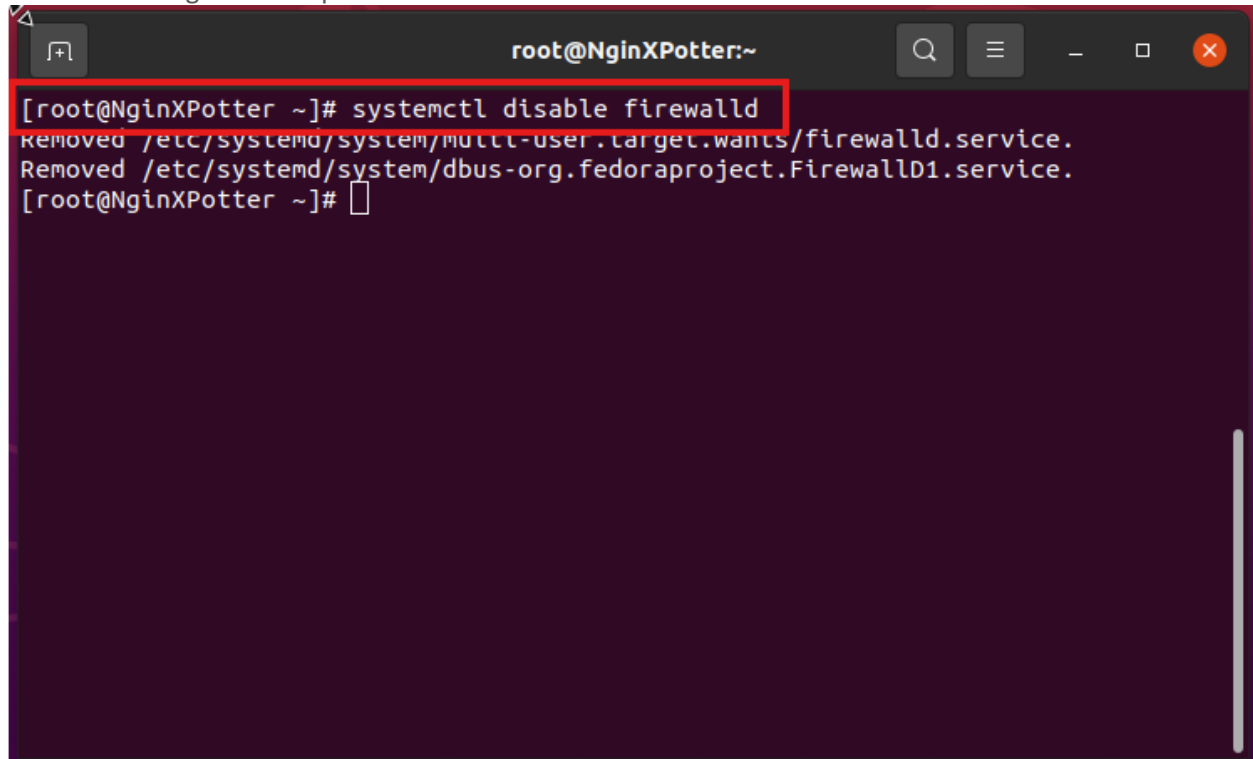
We now need to Stop the firewall on the NginX to do this we will enter the command “**systemctl stop firewalld**” Note: There is no output for the command. This is normal.

A terminal window titled 'root@NginXPotter:~' with standard window controls. The command '[root@NginXPotter ~]# systemctl stop firewalld' is entered and highlighted with a red box. The prompt changes to '[root@NginXPotter ~]#' on the next line, indicating the command has been executed without any visible output.

```
root@NginXPotter:~  
[root@NginXPotter ~]# systemctl stop firewalld  
[root@NginXPotter ~]#
```

Disable Firewall

We now need to disable the firewall on the NginX to do this we will enter the command “**systemctl disable firewalld**” and get this output:

A terminal window titled 'root@NginXPotter:~' with a dark background. The command '[root@NginXPotter ~]# systemctl disable firewalld' is entered and highlighted with a red box. The output shows two lines: 'Removed /etc/systemd/system/multi-user.target.wants/firewalld.service.' and 'Removed /etc/systemd/system/dbus-org.fedoraproject.FirewallD1.service.'. The prompt '[root@NginXPotter ~]#' is followed by a cursor.

```
[root@NginXPotter ~]# systemctl disable firewalld
Removed /etc/systemd/system/multi-user.target.wants/firewalld.service.
Removed /etc/systemd/system/dbus-org.fedoraproject.FirewallD1.service.
[root@NginXPotter ~]#
```

NginX

Install NginX

We will now install NginX on this System we will do this by entering the command “**sudo yum install nginx -y**” you should get something that looks like this output:

```
root@NginXPotter:~  
[root@NginXPotter ~]# sudo yum install nginx -y  
Rocky Linux 8 - AppStream          18 kB/s | 4.8 kB      00:00  
Rocky Linux 8 - AppStream          9.0 MB/s | 26 MB      00:02  
Rocky Linux 8 - BaseOS             20 kB/s | 4.3 kB      00:00  
Rocky Linux 8 - BaseOS            9.5 MB/s | 51 MB      00:05  
Rocky Linux 8 - Extras             5.1 kB/s | 3.1 kB      00:00  
Rocky Linux 8 - Extras            44 kB/s | 15 kB      00:00  
Dependencies resolved.  
=====
```

Package	Arch	Version	Repo	Size
---------	------	---------	------	------

```
=====
```

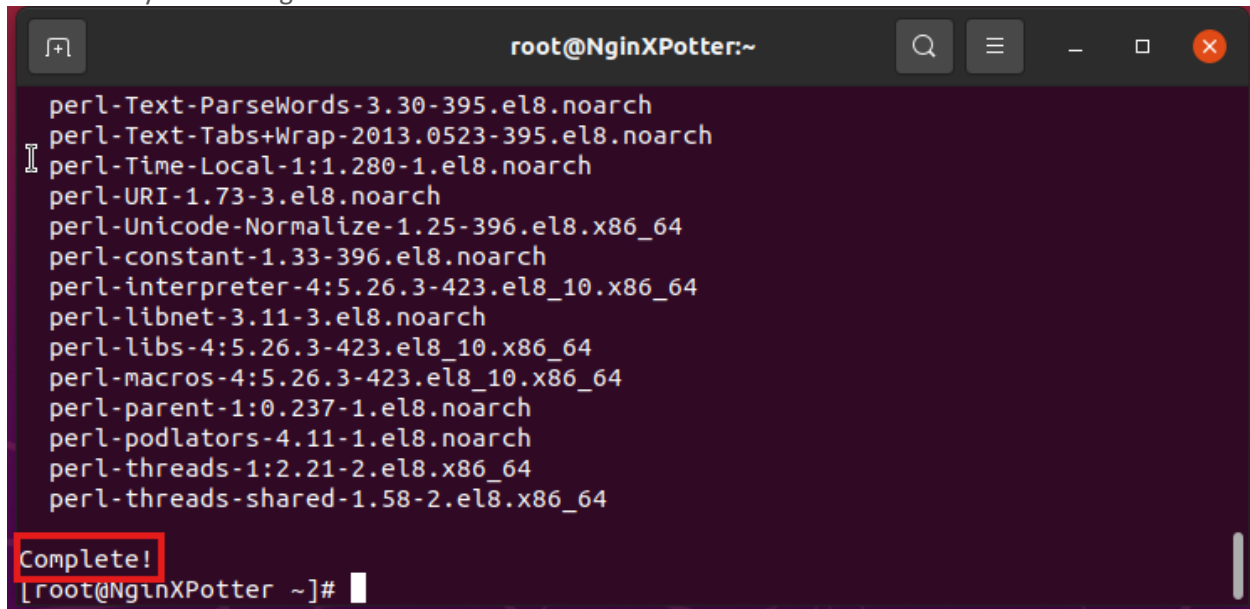
Installing:

nginx	x86_64	1:1.14.1-9.module+el8.4.0+542+81547229	appstream	566 k
-------	--------	--	-----------	-------

Installing dependencies:

gd	x86_64	2.2.5-1.el8	appstream	143 k
jbigkit-libs	x86_64	2.1-14.el8	appstream	54 k
libXpm	x86_64	3.5.12-11.el8	appstream	58 k
libjpeg-turbo	x86_64	1.5.3-14.el8_10	appstream	156 k
libtiff	x86_64	4.0.9-36.el8_10	appstream	190 k
libwebp	x86_64	1.0.0-11.el8_10	appstream	273 k
nginx-all-modules	noarch	1:1.14.1-9.module+el8.4.0+542+81547229	appstream	22 k
nginx-filesystem	noarch	1:1.14.1-9.module+el8.4.0+542+81547229	appstream	23 k
nginx-mod-http-image-filter	x86_64	1:1.14.1-9.module+el8.4.0+542+81547229	appstream	34 k
nginx-mod-http-perl	x86_64	1:1.14.1-9.module+el8.4.0+542+81547229	appstream	45 k
nginx-mod-http-xslt-filter	x86_64	1:1.14.1-9.module+el8.4.0+542+81547229	appstream	32 k
nginx-mod-mail	x86_64	1:1.14.1-9.module+el8.4.0+542+81547229	appstream	

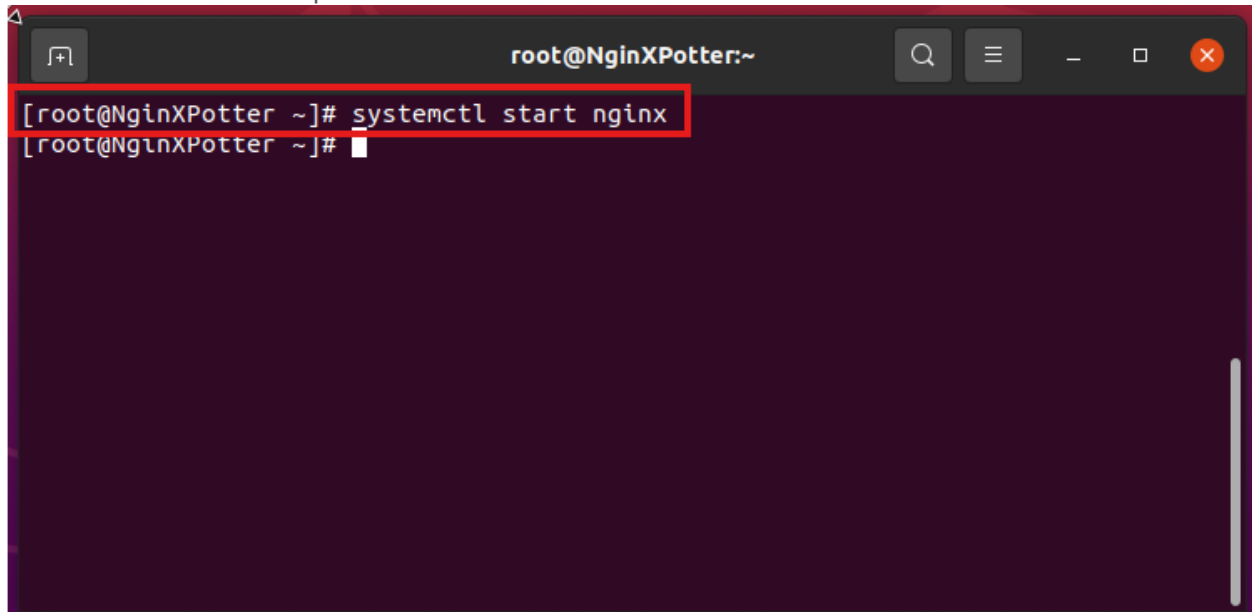
This is that you should get when that is done:



```
root@NginXPotter:~  
perl-Text-ParseWords-3.30-395.el8.noarch  
perl-Text-Tabs+Wrap-2013.0523-395.el8.noarch  
perl-Time-Local-1:1.280-1.el8.noarch  
perl-URI-1.73-3.el8.noarch  
perl-Unicode-Normalize-1.25-396.el8.x86_64  
perl-constant-1.33-396.el8.noarch  
perl-interpreter-4:5.26.3-423.el8_10.x86_64  
perl-libnet-3.11-3.el8.noarch  
perl-libs-4:5.26.3-423.el8_10.x86_64  
perl-macros-4:5.26.3-423.el8_10.x86_64  
perl-parent-1:0.237-1.el8.noarch  
perl-podlators-4.11-1.el8.noarch  
perl-threads-1:2.21-2.el8.x86_64  
perl-threads-shared-1.58-2.el8.x86_64  
Complete!  
[root@NginXPotter ~]#
```


Start NginX

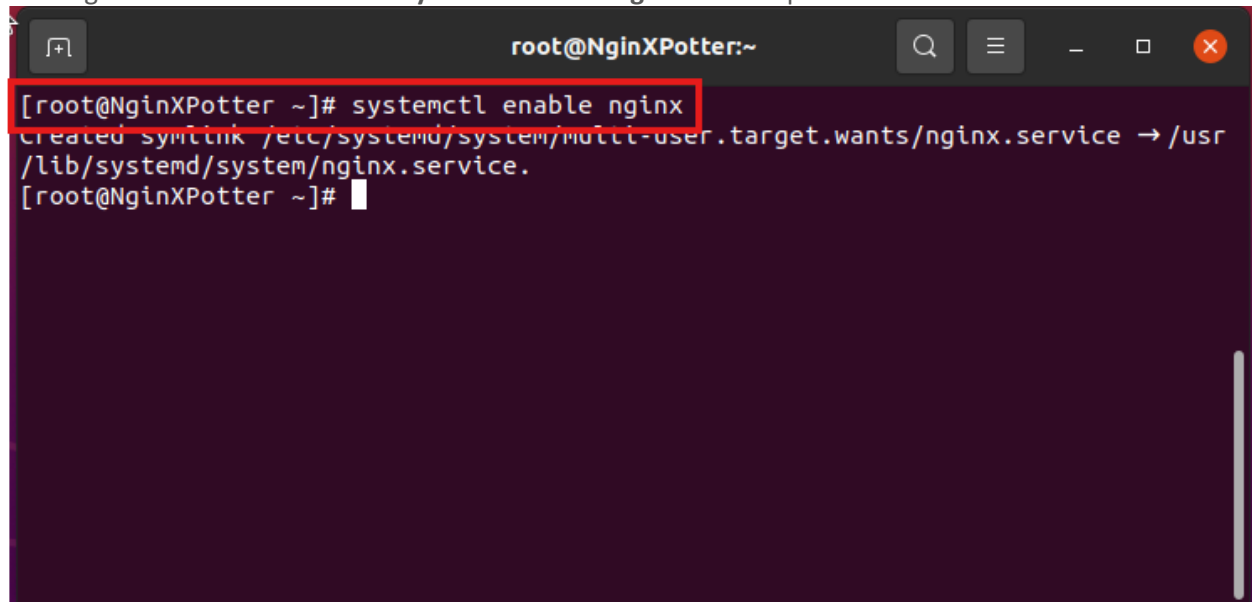
Once The installation is complete we need to start NginX we do this using the "**systemctl start nginx**" Note: This command has no output. This is normal.

A terminal window titled 'root@NginXPotter:~' with a dark background. The command '[root@NginXPotter ~]# systemctl start nginx' is entered and highlighted with a red box. The prompt '[root@NginXPotter ~]#' is visible on the next line, indicating the command executed successfully without any output.

```
root@NginXPotter:~  
[root@NginXPotter ~]# systemctl start nginx  
[root@NginXPotter ~]#
```

Enable NginX

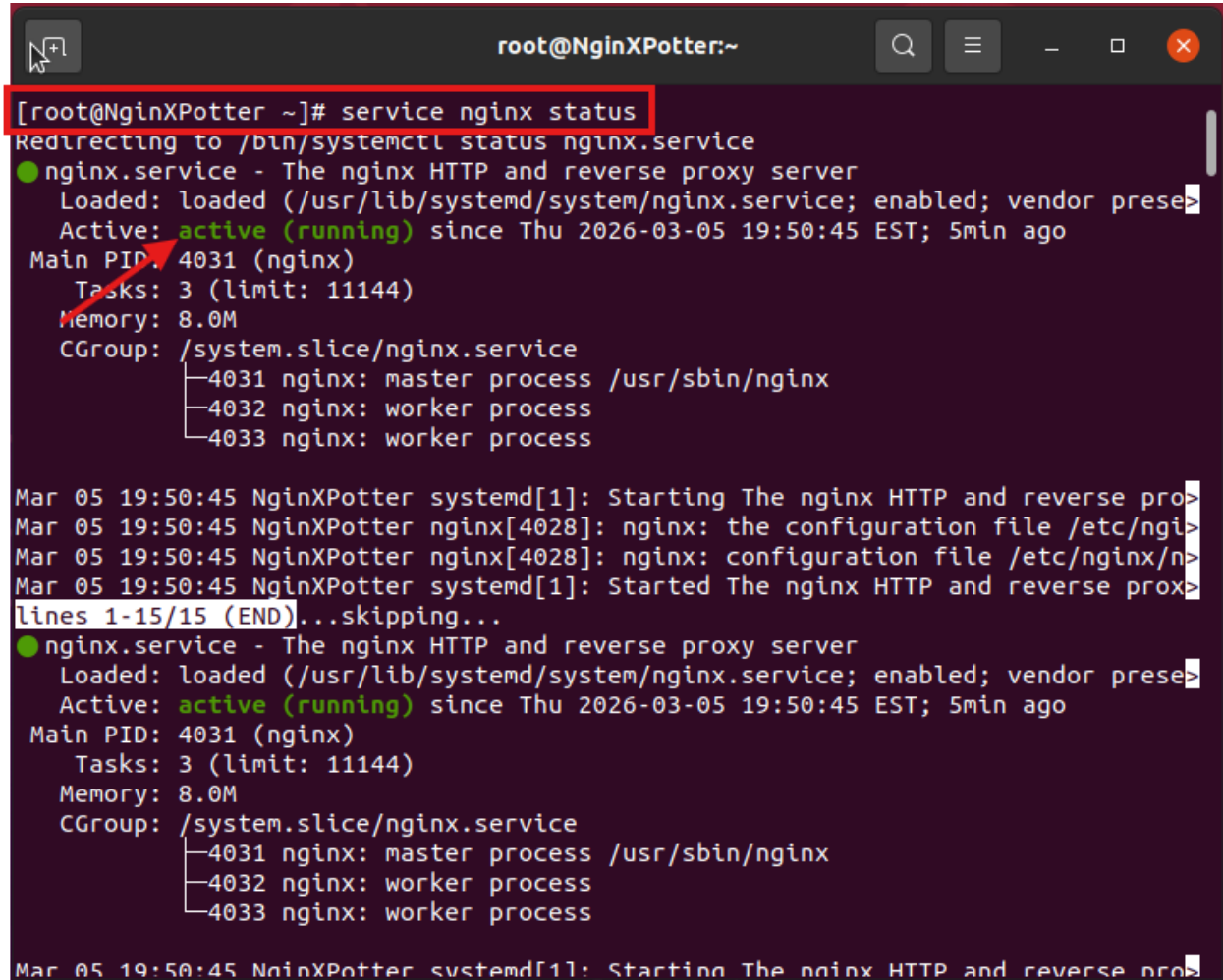
We now need to enable NginX on the system so that it starts automatically whenever the system is running the command for the is "**systemctl enable nginx**" the output should look like this:

A terminal window titled 'root@NginXPotter:~' with a dark background. The command '[root@NginXPotter ~]# systemctl enable nginx' is entered and highlighted with a red box. The output 'Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service -> /usr/lib/systemd/system/nginx.service.' is displayed on the next line. The prompt '[root@NginXPotter ~]#' is visible on the line following the output.

```
root@NginXPotter:~  
[root@NginXPotter ~]# systemctl enable nginx  
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service -> /usr/lib/systemd/system/nginx.service.  
[root@NginXPotter ~]#
```

Confirm NginX Status

We now want to ensure nginx is running we will use the command “**service nginx status**” you should see this:

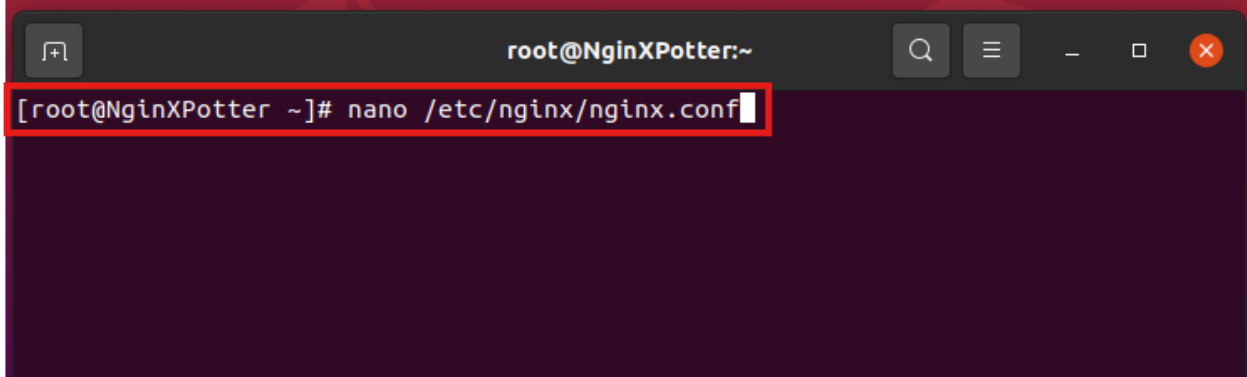


```
root@NginXPotter:~  
[root@NginXPotter ~]# service nginx status  
Redirecting to /bin/systemctl status nginx.service  
● nginx.service - The nginx HTTP and reverse proxy server  
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; vendor prese>  
   Active: active (running) since Thu 2026-03-05 19:50:45 EST; 5min ago  
 Main PID: 4031 (nginx)  
    Tasks: 3 (limit: 11144)  
   Memory: 8.0M  
    CGroup: /system.slice/nginx.service  
            └─4031 nginx: master process /usr/sbin/nginx  
              └─4032 nginx: worker process  
                └─4033 nginx: worker process  
  
Mar 05 19:50:45 NginXPotter systemd[1]: Starting The nginx HTTP and reverse pro>  
Mar 05 19:50:45 NginXPotter nginx[4028]: nginx: the configuration file /etc/ngi>  
Mar 05 19:50:45 NginXPotter nginx[4028]: nginx: configuration file /etc/nginx/n>  
Mar 05 19:50:45 NginXPotter systemd[1]: Started The nginx HTTP and reverse prox>  
lines 1-15/15 (END)...skipping...  
● nginx.service - The nginx HTTP and reverse proxy server  
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; vendor prese>  
   Active: active (running) since Thu 2026-03-05 19:50:45 EST; 5min ago  
 Main PID: 4031 (nginx)  
    Tasks: 3 (limit: 11144)  
   Memory: 8.0M  
    CGroup: /system.slice/nginx.service  
            └─4031 nginx: master process /usr/sbin/nginx  
              └─4032 nginx: worker process  
                └─4033 nginx: worker process  
  
Mar 05 19:50:45 NginXPotter systemd[1]: Starting The nginx HTTP and reverse pro>
```

Reverse Proxy for Ghost Site

Edit NginX configuration file

We now need to edit the configuration file for nginx, we will use the command "**nano /etc/nginx/nginx.conf**"

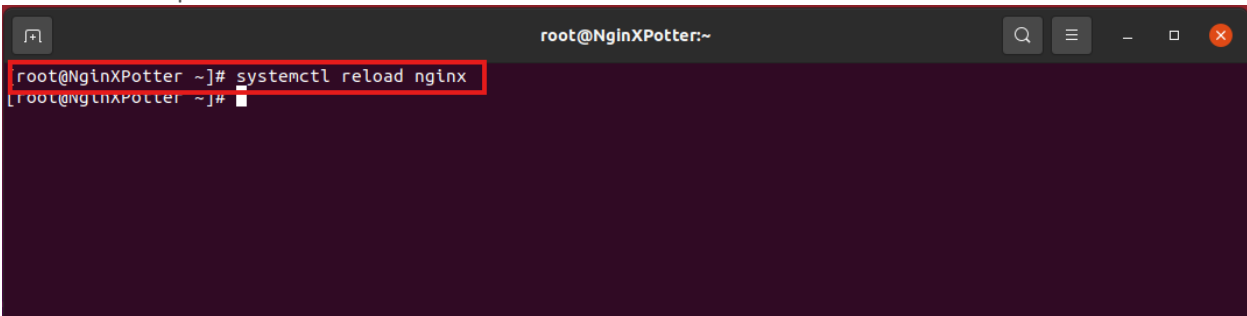


```
root@NginXPotter:~  
[root@NginXPotter ~]# nano /etc/nginx/nginx.conf
```

The Configurations Changes that need to be made are in Appendix A

Reload NginX service

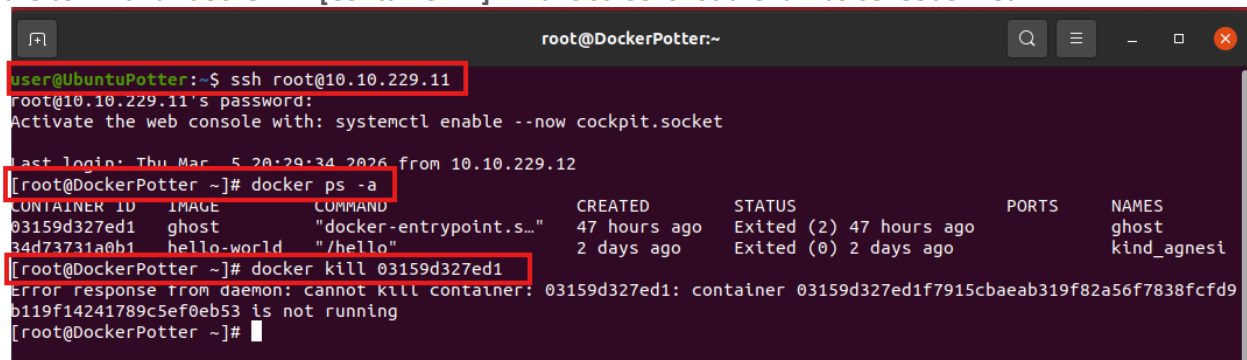
After the changes are made we need to reload nginx using the command "**systemctl reload nginx**" Note: there is no output for this command.



```
root@NginXPotter:~  
[root@NginXPotter ~]# systemctl reload nginx  
[root@NginXPotter ~]#
```

Terminate Docker

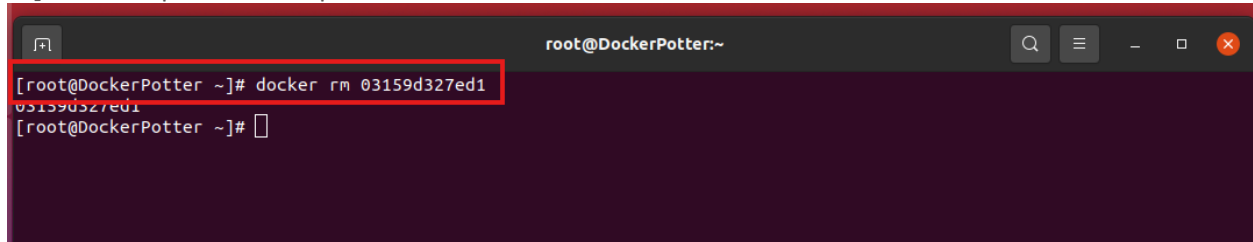
We now want to move back to our Docker system. Use the command "**exit**" to log out of the NginX system and log back into the docker with the command "**ssh root@10.10.229.11**". Once logged in, Use the command "**docker ps -a**" this will give you the container ID you will need for the next command. Use the command "**docker kill [Container ID]**" in this screenshot the Id was **03159d327ed1**:



```
user@UbuntuPotter:~$ ssh root@10.10.229.11  
root@10.10.229.11's password:  
Activate the web console with: systemctl enable --now cockpit.socket  
  
Last login: Thu Mar  5 20:29:34 2026 from 10.10.229.12  
[root@DockerPotter ~]# docker ps -a  
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS    NAMES  
03159d327ed1   ghost         "docker-entrypoint.s..." 47 hours ago  Exited (2) 47 hours ago    ghost  
34d73731a0b1   hello-world   "/hello"                 2 days ago  Exited (0) 2 days ago    kind_agnesi  
[root@DockerPotter ~]# docker kill 03159d327ed1  
Error response from daemon: cannot kill container: 03159d327ed1: container 03159d327ed1f7915cbaeab319f82a56f7838fcfd9b119f14241789c5ef0eb53 is not running  
[root@DockerPotter ~]#
```

Delete First Ghost Container

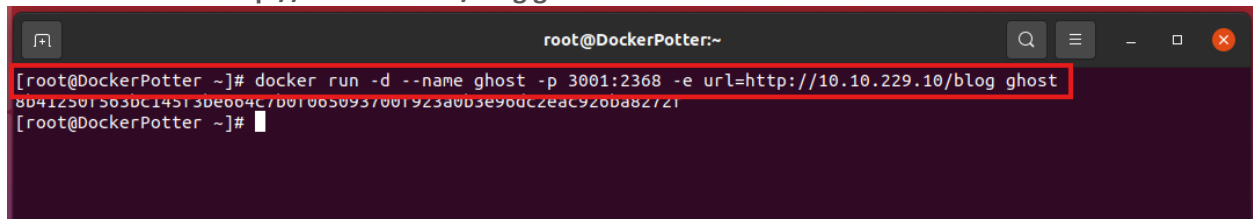
We then want to remove the Ghost container we just stopped the command for this is “**docker rm [Container ID]**”. The output should repeat the container Id like this:



```
root@DockerPotter:~  
[root@DockerPotter ~]# docker rm 03159d327ed1  
03159d327ed1  
[root@DockerPotter ~]#
```

Create New Ghost Container

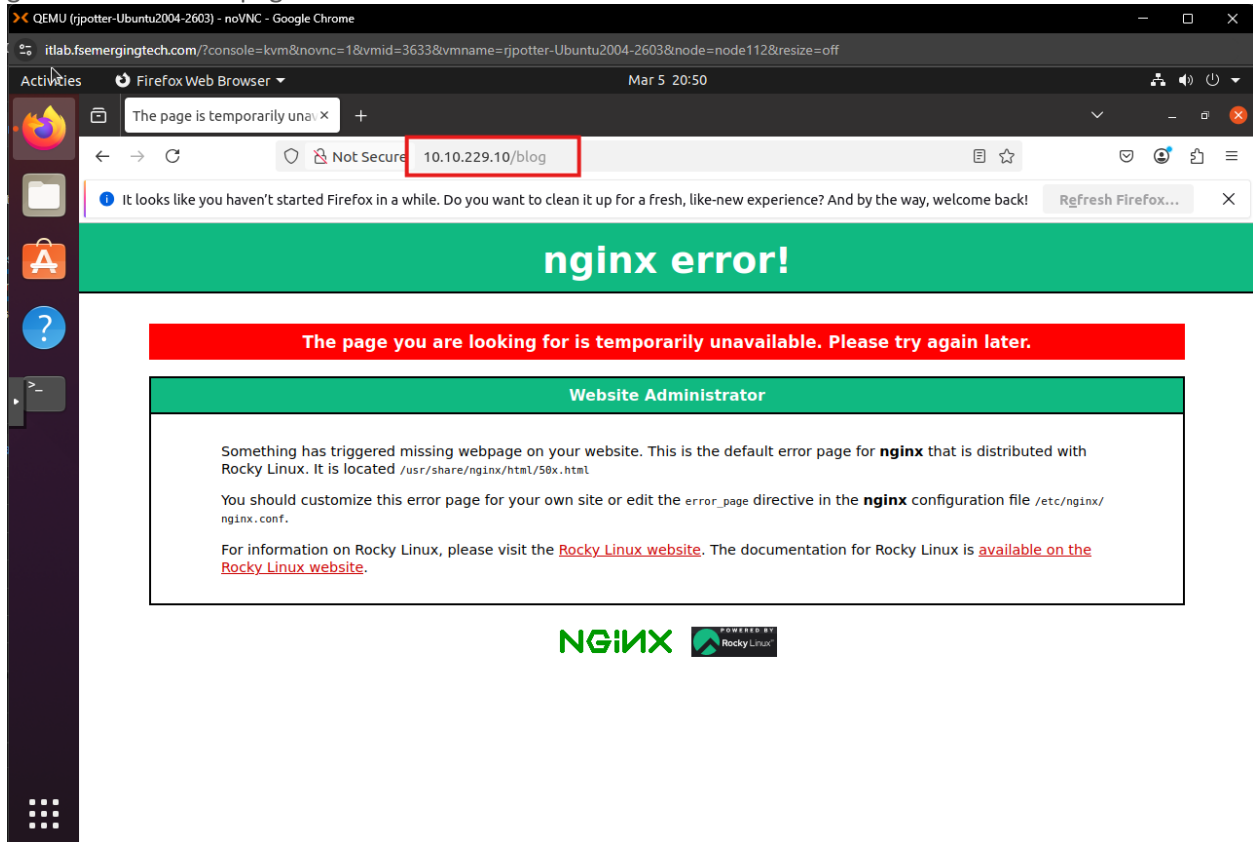
We will then Create a new ghost container using the following command “**docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.10/blog ghost**”.



```
root@DockerPotter:~  
[root@DockerPotter ~]# docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.10/blog ghost  
8b4125b15b3bc14513b0004c7b010050937001923a0b3e90dc2eac920ba82721  
[root@DockerPotter ~]#
```

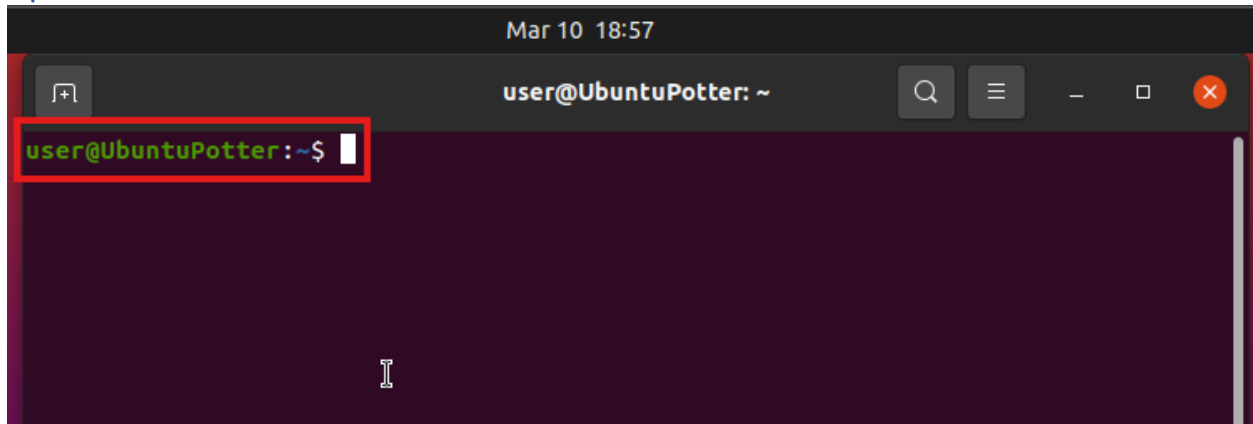
Browse to Ghost from Firefox on Ubuntu (10.10.229.10/blog)

We now want to test that our nginx config worked correctly and that the gost contained is created and running we will do this by opening firefox and navigating to the address "10.10.229.10/blog" This should generate and error page that looks like this:



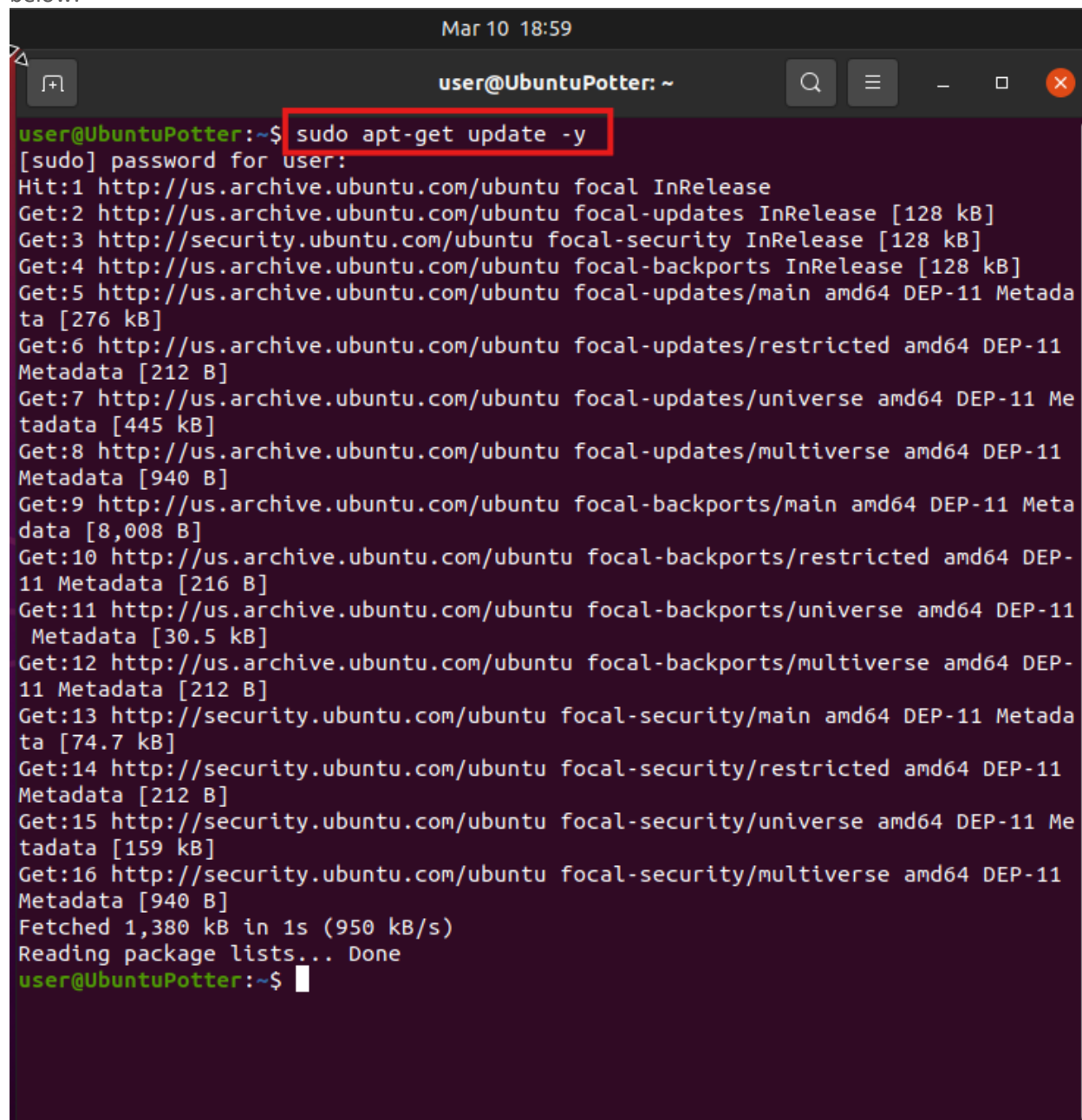
WordPress on Ubuntu - LAMP Stack

Update Ubuntu Host Name



Update Ubuntu

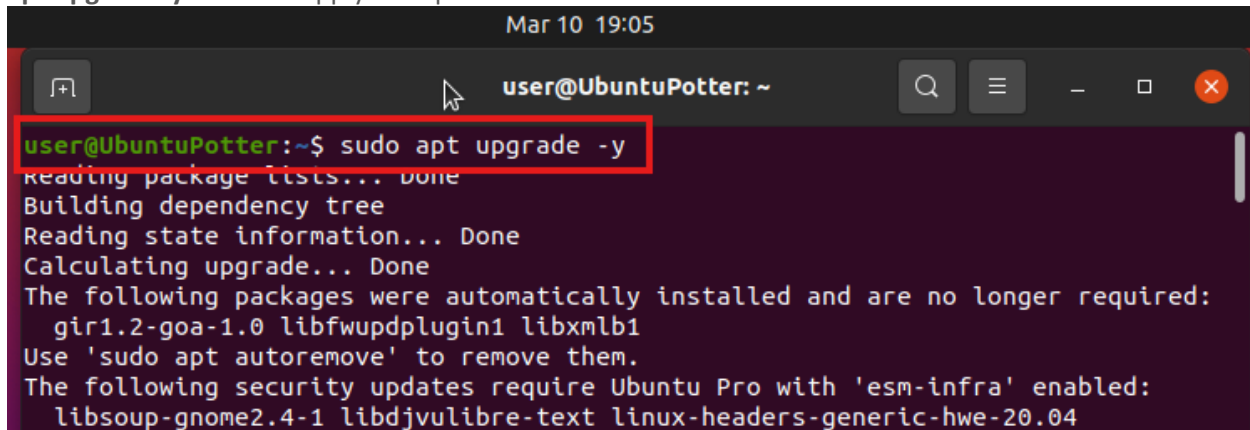
So before we start our installation of the LAMP stack we first want to make sure that our operating system is up to date. We will do this by using the command “**sudo apt-get update -y**”. The output may look like below:

A terminal window titled 'user@UbuntuPotter: ~' with a timestamp 'Mar 10 18:59'. The command 'sudo apt-get update -y' is entered and highlighted with a red box. The output shows the system fetching updates from various Ubuntu repositories, including focal, focal-security, focal-backports, and focal-updates for amd64 architecture. The process completes with 'Reading package lists... Done'.

```
Mar 10 18:59
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo apt-get update -y
[sudo] password for user:
Hit:1 http://us.archive.ubuntu.com/ubuntu focal InRelease
Get:2 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:3 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Get:4 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Get:5 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 DEP-11 Metadata [276 kB]
Get:6 http://us.archive.ubuntu.com/ubuntu focal-updates/restricted amd64 DEP-11 Metadata [212 B]
Get:7 http://us.archive.ubuntu.com/ubuntu focal-updates/universe amd64 DEP-11 Metadata [445 kB]
Get:8 http://us.archive.ubuntu.com/ubuntu focal-updates/multiverse amd64 DEP-11 Metadata [940 B]
Get:9 http://us.archive.ubuntu.com/ubuntu focal-backports/main amd64 DEP-11 Metadata [8,008 B]
Get:10 http://us.archive.ubuntu.com/ubuntu focal-backports/restricted amd64 DEP-11 Metadata [216 B]
Get:11 http://us.archive.ubuntu.com/ubuntu focal-backports/universe amd64 DEP-11 Metadata [30.5 kB]
Get:12 http://us.archive.ubuntu.com/ubuntu focal-backports/multiverse amd64 DEP-11 Metadata [212 B]
Get:13 http://security.ubuntu.com/ubuntu focal-security/main amd64 DEP-11 Metadata [74.7 kB]
Get:14 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 DEP-11 Metadata [212 B]
Get:15 http://security.ubuntu.com/ubuntu focal-security/universe amd64 DEP-11 Metadata [159 kB]
Get:16 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64 DEP-11 Metadata [940 B]
Fetched 1,380 kB in 1s (950 kB/s)
Reading package lists... Done
user@UbuntuPotter:~$
```

Upgrade Ubuntu

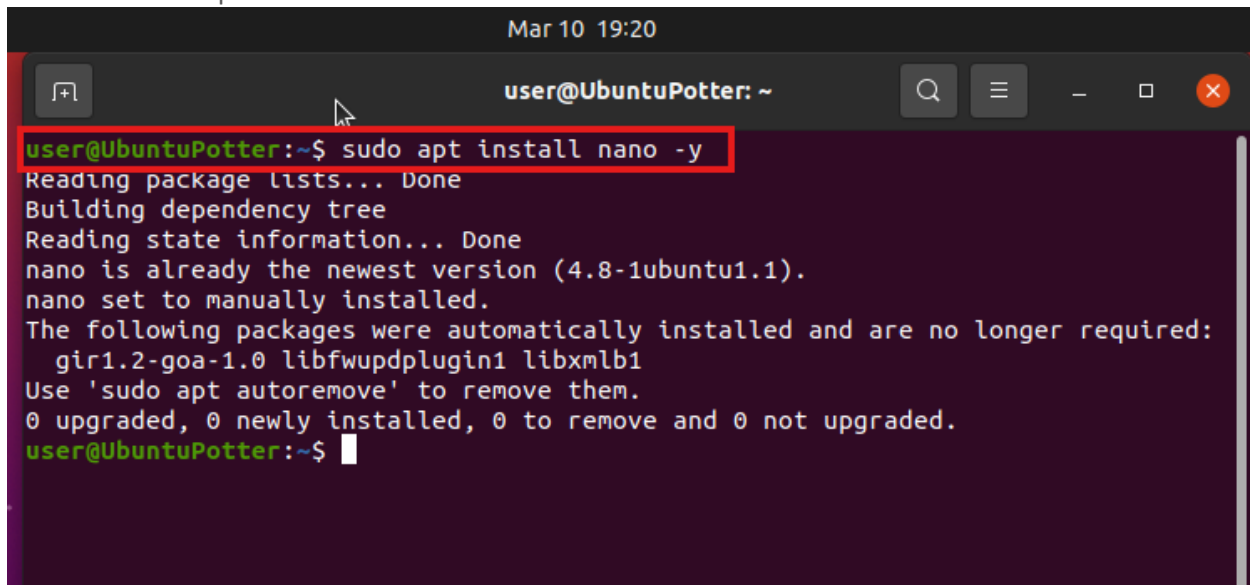
After the above command in order to apply the updates it may have found we will use the command “**sudo apt upgrade -y**” this will apply the updates that are available.



```
Mar 10 19:05
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
The following security updates require Ubuntu Pro with 'esm-infra' enabled:
  libsoup-gnome2.4-1 libdjvulibre-text linux-headers-generic-hwe-20.04
```

Install Nano Editor

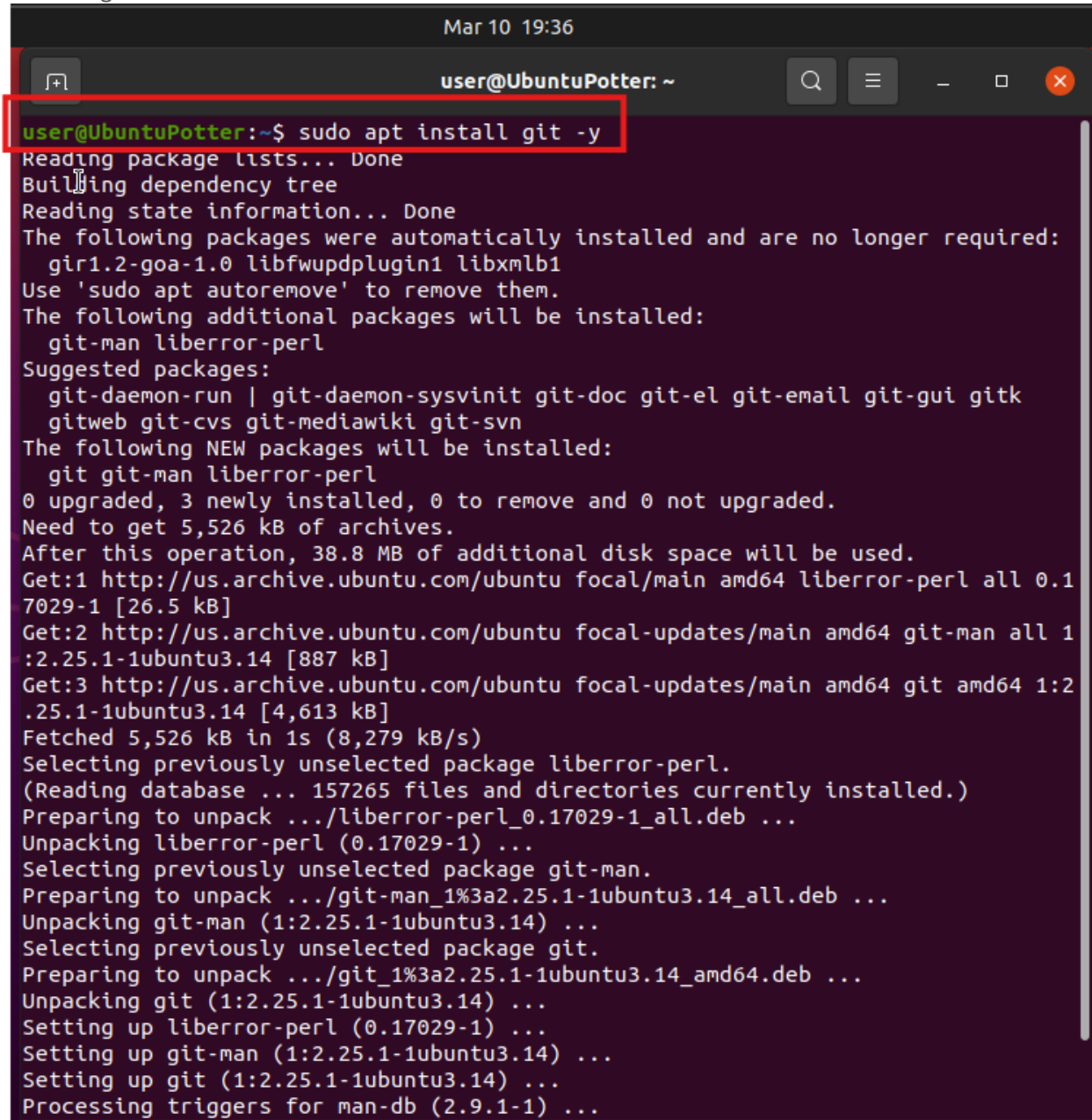
Next, We will be installing nano which is a command line text editor that we will be using later to edit some configuration file we need to change later we will do this with the command “**sudo apt install nano -y**”. You should see an output like this:



```
Mar 10 19:20
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo apt install nano -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
nano is already the newest version (4.8-1ubuntu1.1).
nano set to manually installed.
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
user@UbuntuPotter:~$
```


Install Git

Next, We will be installing Git we will be using this later in order to download templates and some later steps in this project. The command we will use for this is “**sudo apt install git -y**”, the output should look something like this:

A terminal window titled "user@UbuntuPotter: ~" with a timestamp "Mar 10 19:36". The command "sudo apt install git -y" is entered and highlighted with a red box. The output shows the package lists being read, dependencies being built, and state information being read. It lists packages that were automatically installed and are no longer required (gir1.2-goa-1.0, libfwupdplugin1, libxmlb1) and suggests removing them with 'sudo apt autoremove'. It then lists additional packages to be installed (git-man, liberror-perl) and suggested packages (git-daemon-run, git-daemon-sysvinit, git-doc, git-el, git-email, git-gui, gitk, gitweb, git-cvs, git-mediawiki, git-svn). It shows the new packages to be installed (git, git-man, liberror-perl) and the disk space requirements (5,526 kB of archives, 38.8 MB of additional disk space). It then shows the download progress for liberror-perl, git-man, and git, followed by the unpacking and setting up of these packages.

```
Mar 10 19:36
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo apt install git -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  git-man liberror-perl
Suggested packages:
  git-daemon-run | git-daemon-sysvinit git-doc git-el git-email git-gui gitk
  gitweb git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  git git-man liberror-perl
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 5,526 kB of archives.
After this operation, 38.8 MB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu focal/main amd64 liberror-perl all 0.1
7029-1 [26.5 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 git-man all 1
:2.25.1-1ubuntu3.14 [887 kB]
Get:3 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 git amd64 1:2
.25.1-1ubuntu3.14 [4,613 kB]
Fetched 5,526 kB in 1s (8,279 kB/s)
Selecting previously unselected package liberror-perl.
(Reading database ... 157265 files and directories currently installed.)
Preparing to unpack .../liberror-perl_0.17029-1_all.deb ...
Unpacking liberror-perl (0.17029-1) ...
Selecting previously unselected package git-man.
Preparing to unpack .../git-man_1%3a2.25.1-1ubuntu3.14_all.deb ...
Unpacking git-man (1:2.25.1-1ubuntu3.14) ...
Selecting previously unselected package git.
Preparing to unpack .../git_1%3a2.25.1-1ubuntu3.14_amd64.deb ...
Unpacking git (1:2.25.1-1ubuntu3.14) ...
Setting up liberror-perl (0.17029-1) ...
Setting up git-man (1:2.25.1-1ubuntu3.14) ...
Setting up git (1:2.25.1-1ubuntu3.14) ...
Processing triggers for man-db (2.9.1-1) ...
```

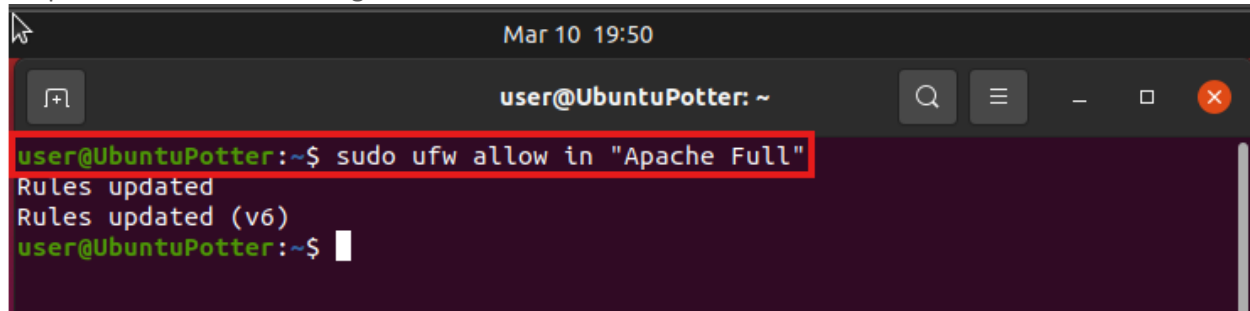
Install Apache2

Our next step is to install Apache2 on our Ubuntu machine, we will do this using the command “**sudo apt install apache2 -y**”. You should see an output like this:

```
Mar 10 19:45
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo apt install apache2 -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libapr1 libaprutil1
  libaprutil1-dbd-sqlite3 libaprutil1-ldap libcurl4 liblua5.2-0
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom
The following NEW packages will be installed:
  apache2 apache2-bin apache2-data apache2-utils libapr1 libaprutil1
  libaprutil1-dbd-sqlite3 libaprutil1-ldap libcurl4 liblua5.2-0
0 upgraded, 10 newly installed, 0 to remove and 0 not upgraded.
Need to get 2,063 kB of archives.
After this operation, 8,692 kB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 libapr1 amd64
  1.6.5-1ubuntu1.1 [91.5 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 libaprutil1 a
  md64 1.6.1-4ubuntu2.2 [85.1 kB]
Get:3 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 libaprutil1-d
  bd-sqlite3 amd64 1.6.1-4ubuntu2.2 [10.5 kB]
Get:4 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 libaprutil1-l
  dap amd64 1.6.1-4ubuntu2.2 [8,752 B]
Get:5 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 libcurl4 amd6
  4 7.68.0-1ubuntu2.25 [235 kB]
Get:6 http://us.archive.ubuntu.com/ubuntu focal/main amd64 liblua5.2-0 amd64 5.2
  .4-1.1build3 [106 kB]
Get:7 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 apache2-bin a
  md64 2.4.41-4ubuntu3.23 [1,188 kB]
Get:8 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 apache2-data
  all 2.4.41-4ubuntu3.23 [159 kB]
Get:9 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 apache2-utils
  amd64 2.4.41-4ubuntu3.23 [84.4 kB]
Get:10 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 apache2 amd6
  4 2.4.41-4ubuntu3.23 [95.6 kB]
```

Open Firewall Ports 80 and 443

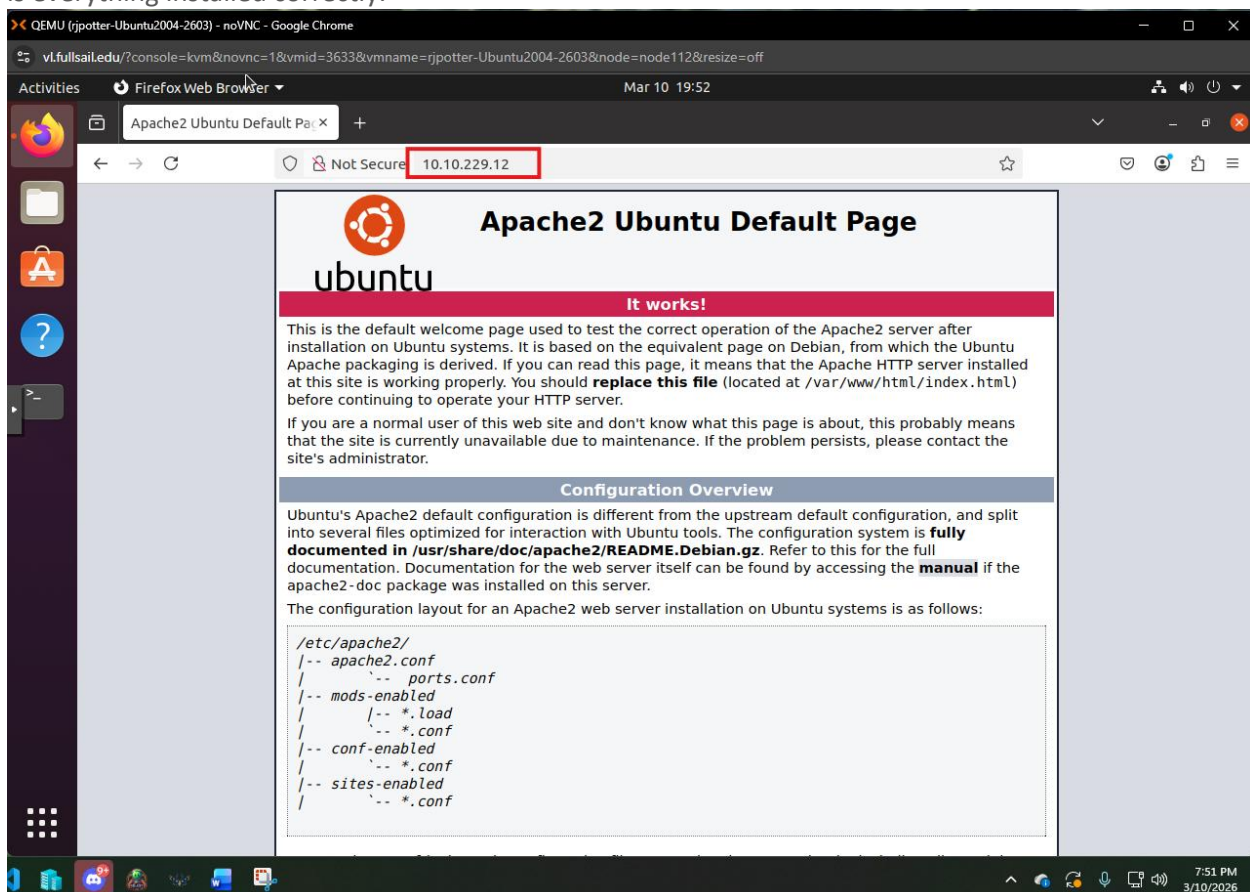
We now need to open Ports 80 and 443 for web traffic to be able to get to our web server, in order to do this we will use the command **"sudo ufw allow in "Apache Full"**". This adds Rules to our firewall and the output should look something like this:



```
Mar 10 19:50
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo ufw allow in "Apache Full"
Rules updated
Rules updated (v6)
user@UbuntuPotter:~$
```

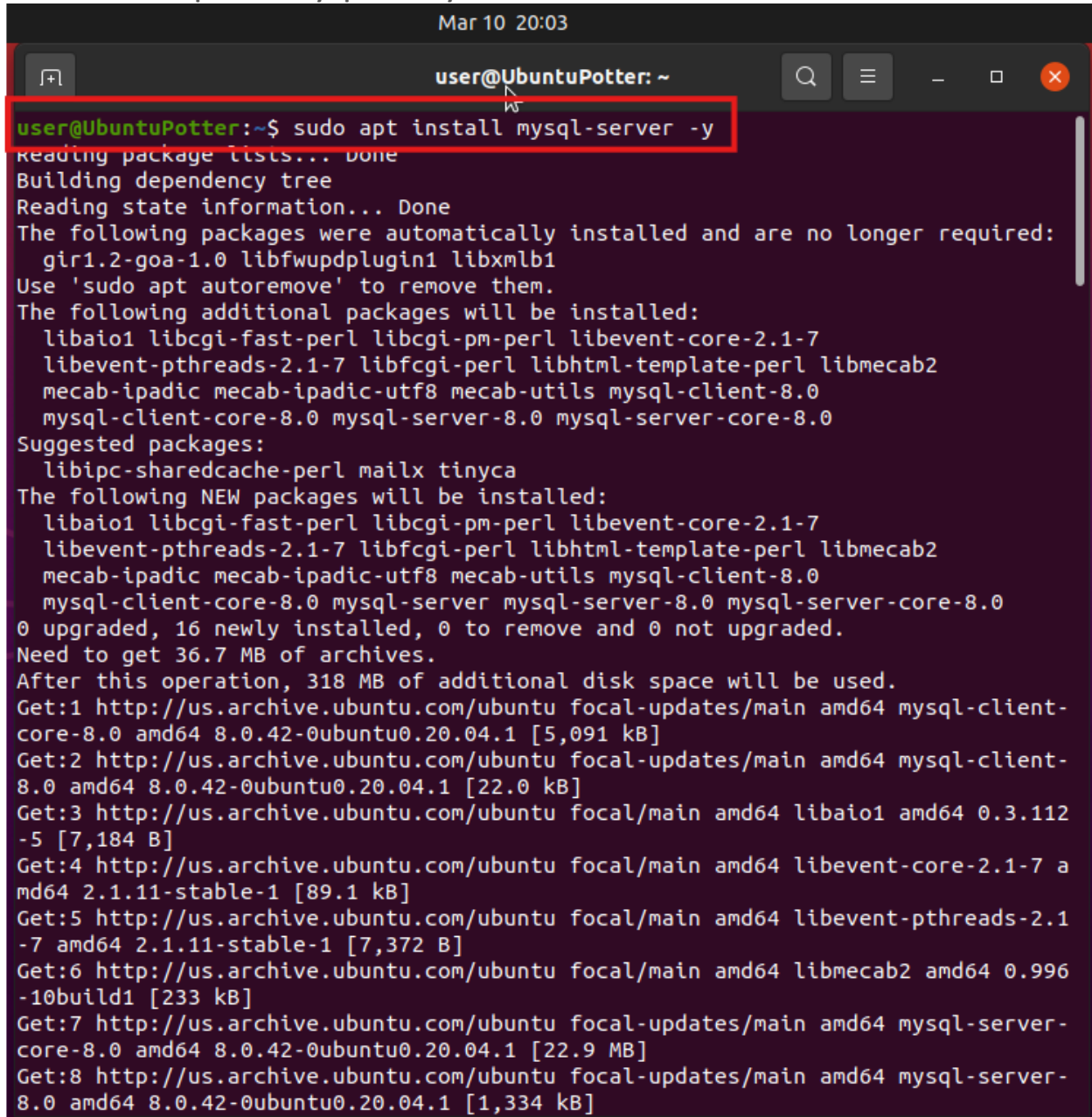
Browse to Apache2 Ubuntu Default Page

Now, we will test to make sure that our firewall rule applied correctly and that we can access our Apache2 install. To do this open the Firefox Browser and navigate to **http://10.10.229.12:80**. You should see this page is everything installed correctly:



Install MySQL

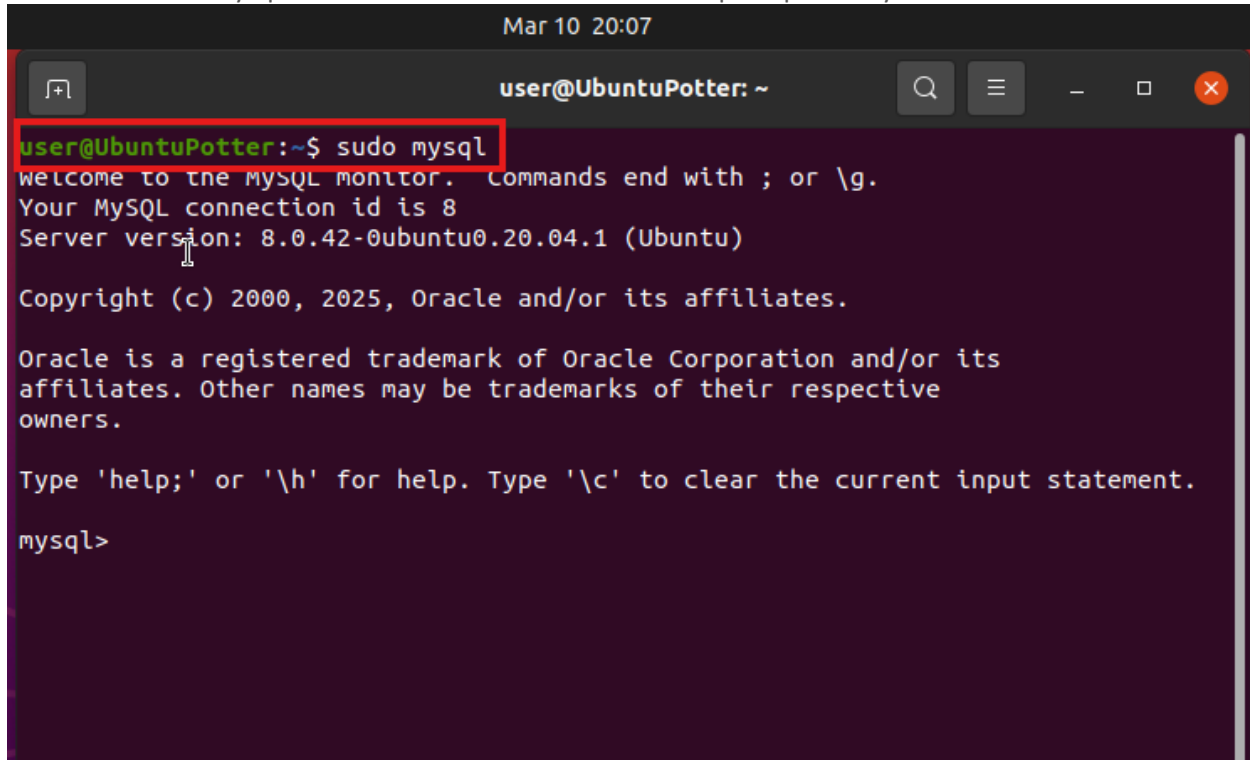
Our next step is to install the MySQL Server Database on our Lamp Stack. We will start this by entering the command “**sudo apt install mysql-server -y**” It should look like this:

A terminal window titled 'user@UbuntuPotter: ~' with a timestamp 'Mar 10 20:03'. The command 'sudo apt install mysql-server -y' is entered and highlighted with a red box. The output shows the installation process, including reading package lists, building a dependency tree, and listing packages to be installed. The command is executed successfully, and the terminal displays the following output:

```
user@UbuntuPotter:~$ sudo apt install mysql-server -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libaio1 libcgi-fast-perl libcgi-pm-perl libevent-core-2.1-7
  libevent-pthreads-2.1-7 libfcgi-perl libhtml-template-perl libmecab2
  mecab-ipadic mecab-ipadic-utf8 mecab-utils mysql-client-8.0
  mysql-client-core-8.0 mysql-server-8.0 mysql-server-core-8.0
Suggested packages:
  libipc-sharedcache-perl mailx tinyca
The following NEW packages will be installed:
  libaio1 libcgi-fast-perl libcgi-pm-perl libevent-core-2.1-7
  libevent-pthreads-2.1-7 libfcgi-perl libhtml-template-perl libmecab2
  mecab-ipadic mecab-ipadic-utf8 mecab-utils mysql-client-8.0
  mysql-client-core-8.0 mysql-server mysql-server-8.0 mysql-server-core-8.0
0 upgraded, 16 newly installed, 0 to remove and 0 not upgraded.
Need to get 36.7 MB of archives.
After this operation, 318 MB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 mysql-client-
core-8.0 amd64 8.0.42-0ubuntu0.20.04.1 [5,091 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 mysql-client-
8.0 amd64 8.0.42-0ubuntu0.20.04.1 [22.0 kB]
Get:3 http://us.archive.ubuntu.com/ubuntu focal/main amd64 libaio1 amd64 0.3.112
-5 [7,184 B]
Get:4 http://us.archive.ubuntu.com/ubuntu focal/main amd64 libevent-core-2.1-7 a
md64 2.1.11-stable-1 [89.1 kB]
Get:5 http://us.archive.ubuntu.com/ubuntu focal/main amd64 libevent-pthreads-2.1
-7 amd64 2.1.11-stable-1 [7,372 B]
Get:6 http://us.archive.ubuntu.com/ubuntu focal/main amd64 libmecab2 amd64 0.996
-10build1 [233 kB]
Get:7 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 mysql-server-
core-8.0 amd64 8.0.42-0ubuntu0.20.04.1 [22.9 MB]
Get:8 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 mysql-server-
8.0 amd64 8.0.42-0ubuntu0.20.04.1 [1,334 kB]
```

Alter root user password (root@localhost)

Once this is installed, we need to login and change the root user password, the first step in this process is the command “sudo mysql” this is how we enter the command prompt for MySQL it should look like this:



```
Mar 10 20:07
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

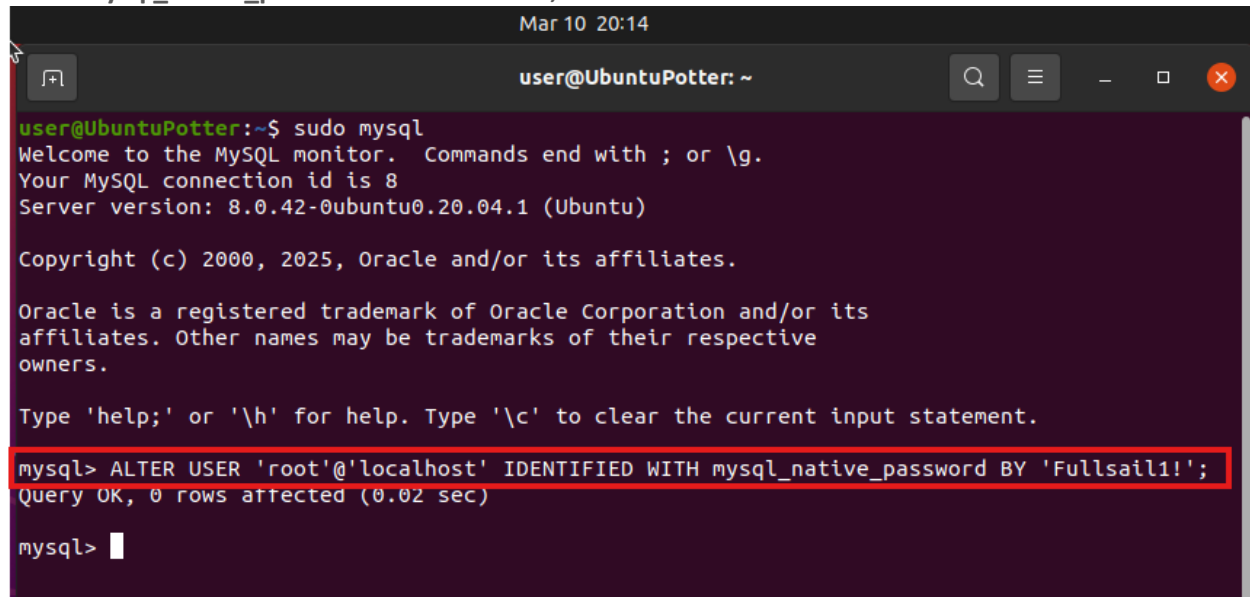
Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

We will then enter this command to change the Root password “ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Fullsail1!' ;”



```
Mar 10 20:14
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

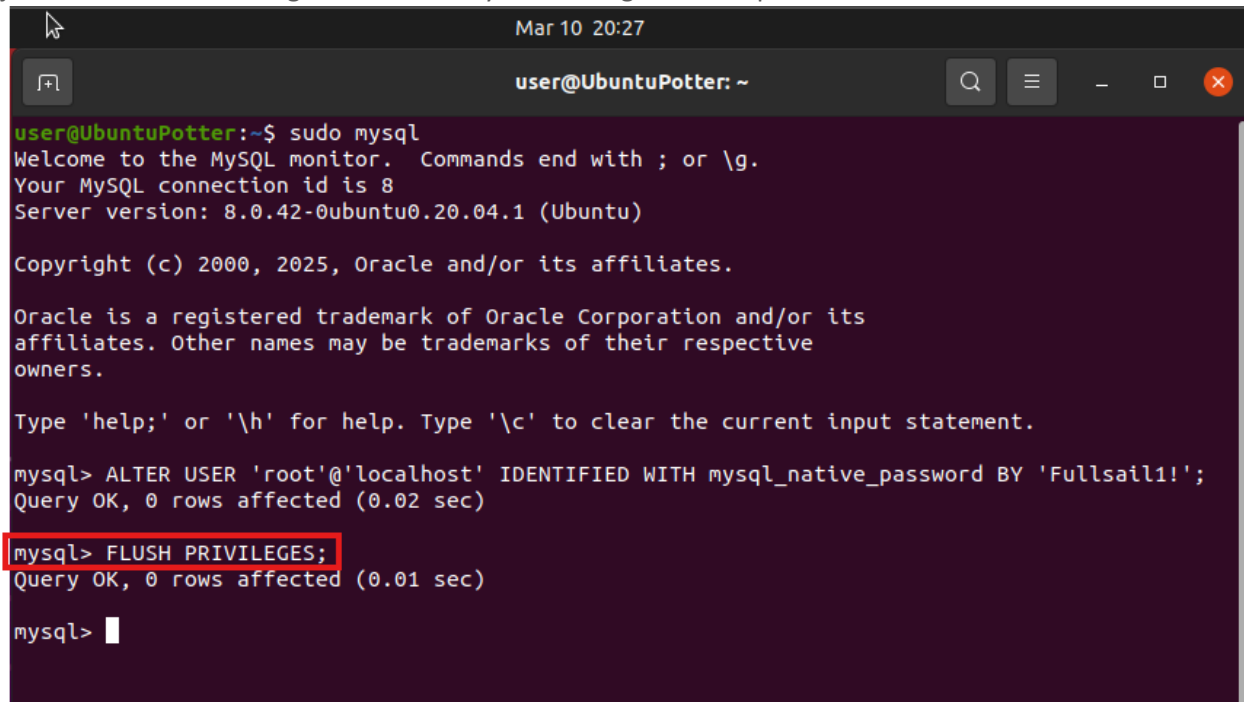
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Fullsail1!' ;
Query OK, 0 rows affected (0.02 sec)

mysql>
```


Flush Privileges

We then want to Use a command “**FLUSH PRIVILEGES;**” this command is used to apply the changes we just made. After entering this command you should get this output:

A terminal window titled 'user@UbuntuPotter: ~' with a date and time of 'Mar 10 20:27'. The terminal shows the execution of 'sudo mysql' which opens the MySQL monitor. The user enters 'ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Fullsail1!';' and receives 'Query OK, 0 rows affected (0.02 sec)'. Then, the user enters 'FLUSH PRIVILEGES;' which is highlighted with a red box, and receives 'Query OK, 0 rows affected (0.01 sec)'. The prompt 'mysql>' is shown at the end.

```
user@UbuntuPotter:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

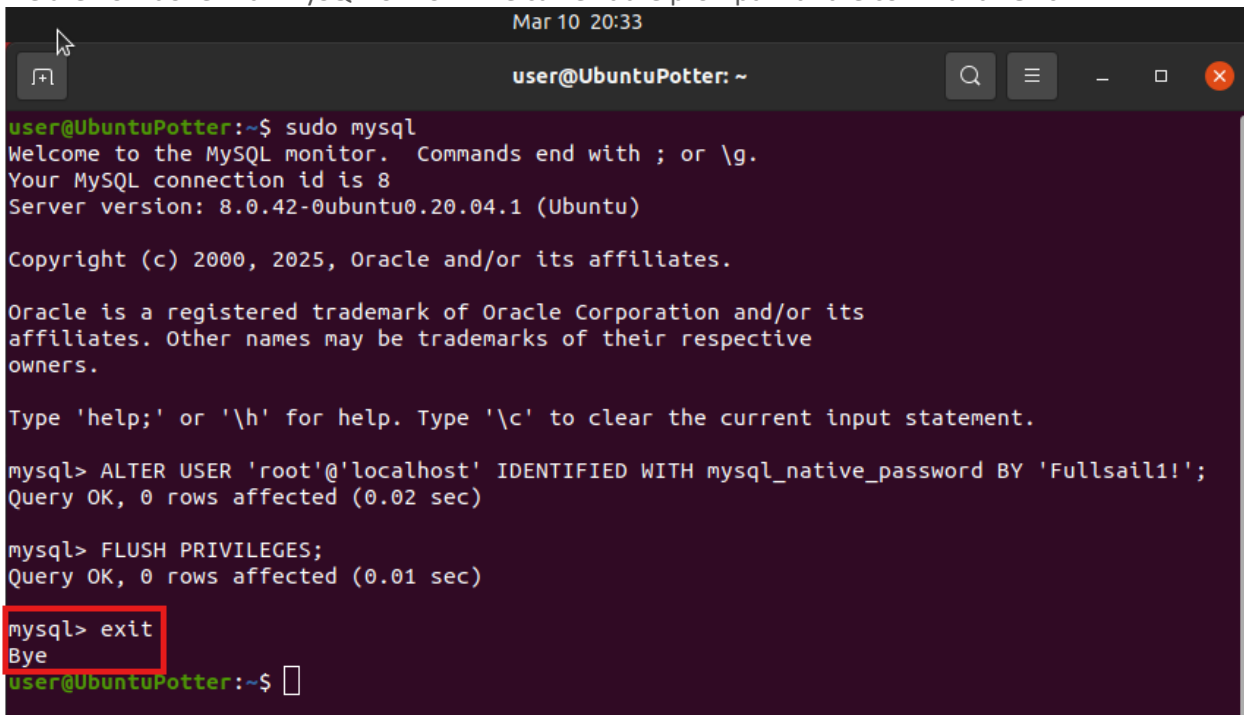
mysql> ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Fullsail1!';
Query OK, 0 rows affected (0.02 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql>
```

Exit MySQL

We are now done with MySQL for now. We can exit the prompt with the command “**exit**”

A terminal window titled 'user@UbuntuPotter: ~' with a date and time of 'Mar 10 20:33'. The terminal shows the execution of 'sudo mysql' which opens the MySQL monitor. The user enters 'ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Fullsail1!';' and receives 'Query OK, 0 rows affected (0.02 sec)'. Then, the user enters 'FLUSH PRIVILEGES;' and receives 'Query OK, 0 rows affected (0.01 sec)'. Finally, the user enters 'exit' which is highlighted with a red box, and receives 'Bye'. The terminal then shows the user back at the shell prompt 'user@UbuntuPotter:~\$'.

```
user@UbuntuPotter:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Fullsail1!';
Query OK, 0 rows affected (0.02 sec)

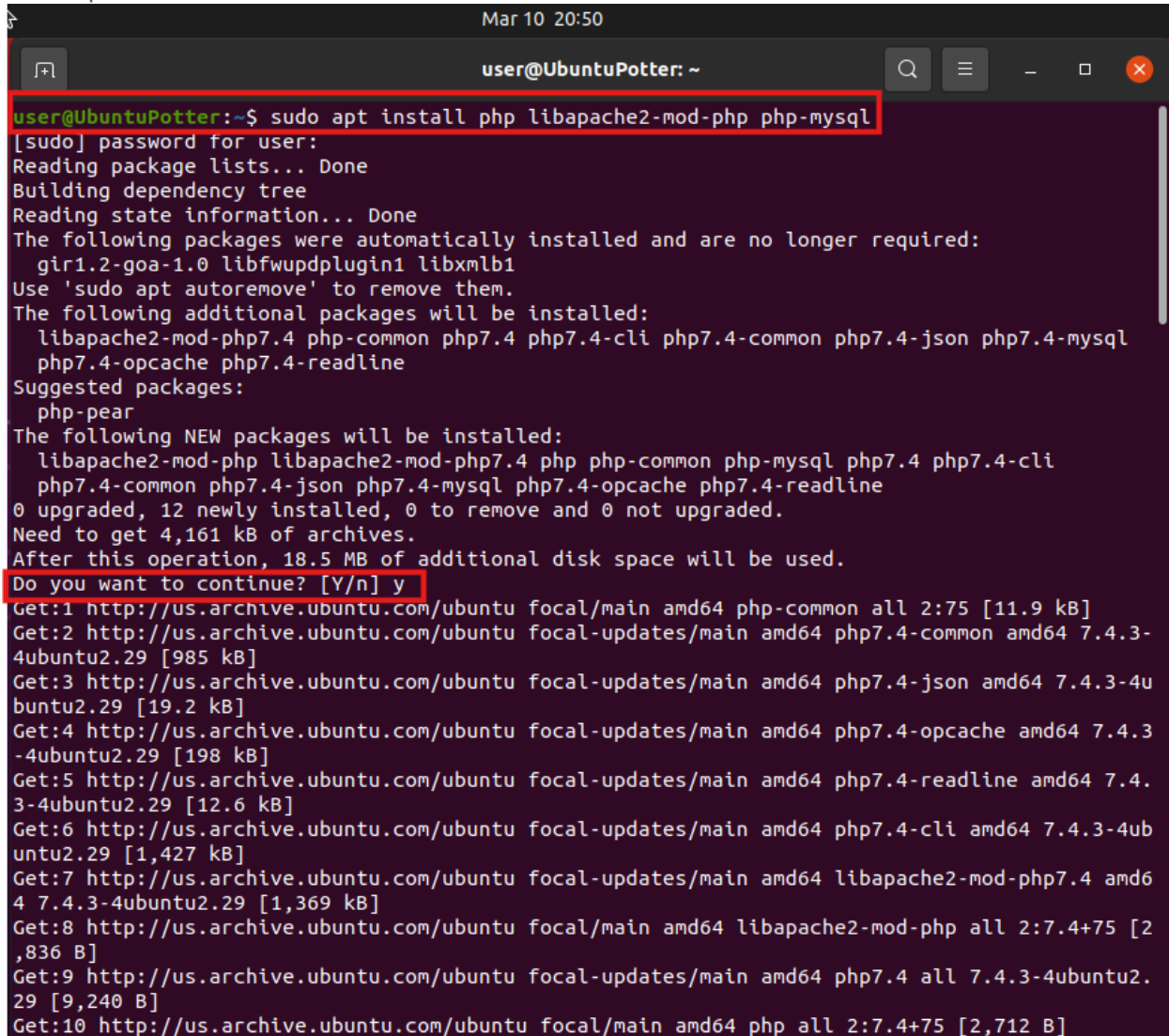
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> exit
Bye
user@UbuntuPotter:~$
```

Install PHP

Install Required PHP Libraries

We are now going to install PHP and some other required packages. We will do this by using the command “**sudo apt install php libapache2-mod-php php-mysql**” and when prompted input “**Y**” to confirm the install. The output should look similar to this:

A terminal window titled "user@UbuntuPotter: ~" with a timestamp of "Mar 10 20:50". The terminal shows the command "sudo apt install php libapache2-mod-php php-mysql" being executed. The output includes package lists, dependency trees, and a confirmation prompt "Do you want to continue? [Y/n] y". The terminal lists several packages to be installed, including libapache2-mod-php, php-common, php7.4, php7.4-cli, php7.4-common, php7.4-json, php7.4-mysql, php7.4-opcache, php7.4-readline, and php-pear. It also shows the disk space requirements and the sources from which the packages are being downloaded.

```
user@UbuntuPotter:~$ sudo apt install php libapache2-mod-php php-mysql
[sudo] password for user:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libapache2-mod-php7.4 php-common php7.4 php7.4-cli php7.4-common php7.4-json php7.4-mysql
  php7.4-opcache php7.4-readline
Suggested packages:
  php-pear
The following NEW packages will be installed:
  libapache2-mod-php libapache2-mod-php7.4 php php-common php-mysql php7.4 php7.4-cli
  php7.4-common php7.4-json php7.4-mysql php7.4-opcache php7.4-readline
0 upgraded, 12 newly installed, 0 to remove and 0 not upgraded.
Need to get 4,161 kB of archives.
After this operation, 18.5 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us.archive.ubuntu.com/ubuntu focal/main amd64 php-common all 2:75 [11.9 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 php7.4-common amd64 7.4.3-4ubuntu2.29 [985 kB]
Get:3 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 php7.4-json amd64 7.4.3-4ubuntu2.29 [19.2 kB]
Get:4 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 php7.4-opcache amd64 7.4.3-4ubuntu2.29 [198 kB]
Get:5 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 php7.4-readline amd64 7.4.3-4ubuntu2.29 [12.6 kB]
Get:6 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 php7.4-cli amd64 7.4.3-4ubuntu2.29 [1,427 kB]
Get:7 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 libapache2-mod-php7.4 amd64 7.4.3-4ubuntu2.29 [1,369 kB]
Get:8 http://us.archive.ubuntu.com/ubuntu focal/main amd64 libapache2-mod-php all 2:7.4+75 [2,836 B]
Get:9 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 php7.4 all 7.4.3-4ubuntu2.29 [9,240 B]
Get:10 http://us.archive.ubuntu.com/ubuntu focal/main amd64 php all 2:7.4+75 [2,712 B]
```

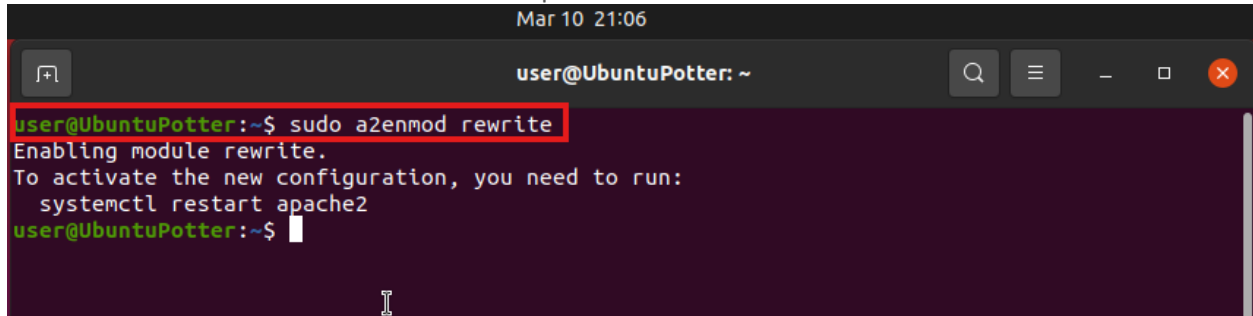
Install Required MySQL Libraries

We now need to install some additional extension for additional functionality. We need to input the following command “**sudo apt install php-curl php-gd php-xml php-mbstring php-xmldrpc php-zip php-soap php-intl -y**”. The output should look like to this:

```
Mar 10 21:01
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo apt install php-curl php-gd php-xml php-mbstring php-xmldrpc php-zip
php-soap php-intl -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libonig5 libxmlrpc-epi0 libzip5 php7.4-curl php7.4-gd php7.4-intl php7.4-mbstring
  php7.4-soap php7.4-xml php7.4-xmldrpc php7.4-zip
The following NEW packages will be installed:
  libonig5 libxmlrpc-epi0 libzip5 php-curl php-gd php-intl php-mbstring php-soap php-xml
  php-xmldrpc php-zip php7.4-curl php7.4-gd php7.4-intl php7.4-mbstring php7.4-soap
  php7.4-xml php7.4-xmldrpc php7.4-zip
0 upgraded, 19 newly installed, 0 to remove and 0 not upgraded.
Need to get 1,064 kB of archives.
After this operation, 3,823 kB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu focal/universe amd64 libonig5 amd64 6.9.4-1 [142 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu focal/universe amd64 libzip5 amd64 1.5.1-0ubuntu1 [46.7 kB]
Get:3 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 php7.4-curl amd64 7.4.3-4ubuntu2.29 [30.9 kB]
Get:4 http://us.archive.ubuntu.com/ubuntu focal/main amd64 php-curl all 2:7.4+75 [2,000 B]
Get:5 http://us.archive.ubuntu.com/ubuntu focal-updates/main amd64 php7.4-gd amd64 7.4.3-4ubuntu2.29 [27.9 kB]
Get:6 http://us.archive.ubuntu.com/ubuntu focal/main amd64 php-gd all 2:7.4+75 [2,000 B]
Get:7 http://us.archive.ubuntu.com/ubuntu focal-updates/universe amd64 php7.4-intl amd64 7.4.3-4ubuntu2.29 [126 kB]
Get:8 http://us.archive.ubuntu.com/ubuntu focal/universe amd64 php-intl all 2:7.4+75 [2,012 B]
Get:9 http://us.archive.ubuntu.com/ubuntu focal-updates/universe amd64 php7.4-mbstring amd64 7.4.3-4ubuntu2.29 [395 kB]
Get:10 http://us.archive.ubuntu.com/ubuntu focal/universe amd64 php-mbstring all 2:7.4+75 [2,012 B]
Get:11 http://us.archive.ubuntu.com/ubuntu focal-updates/universe amd64 php7.4-soap amd64 7.4.3-4ubuntu2.29 [115 kB]
```


Enable URL Rewrites (clean URLs)

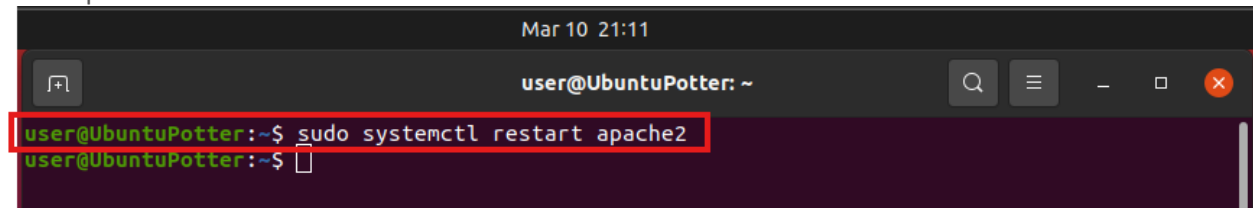
We now want to enable URL to enable them to be more user friendly. We can accomplish this with the command “**sudo a2enmod rewrite**”. The output should look like this:

A terminal window titled 'user@UbuntuPotter: ~' with a timestamp of 'Mar 10 21:06'. The command 'sudo a2enmod rewrite' is entered and highlighted with a red box. The output shows 'Enabling module rewrite.' followed by 'To activate the new configuration, you need to run: systemctl restart apache2'. The prompt returns to 'user@UbuntuPotter:~\$' with a cursor.

```
Mar 10 21:06
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo a2enmod rewrite
Enabling module rewrite.
To activate the new configuration, you need to run:
systemctl restart apache2
user@UbuntuPotter:~$
```

Restart Apache Service

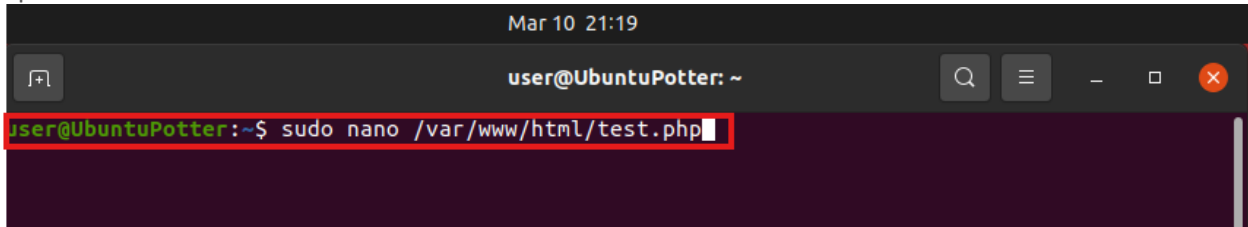
We then need to restart the Apache service using the command “**sudo systemctl restart apache2**” there is no output for this command:

A terminal window titled 'user@UbuntuPotter: ~' with a timestamp of 'Mar 10 21:11'. The command 'sudo systemctl restart apache2' is entered and highlighted with a red box. The prompt returns to 'user@UbuntuPotter:~\$' with a cursor.

```
Mar 10 21:11
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo systemctl restart apache2
user@UbuntuPotter:~$
```

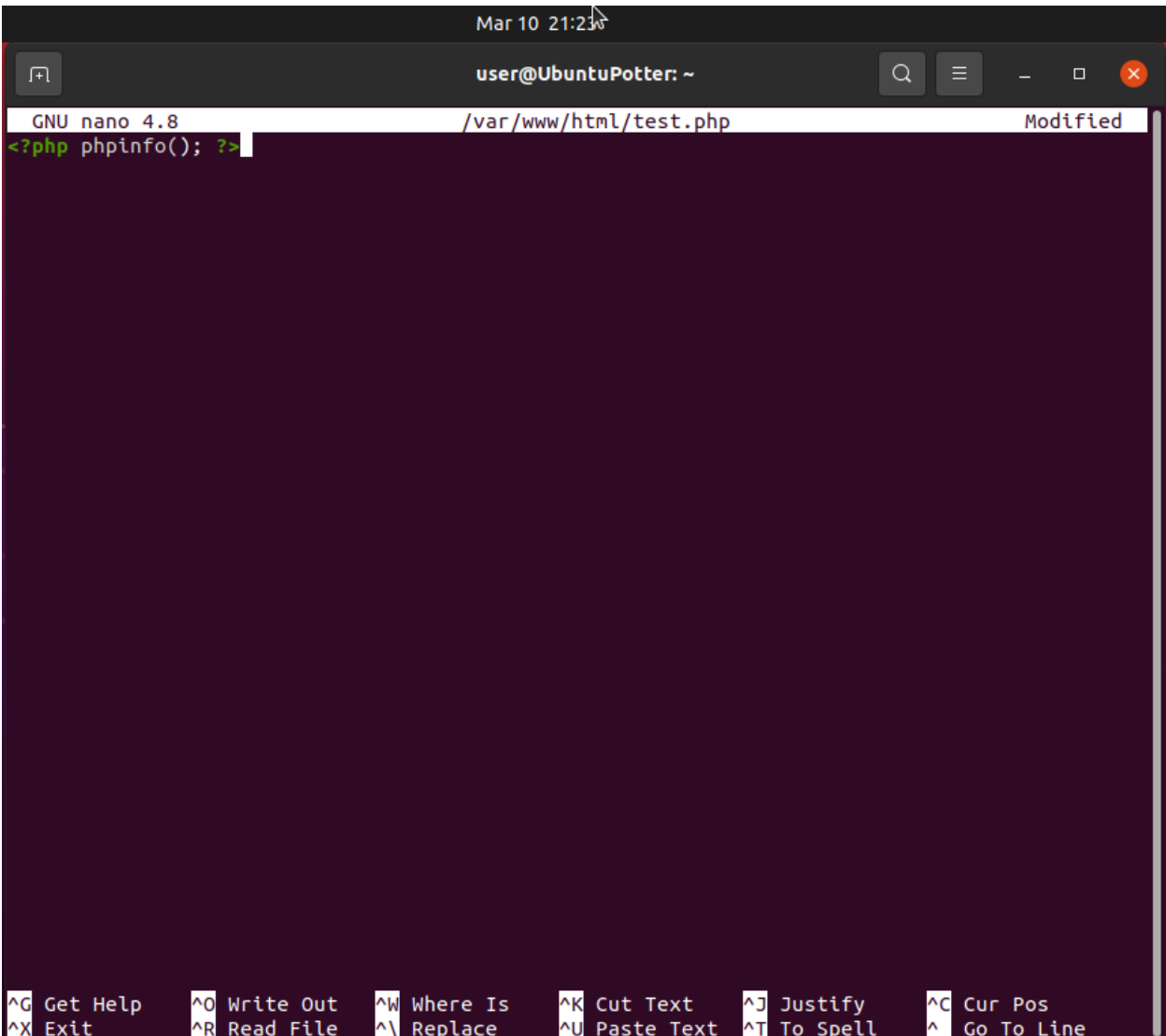
Create a test.php Web Page

Now let's create a test page, we will enter the command "**sudo nano /var/www/html/test.php**" this will open the nano text editor:

A terminal window titled "Mar 10 21:19" with the prompt "user@UbuntuPotter: ~". The command "sudo nano /var/www/html/test.php" is entered and highlighted with a red box. The terminal background is dark purple.

```
user@UbuntuPotter:~$ sudo nano /var/www/html/test.php
```

Once in the text editor, we then want to enter the following text "**<?php phpinfo(); ?>**" it should look like this:

A terminal window titled "Mar 10 21:23" showing the nano text editor. The editor title bar says "GNU nano 4.8 /var/www/html/test.php Modified". The text "<?php phpinfo(); ?>" is entered on the first line. The bottom of the screen shows nano editor shortcuts. The terminal background is dark purple.

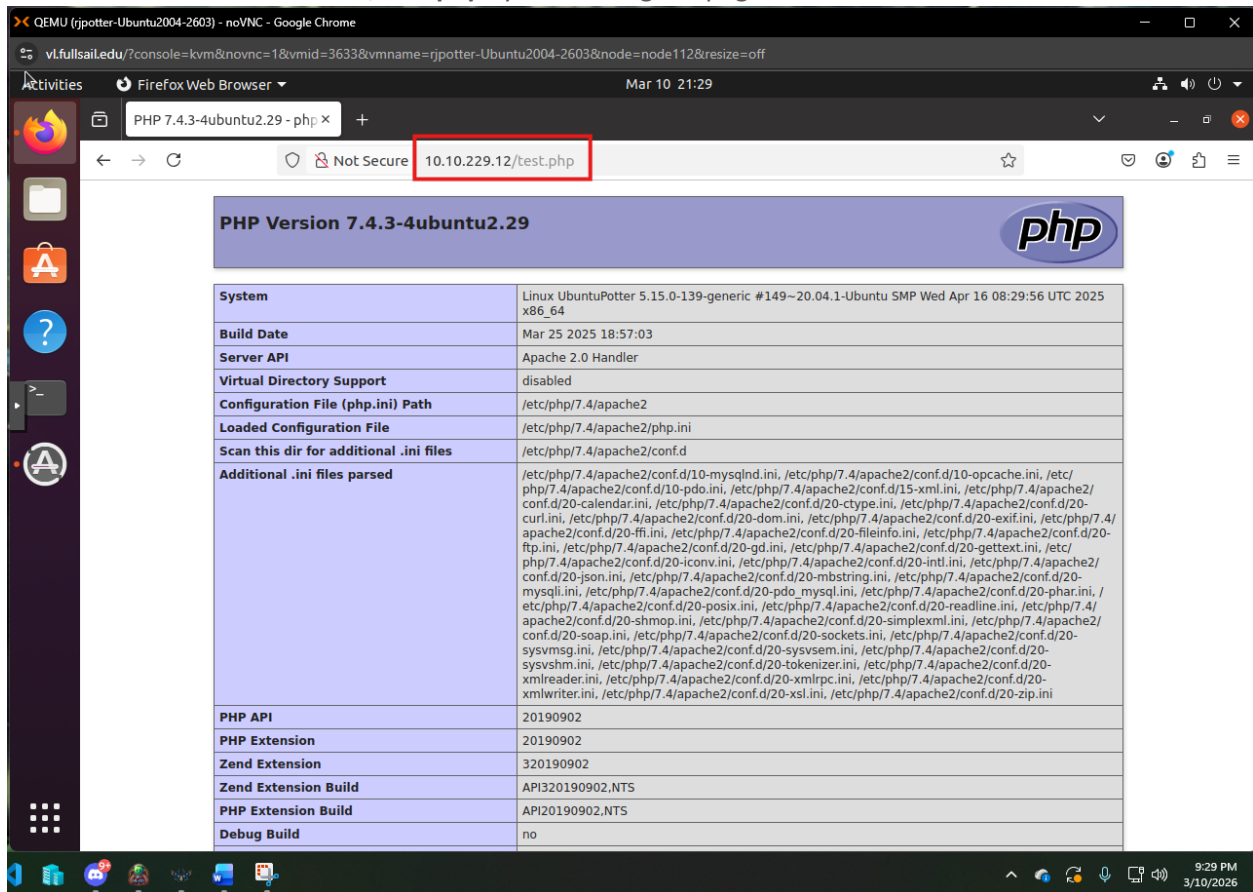
```
GNU nano 4.8 /var/www/html/test.php Modified
<?php phpinfo(); ?>
```

^G Get Help	^O Write Out	^W Where Is	^K Cut Text	^J Justify	^C Cur Pos
^X Exit	^R Read File	^I Replace	^U Paste Text	^T To Spell	^_ Go To Line

Once this is done. Press "**Ctrl-X**", Press "**y**", and then press "**Enter**"

Test the test.php Web Page

We now want to test the page we just made. We will do this by opening a FireFox Browser and enter in the address bar “10.10.229.12/test.php” you should get a page like this:



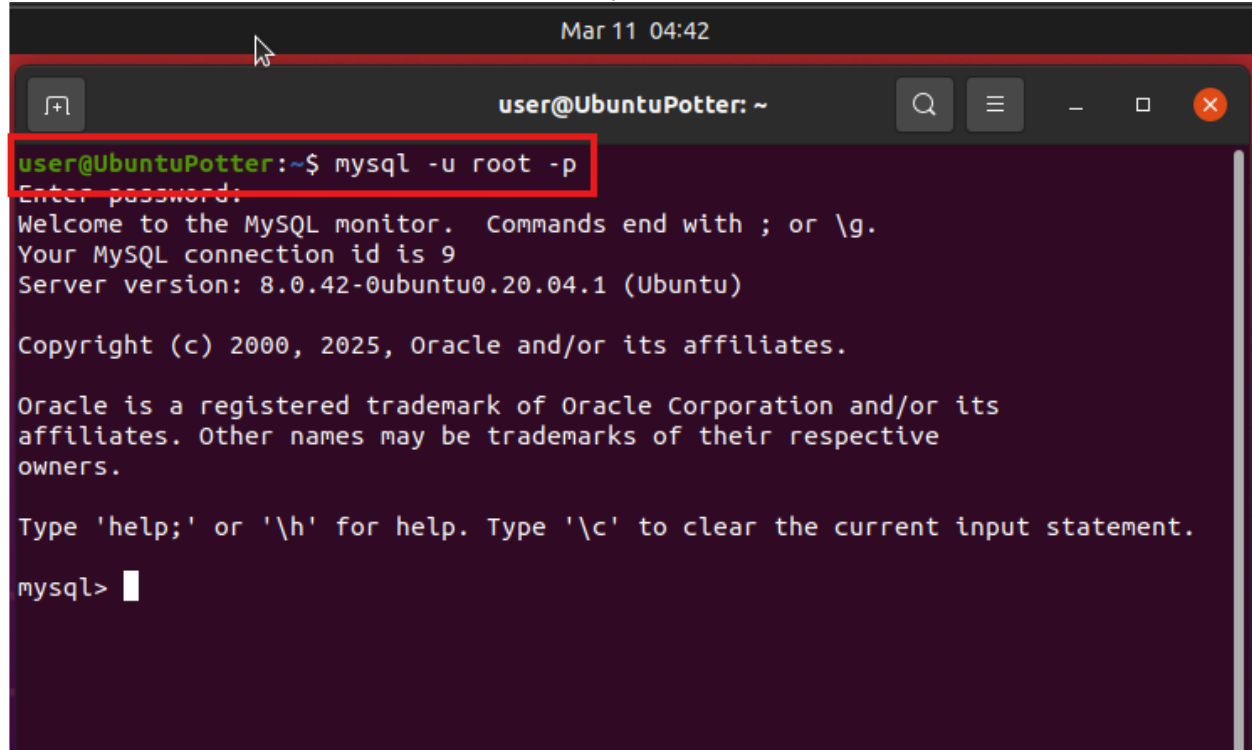
The screenshot shows a web browser window with the address bar displaying "10.10.229.12/test.php". The page content is titled "PHP Version 7.4.3-4ubuntu2.29" and features the PHP logo. Below the title is a table with system and configuration details.

PHP Version 7.4.3-4ubuntu2.29	
System	Linux UbuntuPotter 5.15.0-139-generic #149~20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64
Build Date	Mar 25 2025 18:57:03
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.4/apache2
Loaded Configuration File	/etc/php/7.4/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/7.4/apache2/conf.d
Additional .ini files parsed	/etc/php/7.4/apache2/conf.d/10-mysqld.ini, /etc/php/7.4/apache2/conf.d/10-opcache.ini, /etc/php/7.4/apache2/conf.d/10-pdo.ini, /etc/php/7.4/apache2/conf.d/15-xml.ini, /etc/php/7.4/apache2/conf.d/20-calendar.ini, /etc/php/7.4/apache2/conf.d/20-ctype.ini, /etc/php/7.4/apache2/conf.d/20-curl.ini, /etc/php/7.4/apache2/conf.d/20-dom.ini, /etc/php/7.4/apache2/conf.d/20-exif.ini, /etc/php/7.4/apache2/conf.d/20-ffi.ini, /etc/php/7.4/apache2/conf.d/20-fileinfo.ini, /etc/php/7.4/apache2/conf.d/20-ftp.ini, /etc/php/7.4/apache2/conf.d/20-gd.ini, /etc/php/7.4/apache2/conf.d/20-gettext.ini, /etc/php/7.4/apache2/conf.d/20-iconv.ini, /etc/php/7.4/apache2/conf.d/20-intl.ini, /etc/php/7.4/apache2/conf.d/20-json.ini, /etc/php/7.4/apache2/conf.d/20-mbstring.ini, /etc/php/7.4/apache2/conf.d/20-mysqli.ini, /etc/php/7.4/apache2/conf.d/20-pdo_mysql.ini, /etc/php/7.4/apache2/conf.d/20-phar.ini, /etc/php/7.4/apache2/conf.d/20-posix.ini, /etc/php/7.4/apache2/conf.d/20-readline.ini, /etc/php/7.4/apache2/conf.d/20-shmop.ini, /etc/php/7.4/apache2/conf.d/20-simplexml.ini, /etc/php/7.4/apache2/conf.d/20-soap.ini, /etc/php/7.4/apache2/conf.d/20-sockets.ini, /etc/php/7.4/apache2/conf.d/20-sysvmsg.ini, /etc/php/7.4/apache2/conf.d/20-sysvsem.ini, /etc/php/7.4/apache2/conf.d/20-sysvshm.ini, /etc/php/7.4/apache2/conf.d/20-tokenizer.ini, /etc/php/7.4/apache2/conf.d/20-xmlreader.ini, /etc/php/7.4/apache2/conf.d/20-xmlrpc.ini, /etc/php/7.4/apache2/conf.d/20-xmlwriter.ini, /etc/php/7.4/apache2/conf.d/20-xsl.ini, /etc/php/7.4/apache2/conf.d/20-zip.ini
PHP API	20190902
PHP Extension	20190902
Zend Extension	320190902
Zend Extension Build	API320190902.NTS
PHP Extension Build	API20190902.NTS
Debug Build	no

Database Configuration in MySQL

Log into MySQL

In this Section we will be setting up our MySQL for the WordPress site. So First we need to Login to MySQL we do this with the command “**mysql -u root -p**” you will then be prompted for a password that we setup earlier and can be found in this document. The output looks like below:

A terminal window titled 'user@UbuntuPotter: ~' with a timestamp 'Mar 11 04:42'. The command 'mysql -u root -p' is entered and highlighted with a red box. The prompt 'Enter password:' is shown. The terminal output includes: 'Welcome to the MySQL monitor. Commands end with ; or \g.', 'Your MySQL connection id is 9', 'Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)', 'Copyright (c) 2000, 2025, Oracle and/or its affiliates.', 'Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.', 'Type \'help;\' or \'\\h\' for help. Type \'\\c\' to clear the current input statement.', and the prompt 'mysql>' with a cursor.

```
Mar 11 04:42
user@UbuntuPotter: ~
user@UbuntuPotter:~$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\\h' for help. Type '\\c' to clear the current input statement.
mysql> █
```

Create MySQL WordPress Database (WordPressDB)

Now Lets create the WordPress Database, this is where the options and data for the website will be stored. We will create the Database using the command "**CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;**". Your output should look like this:

```
Mar 11 04:54
user@UbuntuPotter: ~
user@UbuntuPotter:~$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

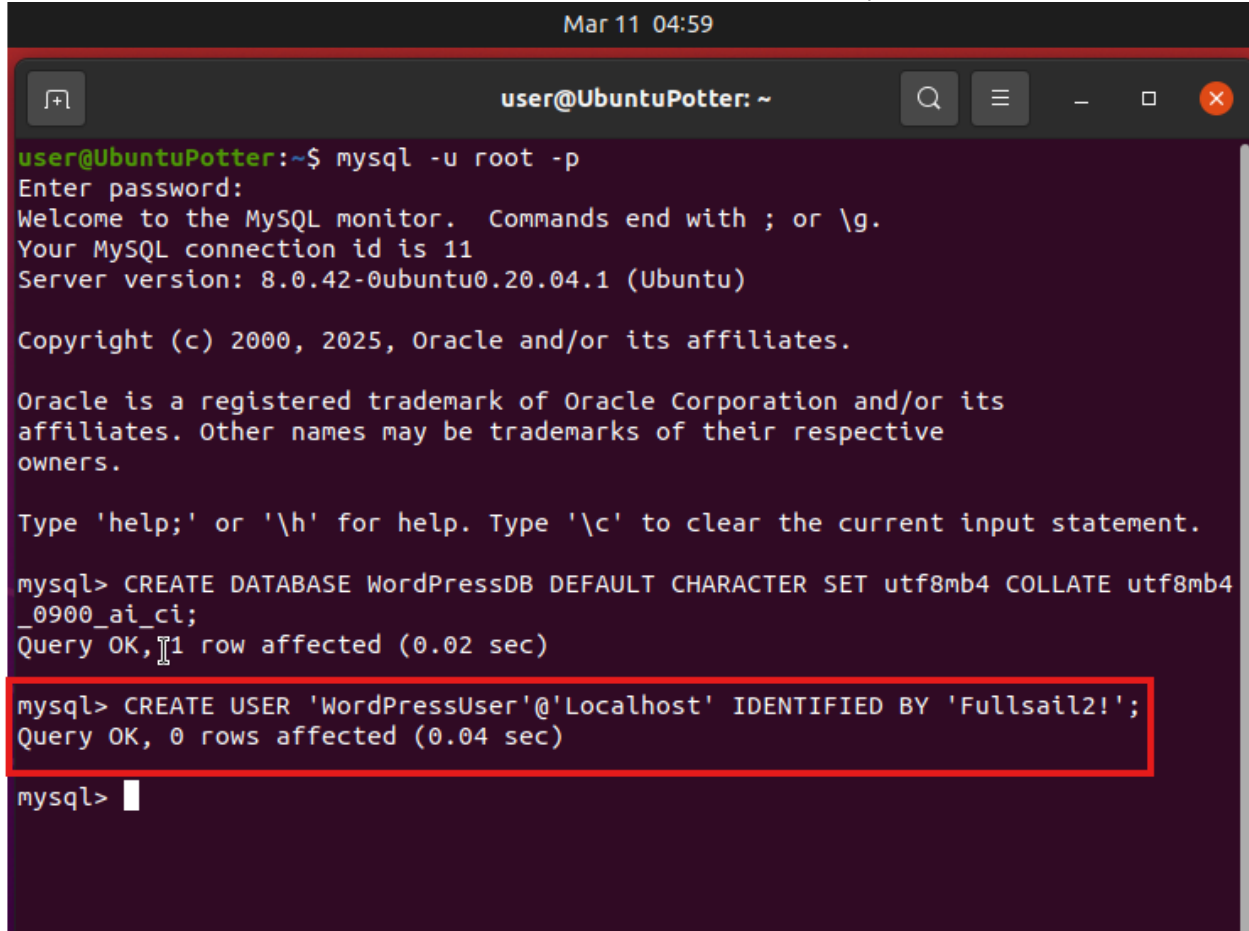
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4
_0900_ai_ci;
Query OK, 1 row affected (0.02 sec)

mysql> 
```

Create MySQL WordPress User (WordPressUser)

We will now create a user account for the wordpress site to use. The command to do this is “**CREATE USER ‘WordPressUser’@‘Localhost’ IDENTIFIED BY ‘Fullsail2!’;**” The output should look like this:

A terminal window titled 'user@UbuntuPotter: ~' with a timestamp 'Mar 11 04:59'. The window shows a MySQL command-line session. The user runs 'mysql -u root -p', enters a password, and is greeted with 'Welcome to the MySQL monitor. Commands end with ; or \g. Your MySQL connection id is 11. Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)'. The user then runs 'CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;' and receives 'Query OK, 1 row affected (0.02 sec)'. Next, the user runs 'CREATE USER 'WordPressUser'@'Localhost' IDENTIFIED BY 'Fullsail2!';', which is highlighted with a red box, and receives 'Query OK, 0 rows affected (0.04 sec)'. The prompt 'mysql>' is shown at the bottom.

```
Mar 11 04:59

user@UbuntuPotter: ~$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

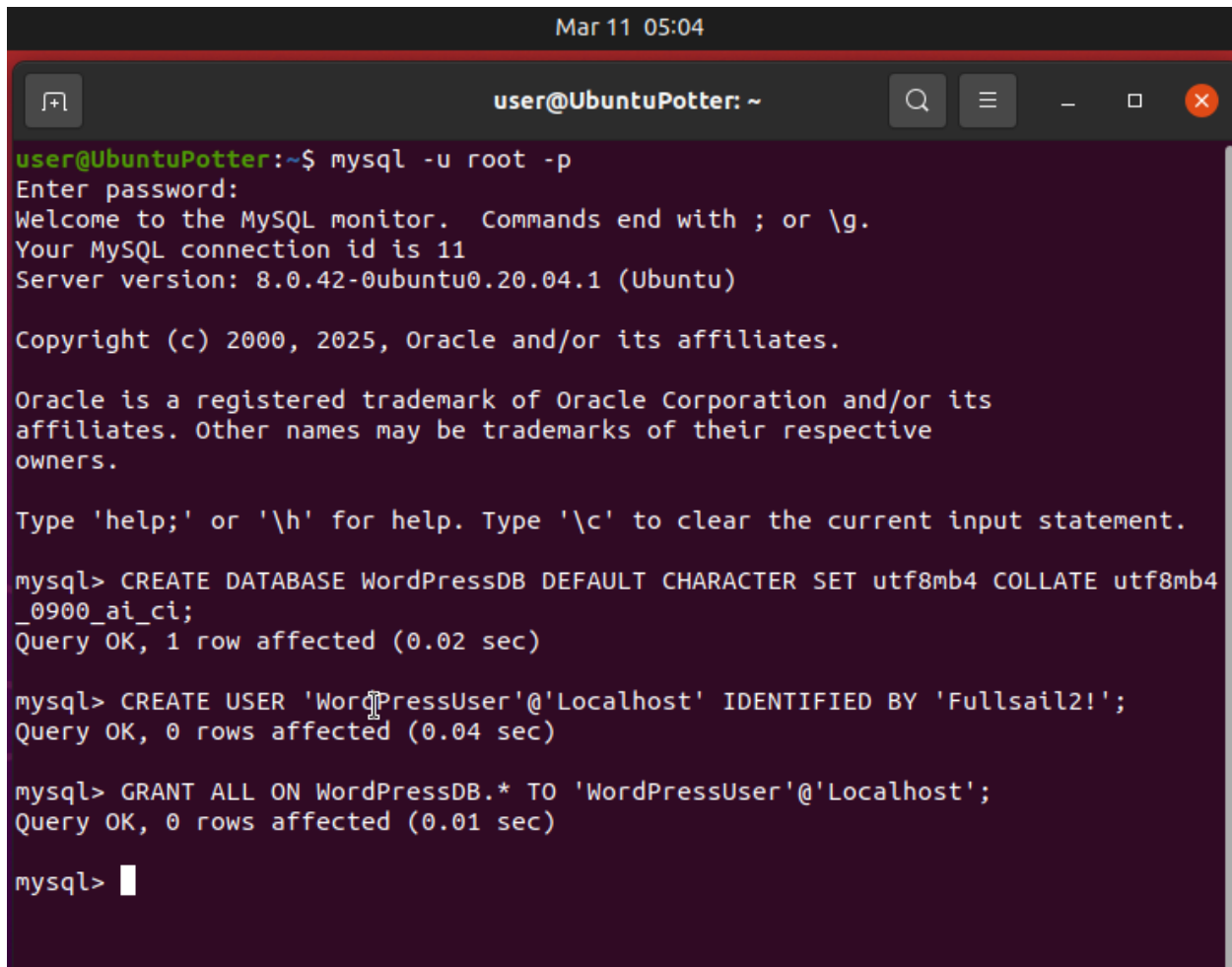
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4
_0900_ai_ci;
Query OK, 1 row affected (0.02 sec)

mysql> CREATE USER 'WordPressUser'@'Localhost' IDENTIFIED BY 'Fullsail2!';
Query OK, 0 rows affected (0.04 sec)

mysql>
```

Grant Privileges to the WordPress Database (WordPressDB) to WordPress User (WordPressUser)
We then need to Grant our new user with permissions to make changes to the WordPressDB. We will do this using the command **"GRANT ALL ON WordPressDB.* TO 'WordPressUser'@'localhost';"**. The output should look like this:

A terminal window titled "user@UbuntuPotter: ~" with a dark background. The window shows the execution of MySQL commands. The user runs "mysql -u root -p", enters a password, and is greeted by the MySQL monitor. They then run three commands: "CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;", "CREATE USER 'WordPressUser'@'Localhost' IDENTIFIED BY 'Fullsail2!';", and "GRANT ALL ON WordPressDB.* TO 'WordPressUser'@'Localhost';". The output for each command is "Query OK, 1 row affected (0.02 sec)", "Query OK, 0 rows affected (0.04 sec)", and "Query OK, 0 rows affected (0.01 sec)" respectively. The terminal ends with the "mysql>" prompt.

```
Mar 11 05:04
user@UbuntuPotter: ~
user@UbuntuPotter:~$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4
_0900_ai_ci;
Query OK, 1 row affected (0.02 sec)

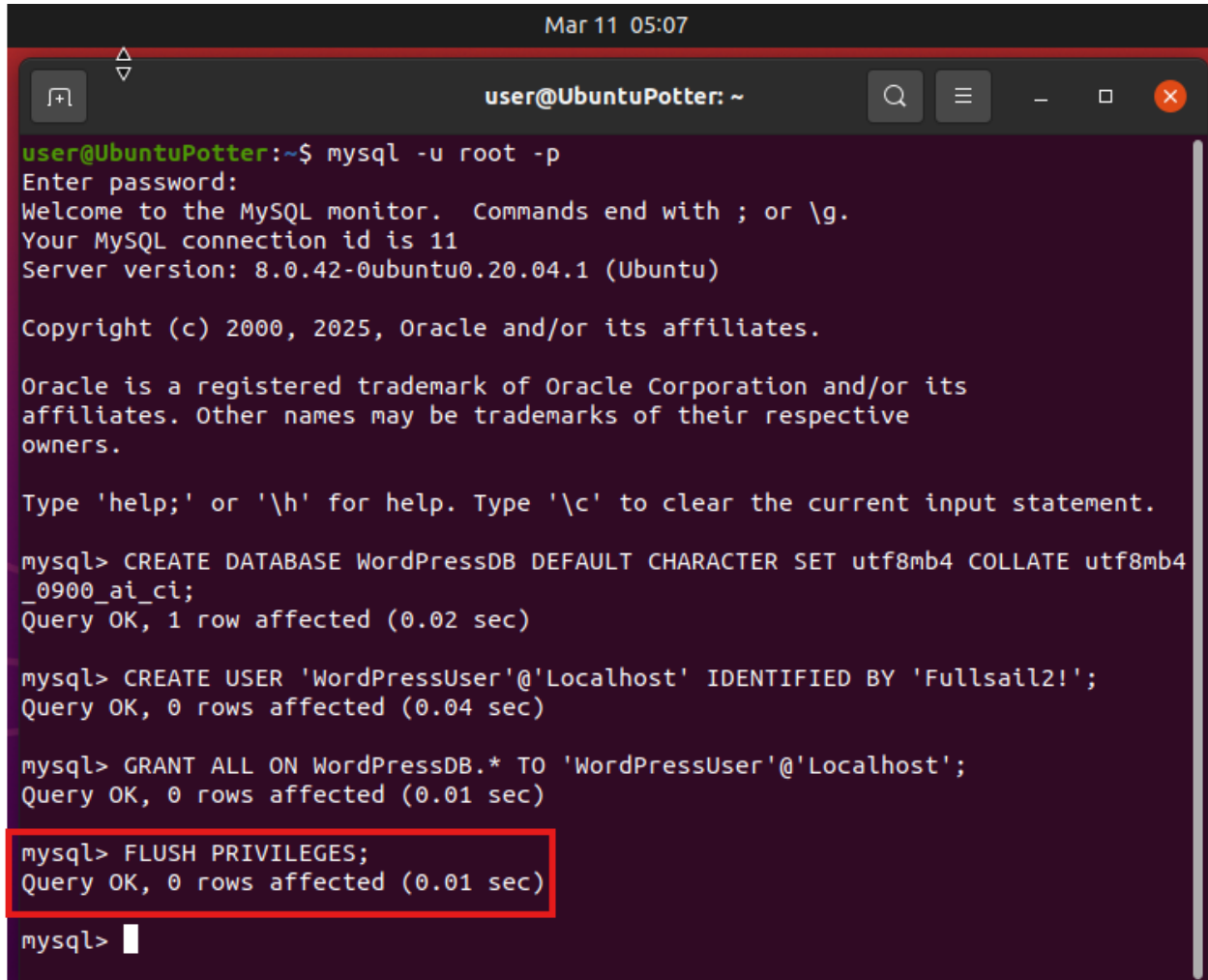
mysql> CREATE USER 'WordPressUser'@'Localhost' IDENTIFIED BY 'Fullsail2!';
Query OK, 0 rows affected (0.04 sec)

mysql> GRANT ALL ON WordPressDB.* TO 'WordPressUser'@'Localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> 
```

Flush Privileges

Now for all these changes to take effect we need to use the command “**FLUSH PRIVILEGES;**”. The output should look like this:

A terminal window titled "user@UbuntuPotter: ~" with a date and time of "Mar 11 05:07". The terminal shows a MySQL session where the user 'root' connects. The output includes the MySQL welcome message, connection ID 11, and server version 8.0.42-0ubuntu0.20.04.1. The user then executes three commands: 'CREATE DATABASE WordPressDB', 'CREATE USER 'WordPressUser'@'localhost'', and 'GRANT ALL ON WordPressDB.* TO 'WordPressUser'@'localhost''. The final command, 'FLUSH PRIVILEGES;', is highlighted with a red box, and its output is also shown.

```
user@UbuntuPotter:~$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4
_0900_ai_ci;
Query OK, 1 row affected (0.02 sec)

mysql> CREATE USER 'WordPressUser'@'localhost' IDENTIFIED BY 'Fullsail2!';
Query OK, 0 rows affected (0.04 sec)

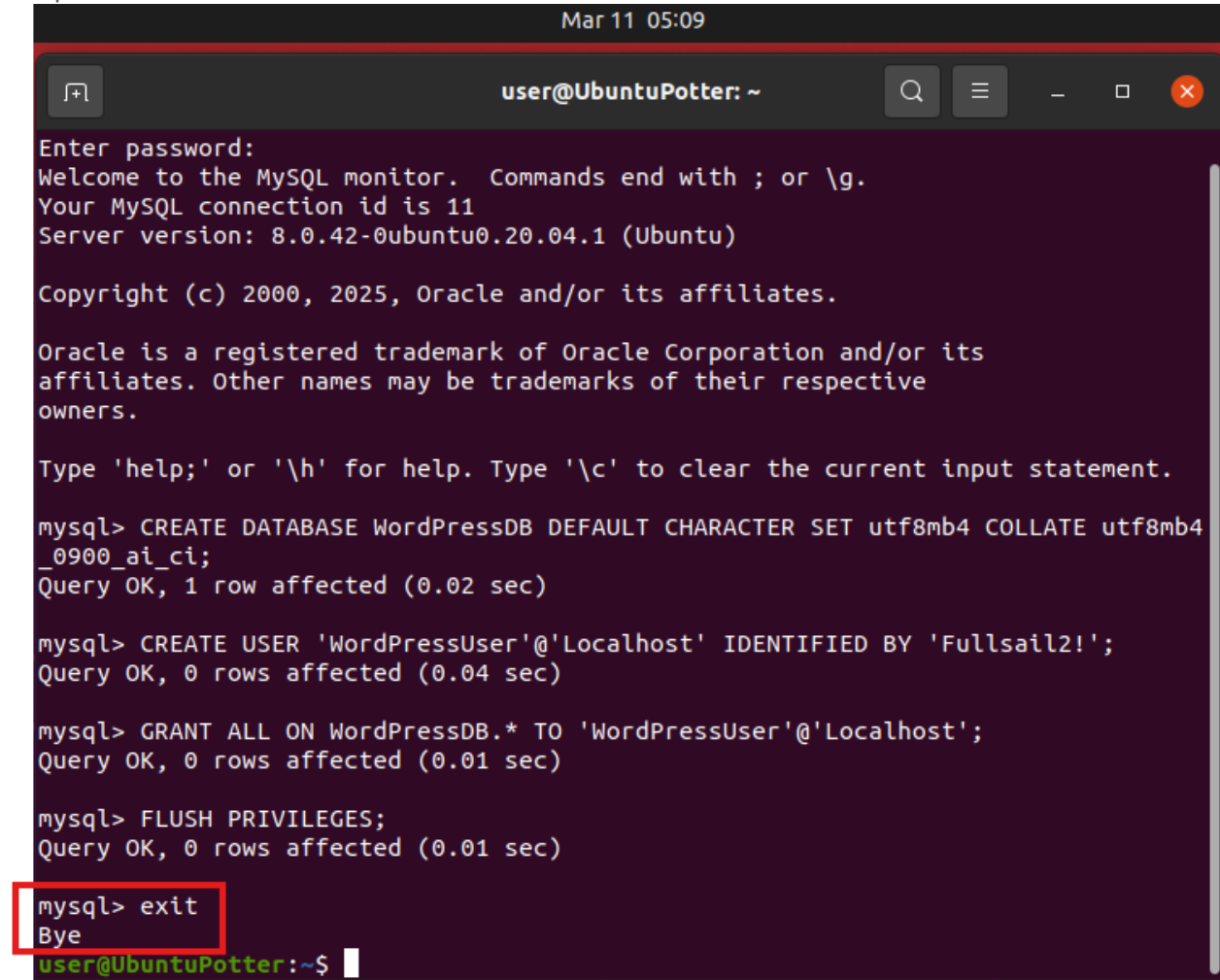
mysql> GRANT ALL ON WordPressDB.* TO 'WordPressUser'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> 
```


Exit MySQL

With this final step complete we can now safely Exit the MySQL prompt using the command “**exit**”. The Output of this command should look like this:



```
Mar 11 05:09
user@UbuntuPotter: ~
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.42-0ubuntu0.20.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE WordPressDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4
_0900_ai_ci;
Query OK, 1 row affected (0.02 sec)

mysql> CREATE USER 'WordPressUser'@'Localhost' IDENTIFIED BY 'Fullsail2!';
Query OK, 0 rows affected (0.04 sec)

mysql> GRANT ALL ON WordPressDB.* TO 'WordPressUser'@'Localhost';
Query OK, 0 rows affected (0.01 sec)

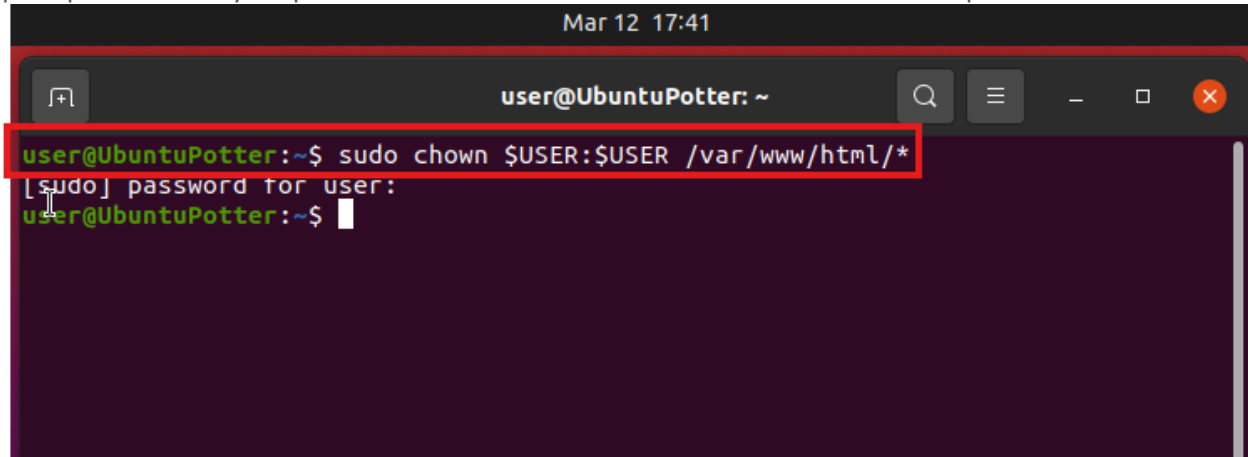
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> exit
Bye
user@UbuntuPotter:~$
```

Install WordPress

Grant Permission to /var/www/html/ Directory to WordPress User

We will now grant permissions for our WordPressUser to be able to modify files in the the /var/www/html/ directory. We will do this with the command “**sudo chown \$USER:\$USER /var/www/html/***”. You will be prompted to enter your password. Enter it and Press Enter. There is no other output:

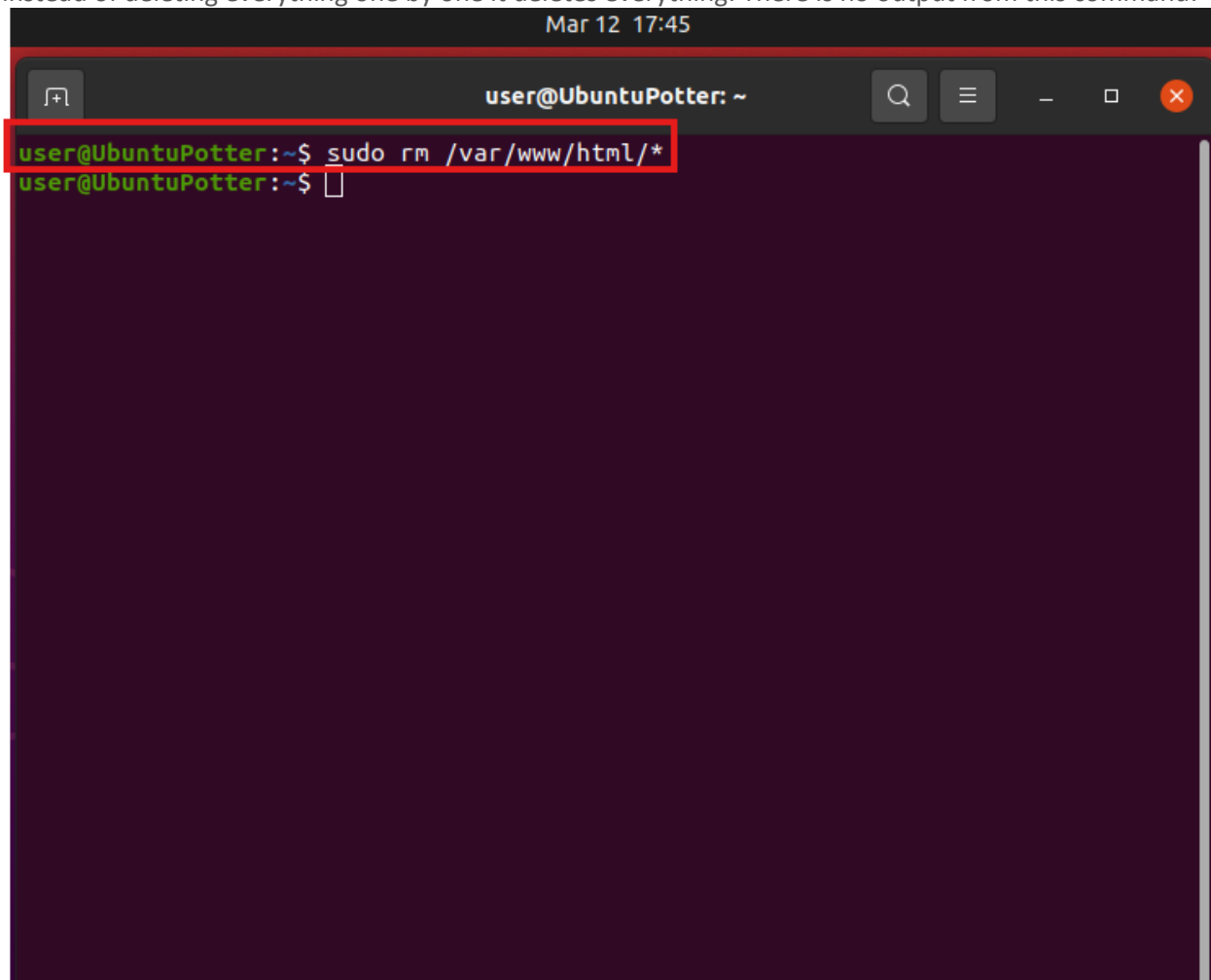


```
Mar 12 17:41
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo chown $USER:$USER /var/www/html/*
[sudo] password for user:
user@UbuntuPotter:~$
```

Delete Files from /var/www/html/ Directory

We now want to delete all the contents of this folder including our test file. We will accomplish this with the command “**sudo rm /var/www/html/***”. The star at the end is a wildcard and means Everything, so of

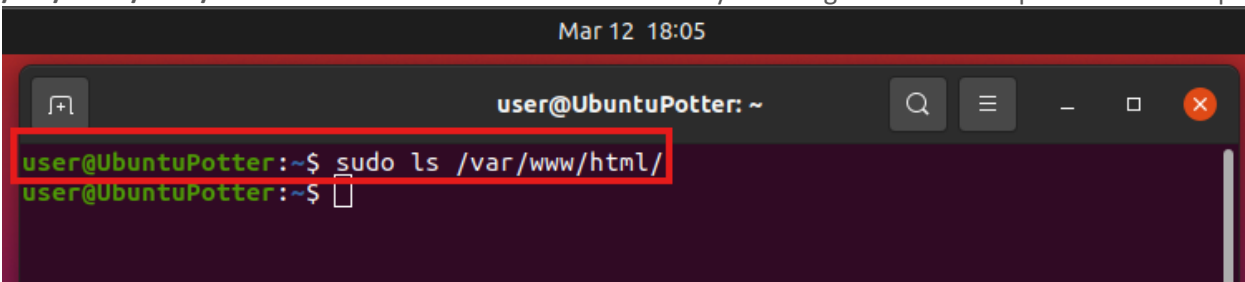
instead of deleting everything one by one it deletes everything. There is no output from this command:



A terminal window titled "user@UbuntuPotter: ~" with a timestamp of "Mar 12 17:45". The window shows a command prompt where the user has entered `sudo rm /var/www/html/*`. The command is highlighted with a red box. The prompt then changes to `user@UbuntuPotter:~$` with a cursor, indicating the command has been executed without any visible output.

Verify /var/www/html/ Directory is Empty

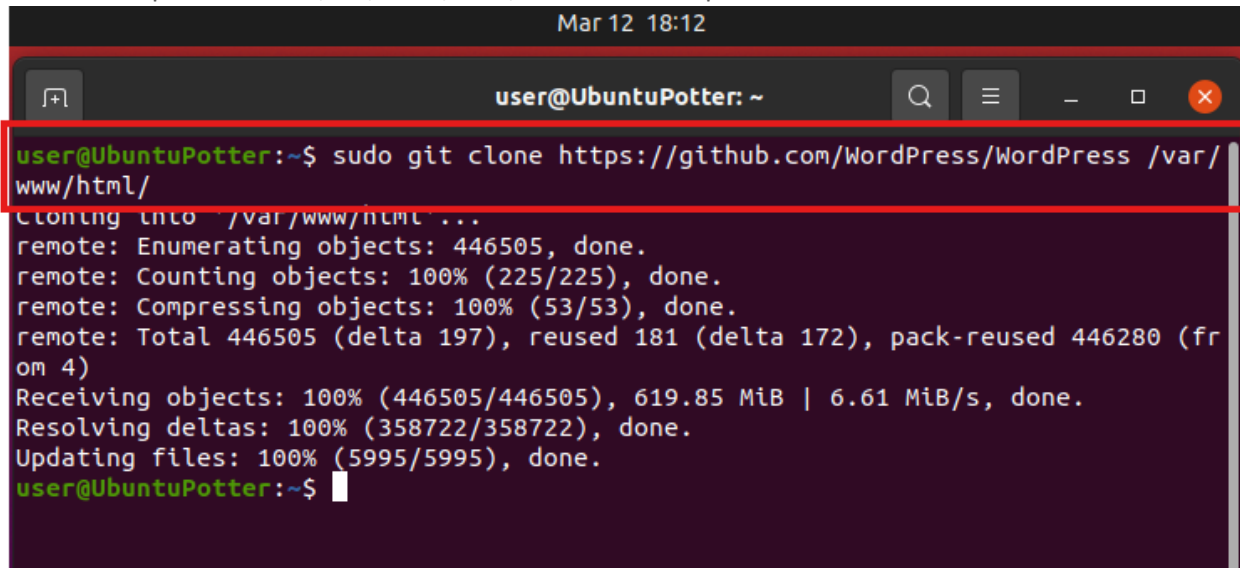
We will not Verify that the Directory after the last step we will do this with the command “**Sudo ls /var/www/html/**”. This will list the contents of the directory. Nothing Should show up because its empty.



A terminal window titled "user@UbuntuPotter: ~" with a timestamp of "Mar 12 18:05". The window shows a command prompt where the user has entered `sudo ls /var/www/html/`. The command is highlighted with a red box. The prompt then changes to `user@UbuntuPotter:~$` with a cursor, indicating the command has been executed without any visible output.

Clone WordPress to /var/www/html/ Directory

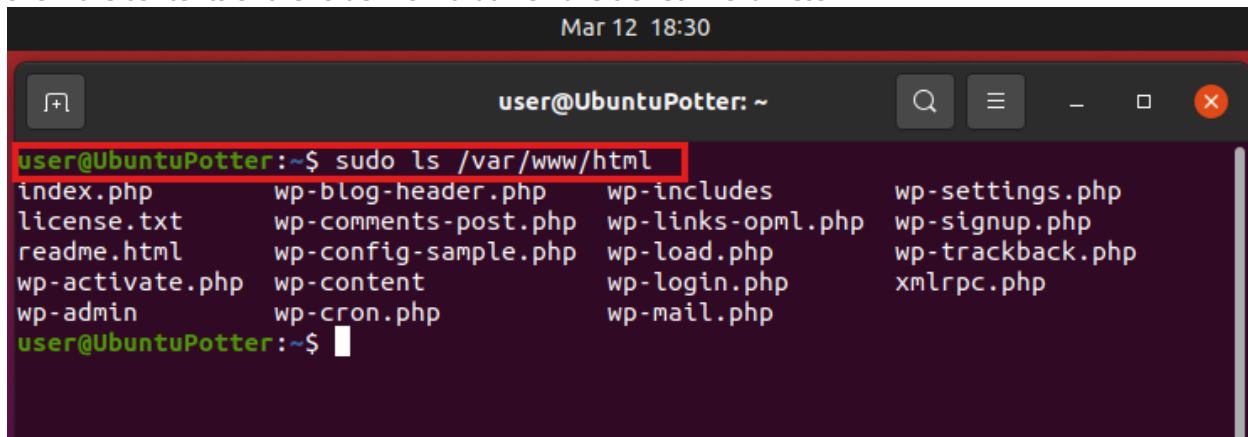
We will now Clone Wordpress from the Github for it using the command “**sudo git clone https://github.com/WordPress/WordPress /var/www/html/**”. This will pull the contents of the Github we entered and place it in the /var/www/html/ folder. The output should look like this:



```
Mar 12 18:12
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo git clone https://github.com/WordPress/WordPress /var/www/html/
Cloning into '/var/www/html' ...
remote: Enumerating objects: 446505, done.
remote: Counting objects: 100% (225/225), done.
remote: Compressing objects: 100% (53/53), done.
remote: Total 446505 (delta 197), reused 181 (delta 172), pack-reused 446280 (from 4)
Receiving objects: 100% (446505/446505), 619.85 MiB | 6.61 MiB/s, done.
Resolving deltas: 100% (358722/358722), done.
Updating files: 100% (5995/5995), done.
user@UbuntuPotter:~$
```

Verify /var/www/html/ Directory Contains WordPress Files

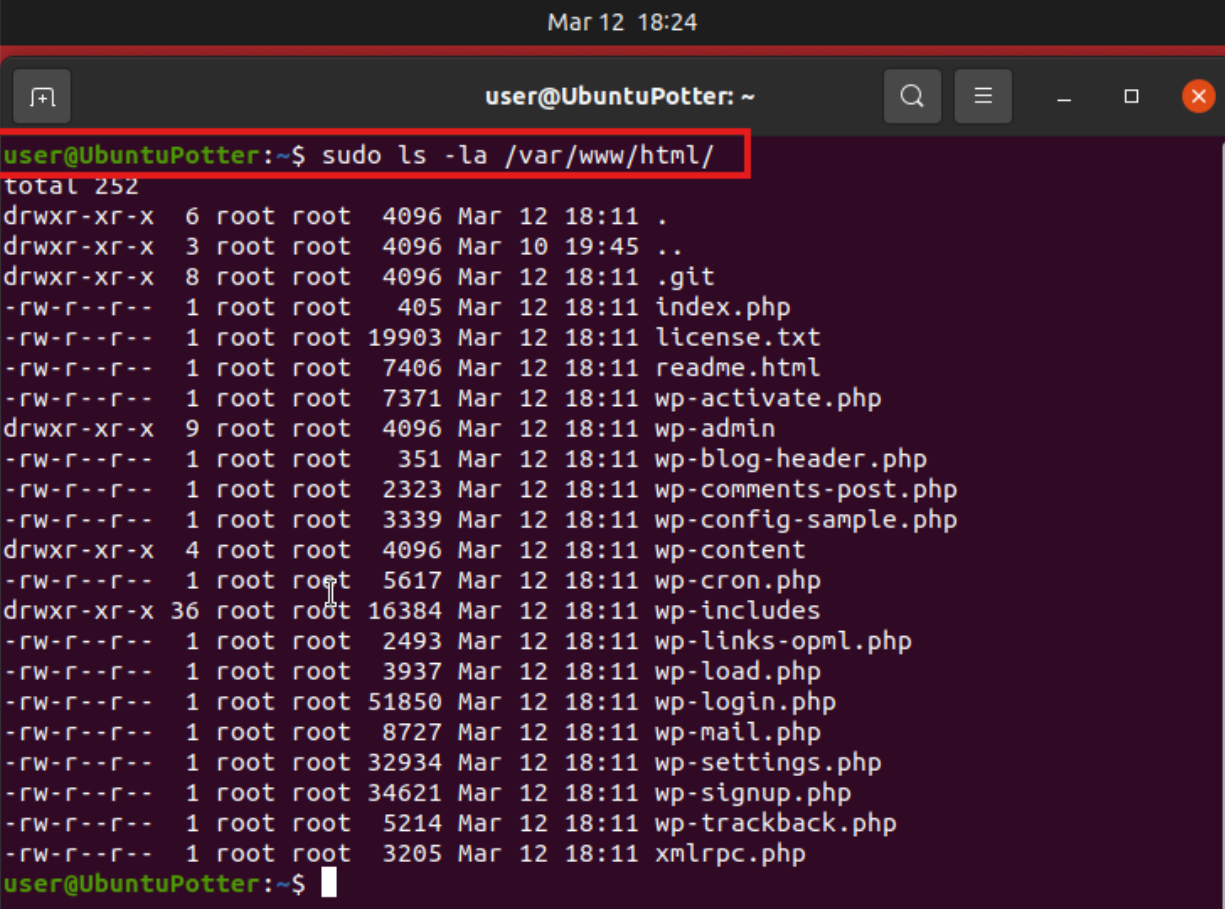
We now want to verify that the process work we will use the command “**sudo ls /var/www/html**” This will show the contents of the folder now that we have cloned WordPress:



```
Mar 12 18:30
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo ls /var/www/html
index.php      wp-blog-header.php  wp-includes        wp-settings.php
license.txt    wp-comments-post.php wp-links-opml.php  wp-signup.php
readme.html    wp-config-sample.php wp-load.php         wp-trackback.php
wp-activate.php wp-content           wp-login.php        xmlrpc.php
wp-admin       wp-cron.php          wp-mail.php
user@UbuntuPotter:~$
```

Verify Permissions on /var/www/html/ Directory

We can now verify the permissions on the files we just downloaded. We will do this with the command “`sudo ls -la /var/www/html/`”. You will see this:

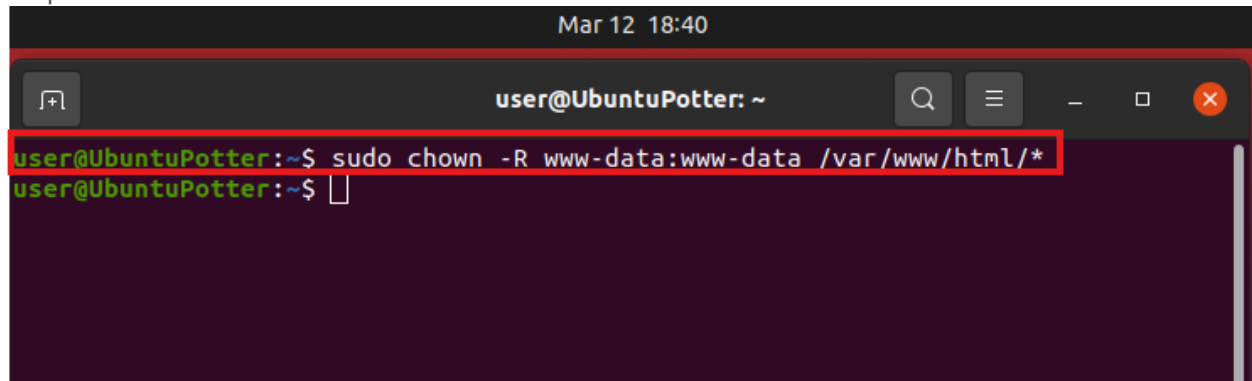


```
Mar 12 18:24
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo ls -la /var/www/html/
total 252
drwxr-xr-x  6 root root 4096 Mar 12 18:11 .
drwxr-xr-x  3 root root 4096 Mar 10 19:45 ..
drwxr-xr-x  8 root root 4096 Mar 12 18:11 .git
-rw-r--r--  1 root root  405 Mar 12 18:11 index.php
-rw-r--r--  1 root root 19903 Mar 12 18:11 license.txt
-rw-r--r--  1 root root  7406 Mar 12 18:11 readme.html
-rw-r--r--  1 root root  7371 Mar 12 18:11 wp-activate.php
drwxr-xr-x  9 root root 4096 Mar 12 18:11 wp-admin
-rw-r--r--  1 root root   351 Mar 12 18:11 wp-blog-header.php
-rw-r--r--  1 root root  2323 Mar 12 18:11 wp-comments-post.php
-rw-r--r--  1 root root  3339 Mar 12 18:11 wp-config-sample.php
drwxr-xr-x  4 root root 4096 Mar 12 18:11 wp-content
-rw-r--r--  1 root root  5617 Mar 12 18:11 wp-cron.php
drwxr-xr-x 36 root root 16384 Mar 12 18:11 wp-includes
-rw-r--r--  1 root root  2493 Mar 12 18:11 wp-links-opml.php
-rw-r--r--  1 root root  3937 Mar 12 18:11 wp-load.php
-rw-r--r--  1 root root 51850 Mar 12 18:11 wp-login.php
-rw-r--r--  1 root root  8727 Mar 12 18:11 wp-mail.php
-rw-r--r--  1 root root 32934 Mar 12 18:11 wp-settings.php
-rw-r--r--  1 root root 34621 Mar 12 18:11 wp-signup.php
-rw-r--r--  1 root root  5214 Mar 12 18:11 wp-trackback.php
-rw-r--r--  1 root root  3205 Mar 12 18:11 xmlrpc.php
user@UbuntuPotter:~$
```

Edit Ownership

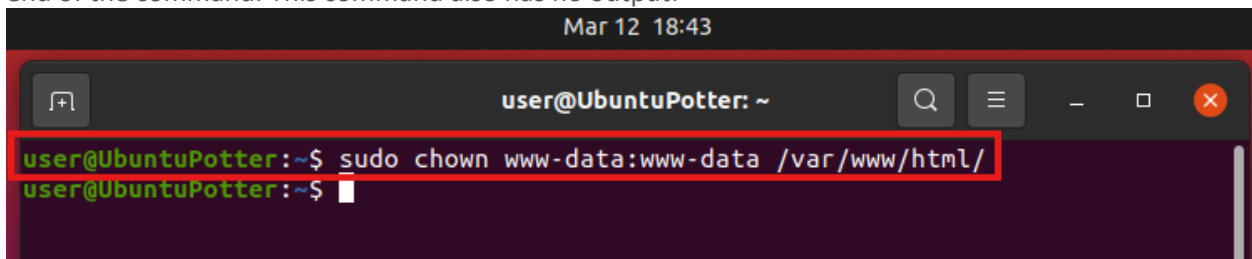
Edit Ownership of and the contents of /var/www/html/ Directory

We will now edit the permissions of the contents of /var/www/html/ directory so that the Apache can change things in it with the command “**sudo chown -R www-data:www-data /var/www/html/***”. This will change the ownership to the Apache User. If prompted for password enter it and hit enter. There is no other output:



```
Mar 12 18:40
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo chown -R www-data:www-data /var/www/html/*
user@UbuntuPotter:~$
```

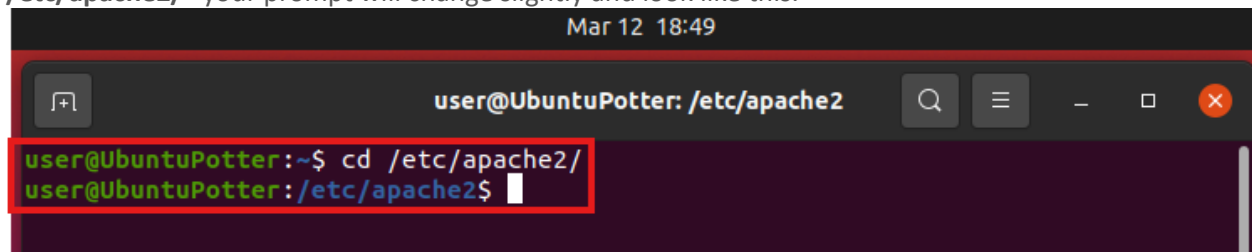
We then want to change the ownership of the directory itself with the command “**sudo chown www-data:www-data /var/www/html/**”. This look similar to the last command but notice there is no * at the end of the command. This command also has no output:



```
Mar 12 18:43
user@UbuntuPotter: ~
user@UbuntuPotter:~$ sudo chown www-data:www-data /var/www/html/
user@UbuntuPotter:~$
```

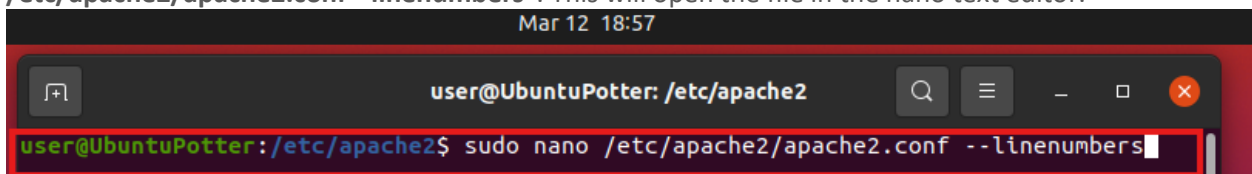
Edit the apache2.conf File

We now need to make a change to the apache2.conf file. To naviage to its location use the command “**cd /etc/apache2/**” your prompt will change slightly and look like this:



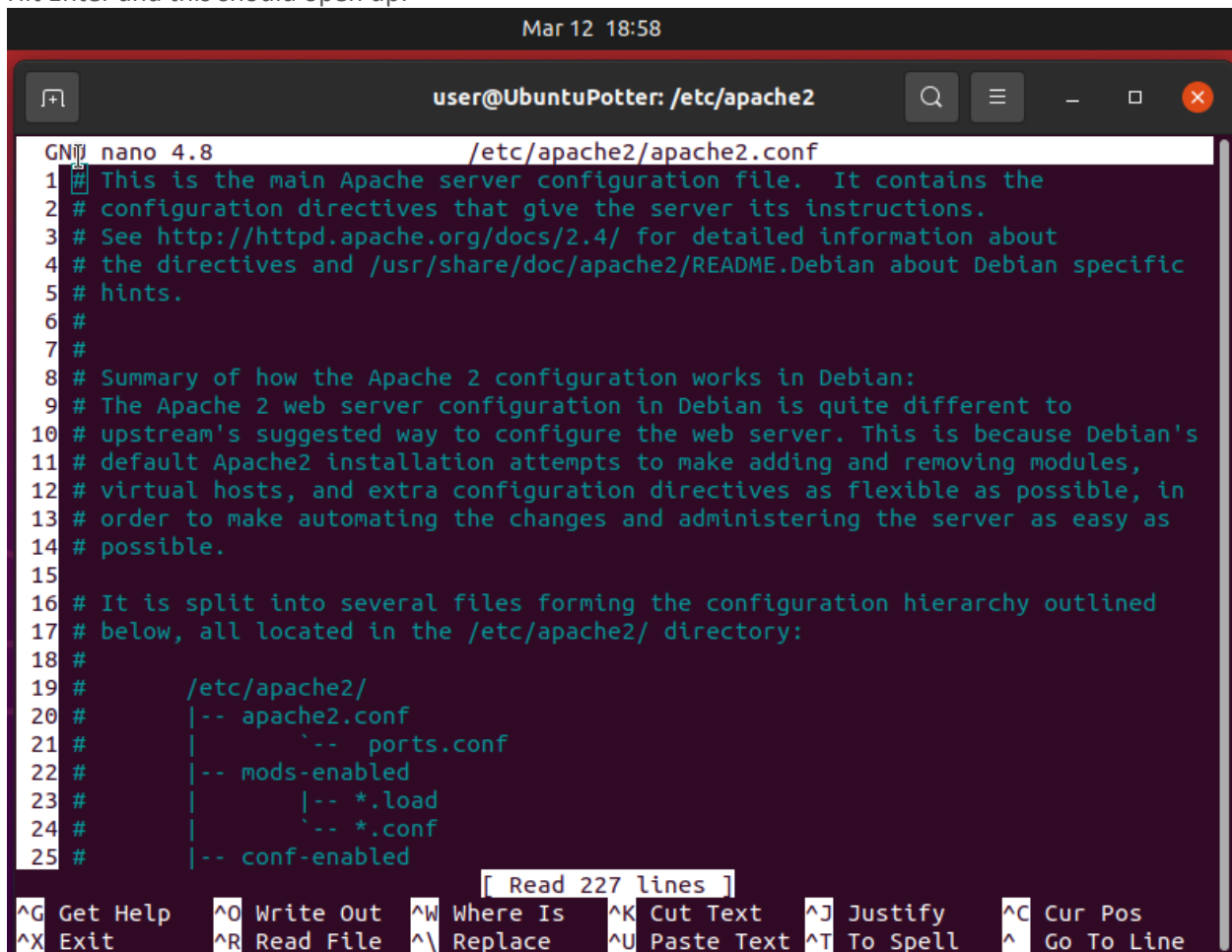
```
Mar 12 18:49
user@UbuntuPotter: /etc/apache2
user@UbuntuPotter:~$ cd /etc/apache2/
user@UbuntuPotter:/etc/apache2$
```

We then want to edit the apache2.conf file we will use the command “**sudo nano /etc/apache2/apache2.conf --linenumbers**”. This will open the file in the nano text editor.



```
Mar 12 18:57
user@UbuntuPotter: /etc/apache2
user@UbuntuPotter:/etc/apache2$ sudo nano /etc/apache2/apache2.conf --linenumbers
```

Hit Enter and this should open up:

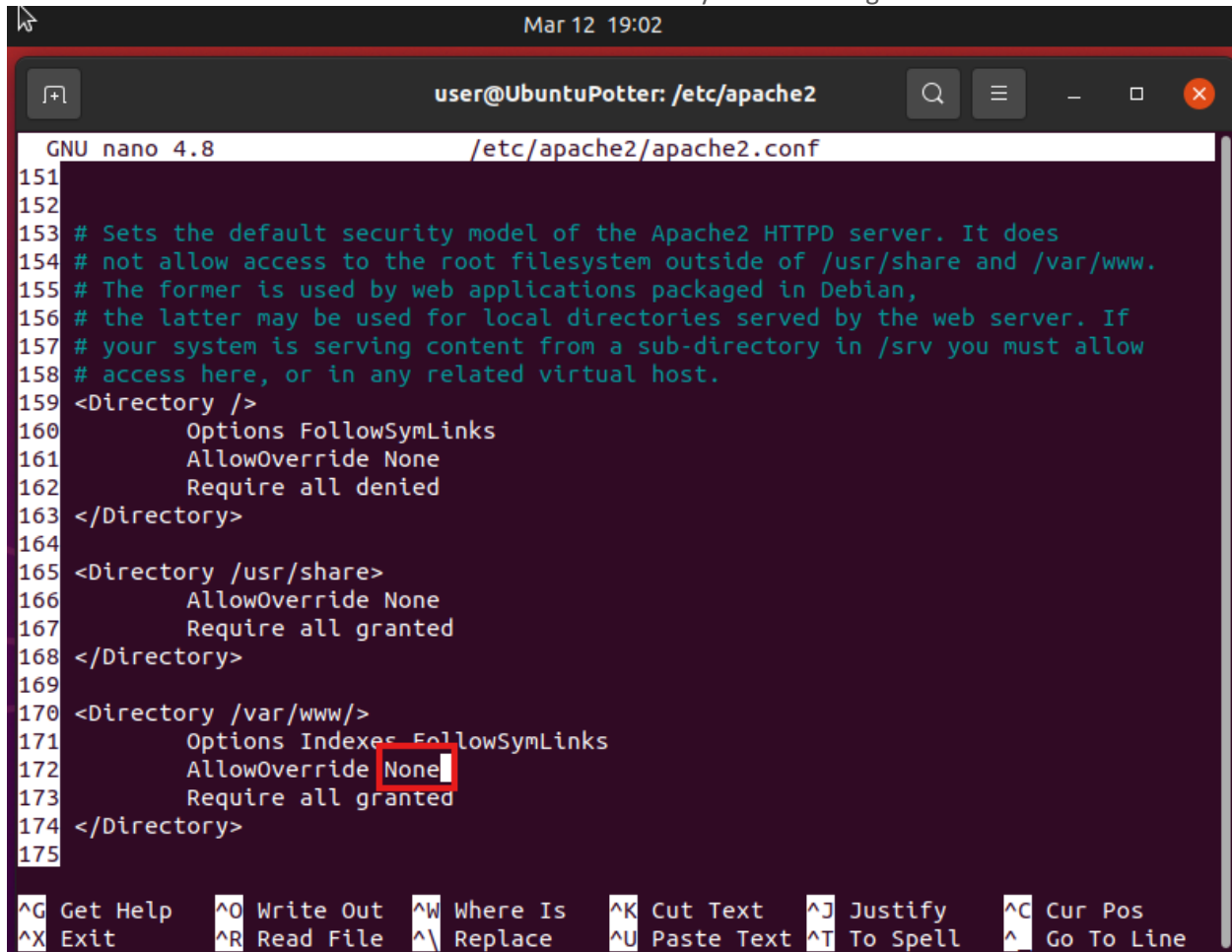


```
Mar 12 18:58
user@UbuntuPotter: /etc/apache2
nano 4.8 /etc/apache2/apache2.conf
1 # This is the main Apache server configuration file. It contains the
2 # configuration directives that give the server its instructions.
3 # See http://httpd.apache.org/docs/2.4/ for detailed information about
4 # the directives and /usr/share/doc/apache2/README.Debian about Debian specific
5 # hints.
6 #
7 #
8 # Summary of how the Apache 2 configuration works in Debian:
9 # The Apache 2 web server configuration in Debian is quite different to
10 # upstream's suggested way to configure the web server. This is because Debian's
11 # default Apache2 installation attempts to make adding and removing modules,
12 # virtual hosts, and extra configuration directives as flexible as possible, in
13 # order to make automating the changes and administering the server as easy as
14 # possible.
15 #
16 # It is split into several files forming the configuration hierarchy outlined
17 # below, all located in the /etc/apache2/ directory:
18 #
19 #     /etc/apache2/
20 #     |-- apache2.conf
21 #     |   |-- ports.conf
22 #     |-- mods-enabled
23 #     |   |-- *.load
24 #     |   |-- *.conf
25 #     |-- conf-enabled
26 #     |-- *.conf
27 #     |-- *.d/*

[ Read 227 lines ]
^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify      ^C Cur Pos
^X Exit          ^R Read File    ^\ Replace      ^U Paste Text   ^T To Spell     ^_ Go To Line
```

Override All Default Apache Directives

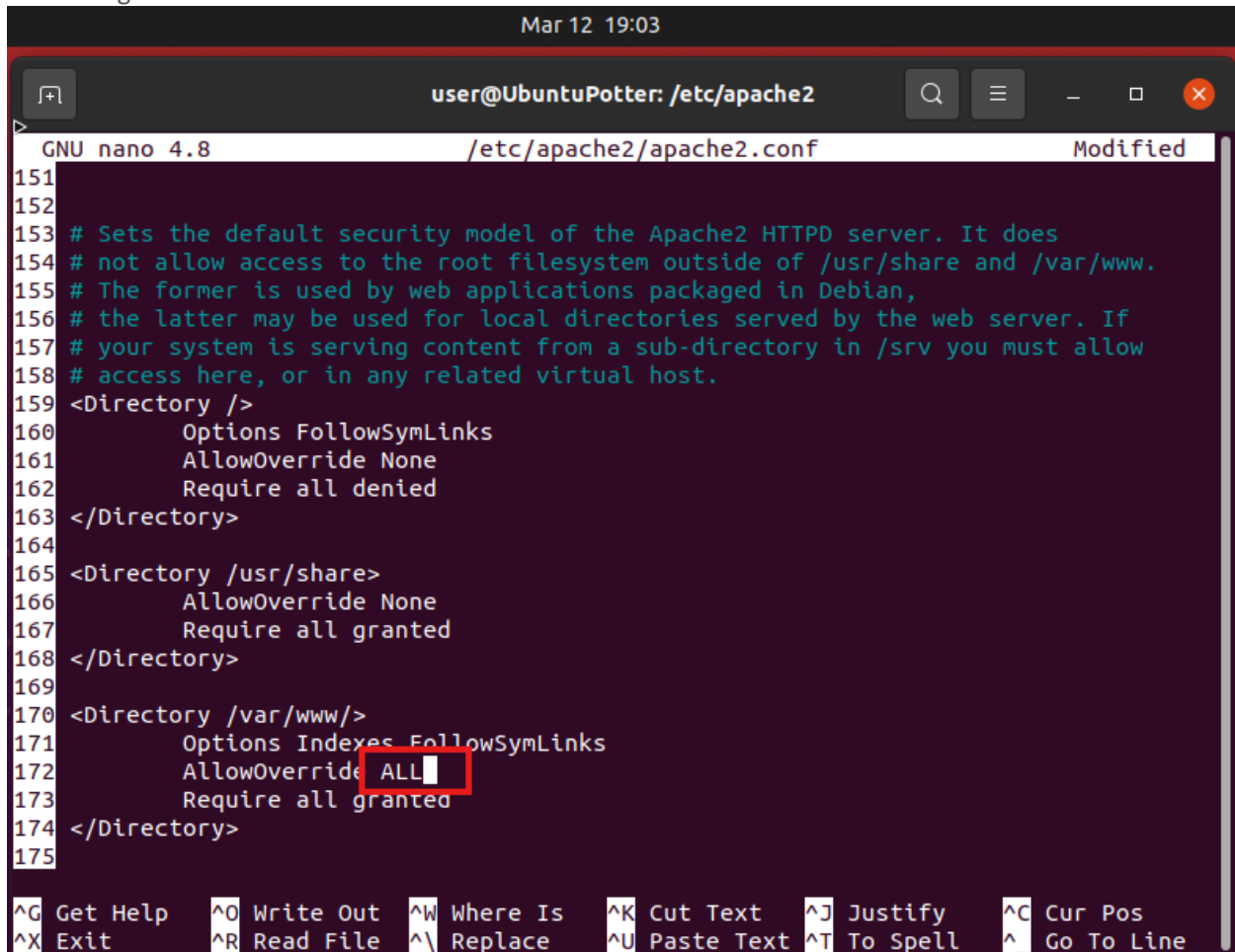
We now want to find the AllowOverride option for the directory /var/www/ and change it to ALL. This command should be located around line 172. Here is what you are looking for:



```
Mar 12 19:02
user@UbuntuPotter: /etc/apache2
GNU nano 4.8 /etc/apache2/apache2.conf
151
152
153 # Sets the default security model of the Apache2 HTTPD server. It does
154 # not allow access to the root filesystem outside of /usr/share and /var/www.
155 # The former is used by web applications packaged in Debian,
156 # the latter may be used for local directories served by the web server. If
157 # your system is serving content from a sub-directory in /srv you must allow
158 # access here, or in any related virtual host.
159 <Directory />
160     Options FollowSymLinks
161     AllowOverride None
162     Require all denied
163 </Directory>
164
165 <Directory /usr/share>
166     AllowOverride None
167     Require all granted
168 </Directory>
169
170 <Directory /var/www/>
171     Options Indexes FollowSymLinks
172     AllowOverride None
173     Require all granted
174 </Directory>
175

^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Paste Text ^T To Spell  ^_ Go To Line
```


And change this to ALL:

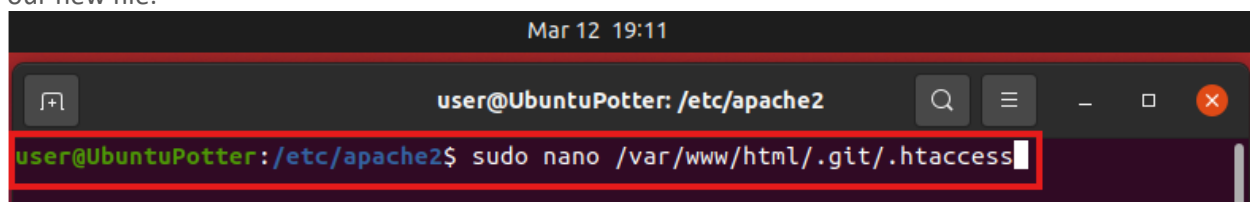


```
Mar 12 19:03
user@UbuntuPotter: /etc/apache2
GNU nano 4.8 /etc/apache2/apache2.conf Modified
151
152
153 # Sets the default security model of the Apache2 HTTPD server. It does
154 # not allow access to the root filesystem outside of /usr/share and /var/www.
155 # The former is used by web applications packaged in Debian,
156 # the latter may be used for local directories served by the web server. If
157 # your system is serving content from a sub-directory in /srv you must allow
158 # access here, or in any related virtual host.
159 <Directory />
160     Options FollowSymLinks
161     AllowOverride None
162     Require all denied
163 </Directory>
164
165 <Directory /usr/share>
166     AllowOverride None
167     Require all granted
168 </Directory>
169
170 <Directory /var/www/>
171     Options Indexes FollowSymLinks
172     AllowOverride ALL
173     Require all granted
174 </Directory>
175
^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text   ^J Justify    ^C Cur Pos
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To Spell   ^ Go To Line
```

Press **Ctrl+X** Then **Y** then **Enter** to save the change.

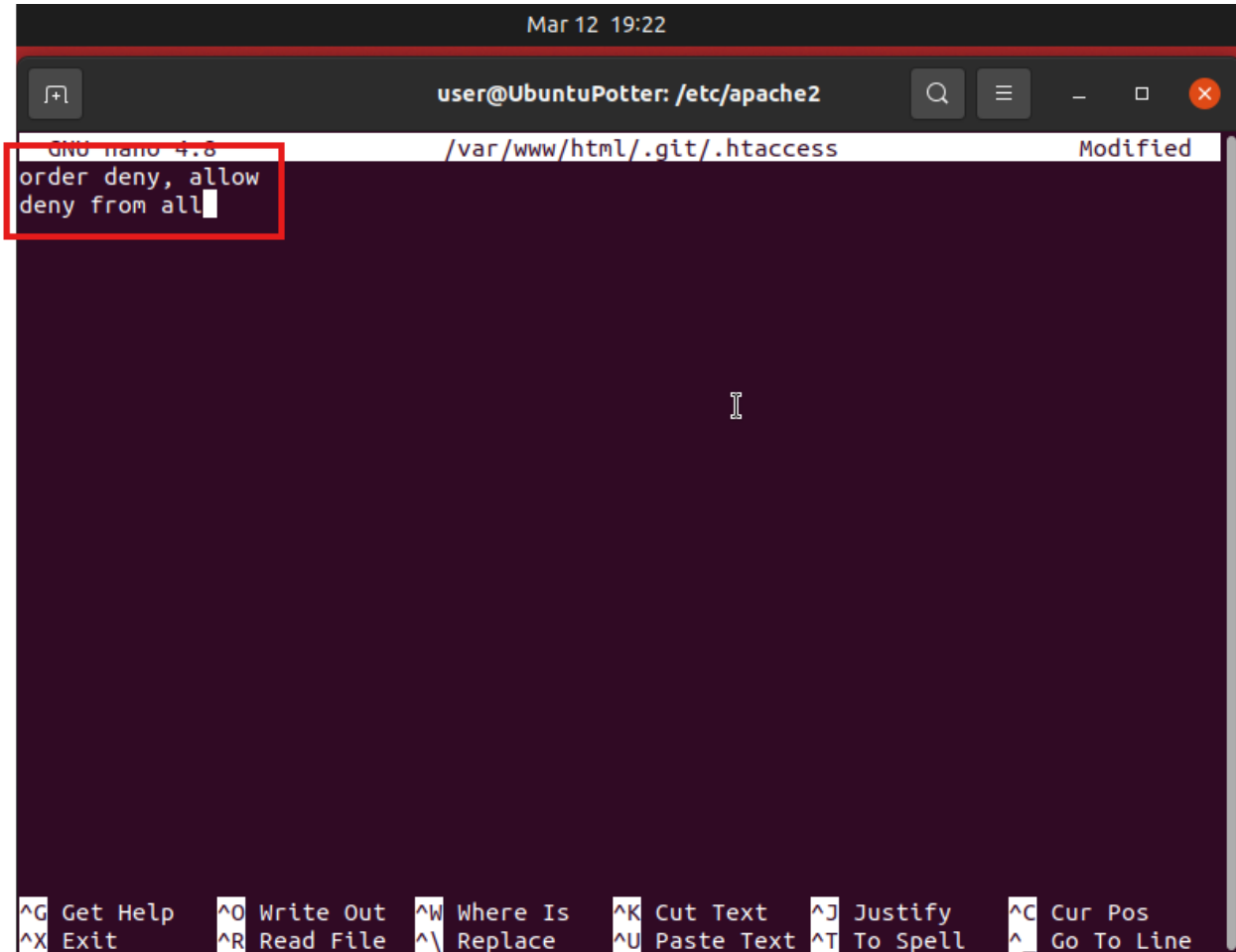
Create a .htaccess File in the /var/www/html/.git/ Directory

We then want to create a .htaccess file under the .git folder to secure access to it. We will do this by using the command “**sudo nano /var/www/html/.git/.htaccess**” This will open the Text editor to create our new file:



```
Mar 12 19:11
user@UbuntuPotter: /etc/apache2
user@UbuntuPotter: /etc/apache2$ sudo nano /var/www/html/.git/.htaccess
```

Once in the Text editor enter this text **order deny, allow** and on the next line **deny from all**. Just like below:



The screenshot shows a terminal window with a nano text editor. The title bar indicates the user is 'user@UbuntuPotter' and the current directory is '/etc/apache2'. The editor is editing the file '/var/www/html/.git/.htaccess'. The content of the file is as follows:

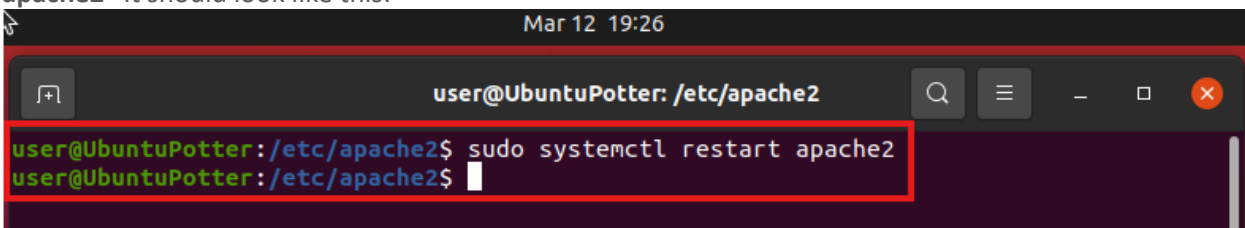
```
GNU nano 4.8 /var/www/html/.git/.htaccess Modified
order deny, allow
deny from all
```

The bottom of the window displays a series of keyboard shortcuts for nano, such as '^G Get Help', '^O Write Out', '^W Where Is', '^K Cut Text', '^J Justify', '^C Cur Pos', '^X Exit', '^R Read File', '^_ Replace', '^U Paste Text', '^T To Spell', and '^_ Go To Line'.

Press **Ctrl+X** Then **Y** then **Enter** to save the change.

Restart the Apache Service

The last step of this process is to restart the apache2 service using the command “**Sudo systemctl restart apache2**” It should look like this:



The screenshot shows a terminal window with the same title bar as the previous one. The command 'sudo systemctl restart apache2' has been entered and executed. The output shows the command being run as root:

```
user@UbuntuPotter:/etc/apache2$ sudo systemctl restart apache2
user@UbuntuPotter:/etc/apache2$
```

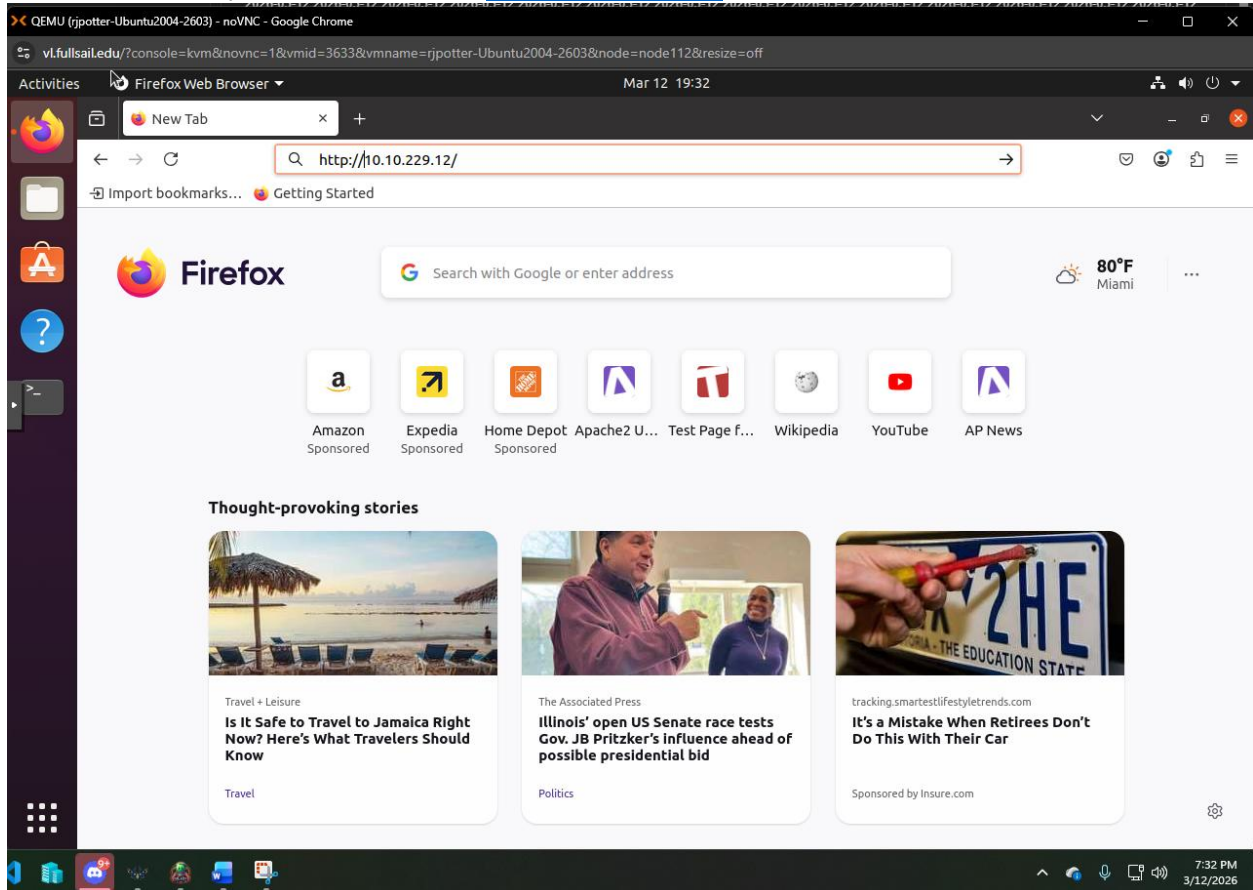
We can now close our terminal as we are done in it for now.

WordPress Configuration

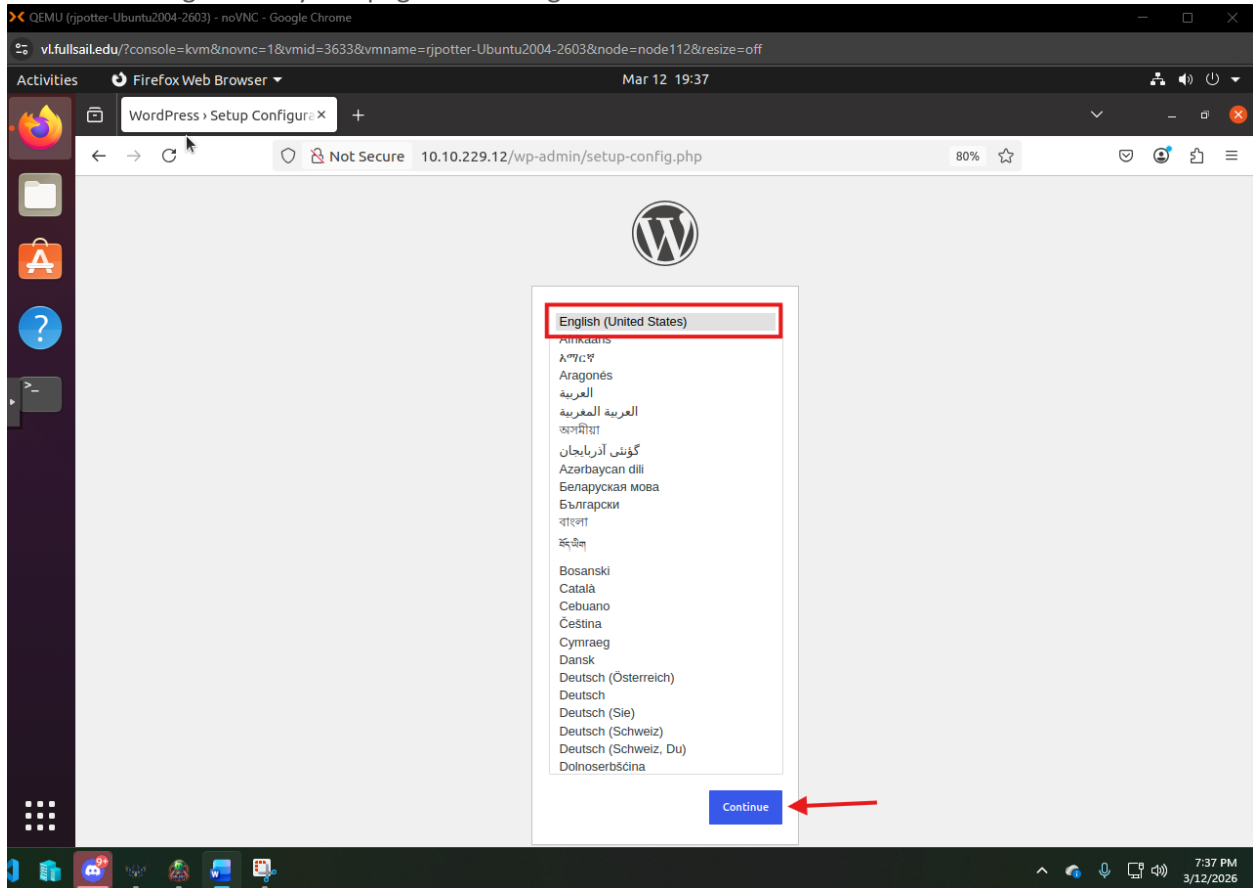
Configure WordPress

WordPress Configuration Selections

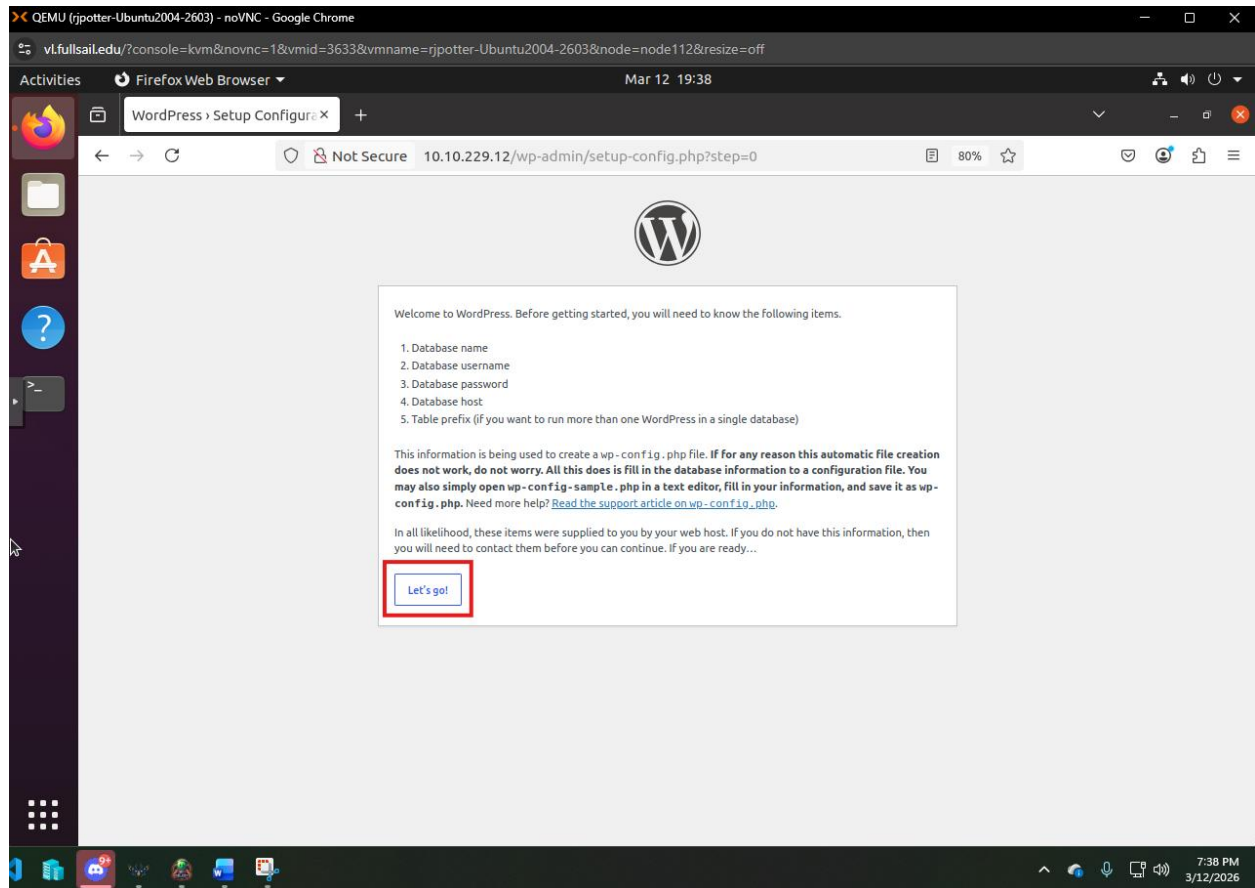
We now want to open firefox and Enter <http://10.10.229.12> like this:



You should be greeted by this page. Select English and Continue:



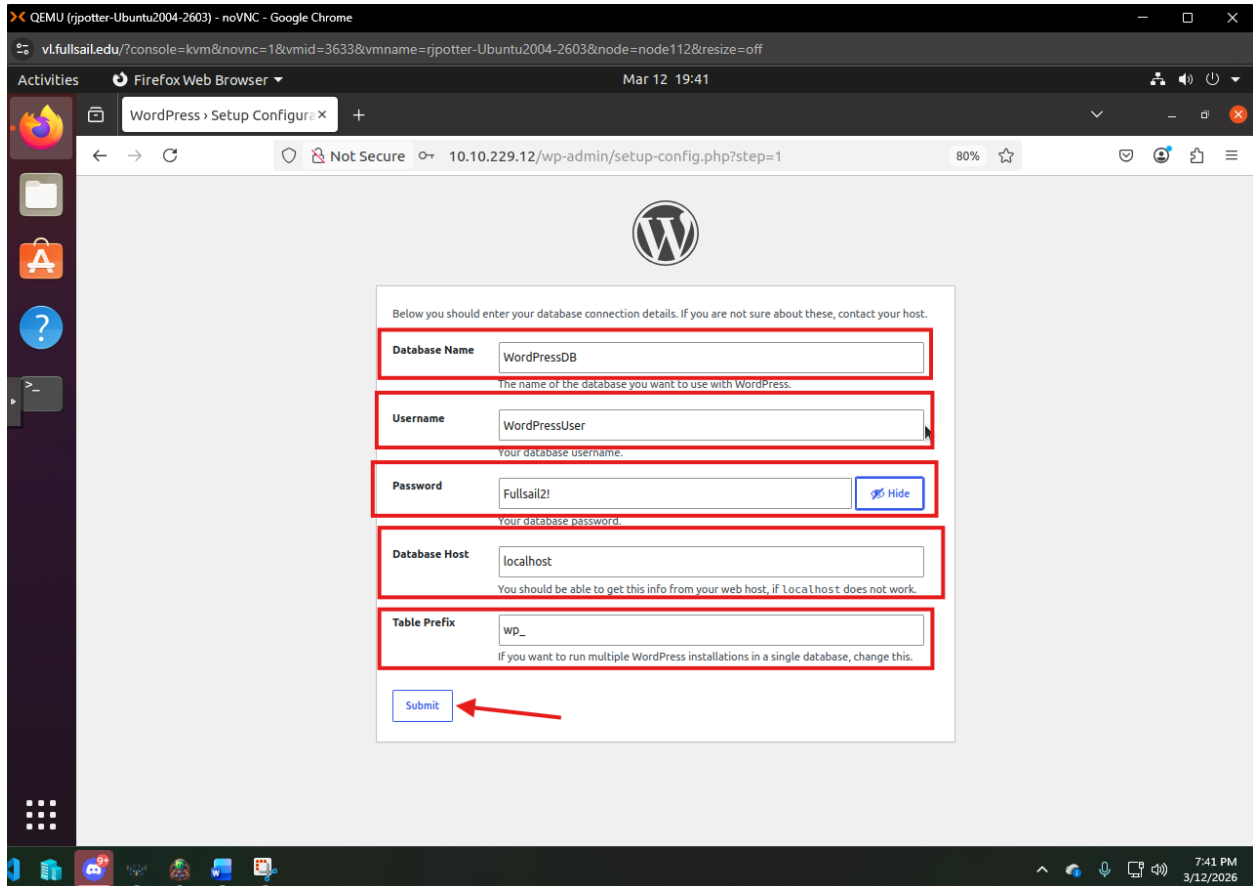
Click "Let's Go!":



Enter the info below into the Correct fields:

- **Database Name:** WordPressDB
- **Username:** WordPressUser
- **Password:** Fullsail2!
- **Database Host:** localhost
- **Table Prefix:** wp_

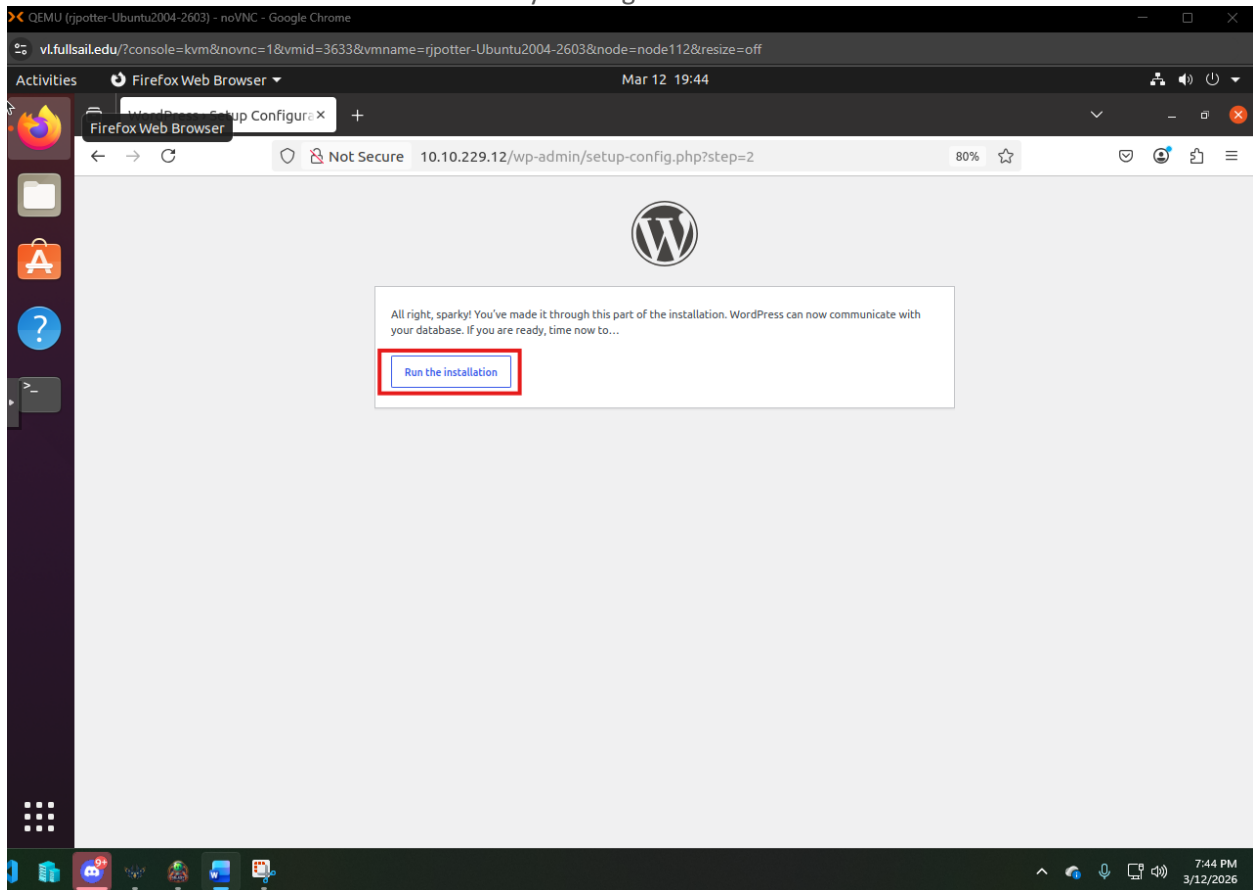
Like this:



Then click Submit.

Run Installation

We then want to run the installation with by clicking “Run the installation”:

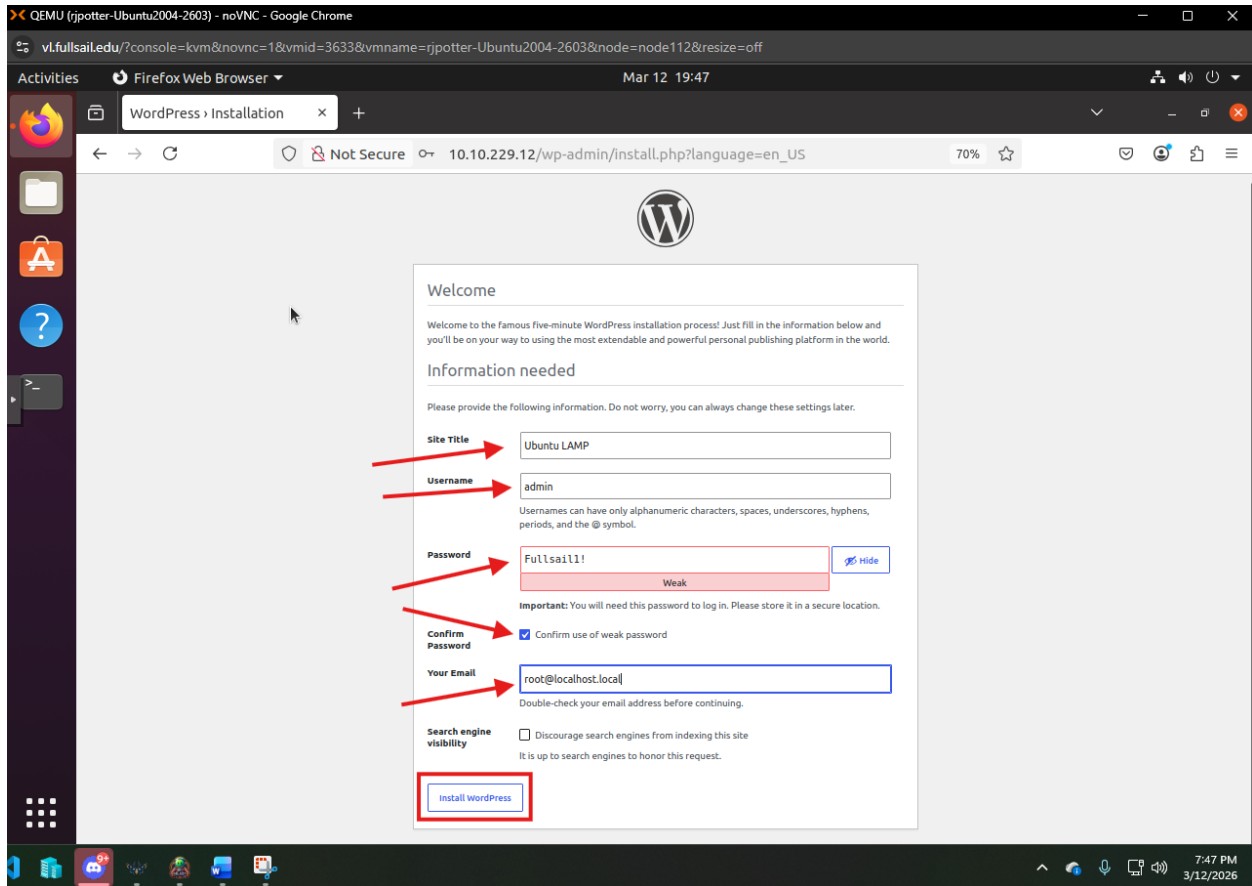


Create an Admin WordPress User

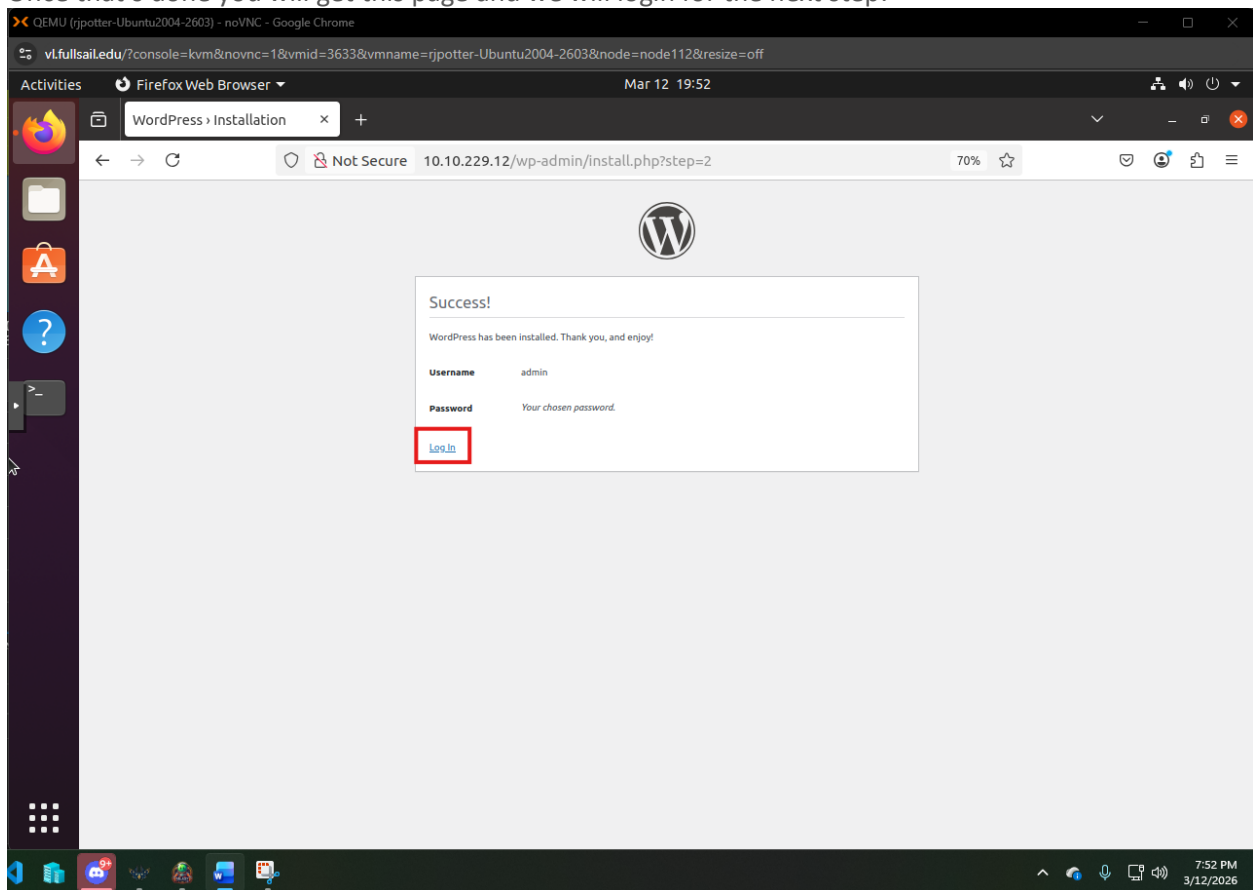
It will then Ask us to fill out all the field for the admin account of the wordpress site use these details:

- **Site Title:** Ubuntu LAMP
- **Username:** admin
- **Password:** Fullsail1!
- **Your email:** root@localhost.local
- **Search Engine Visibility:** leave unchecked

It should Look like this:

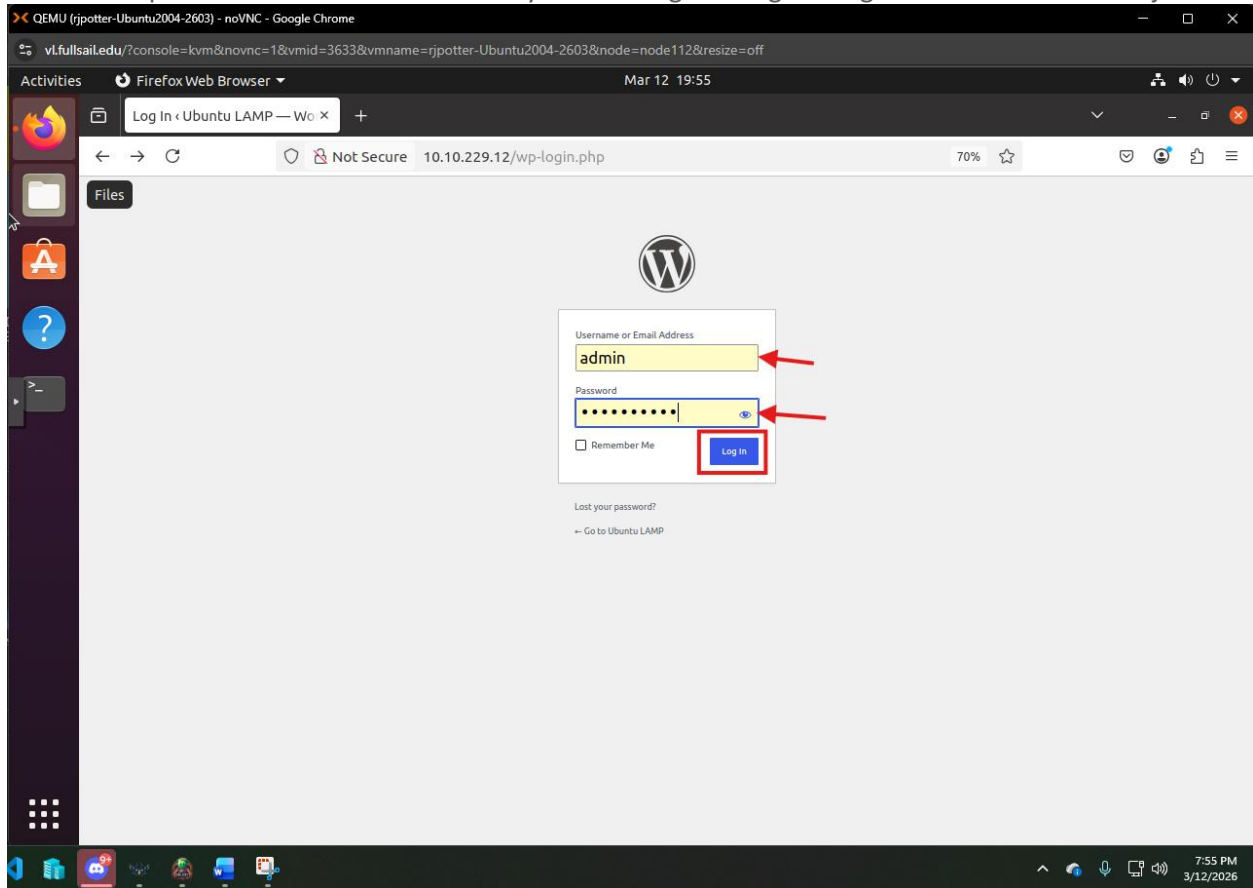


Once that's done you will get this page and we will login for the next step:

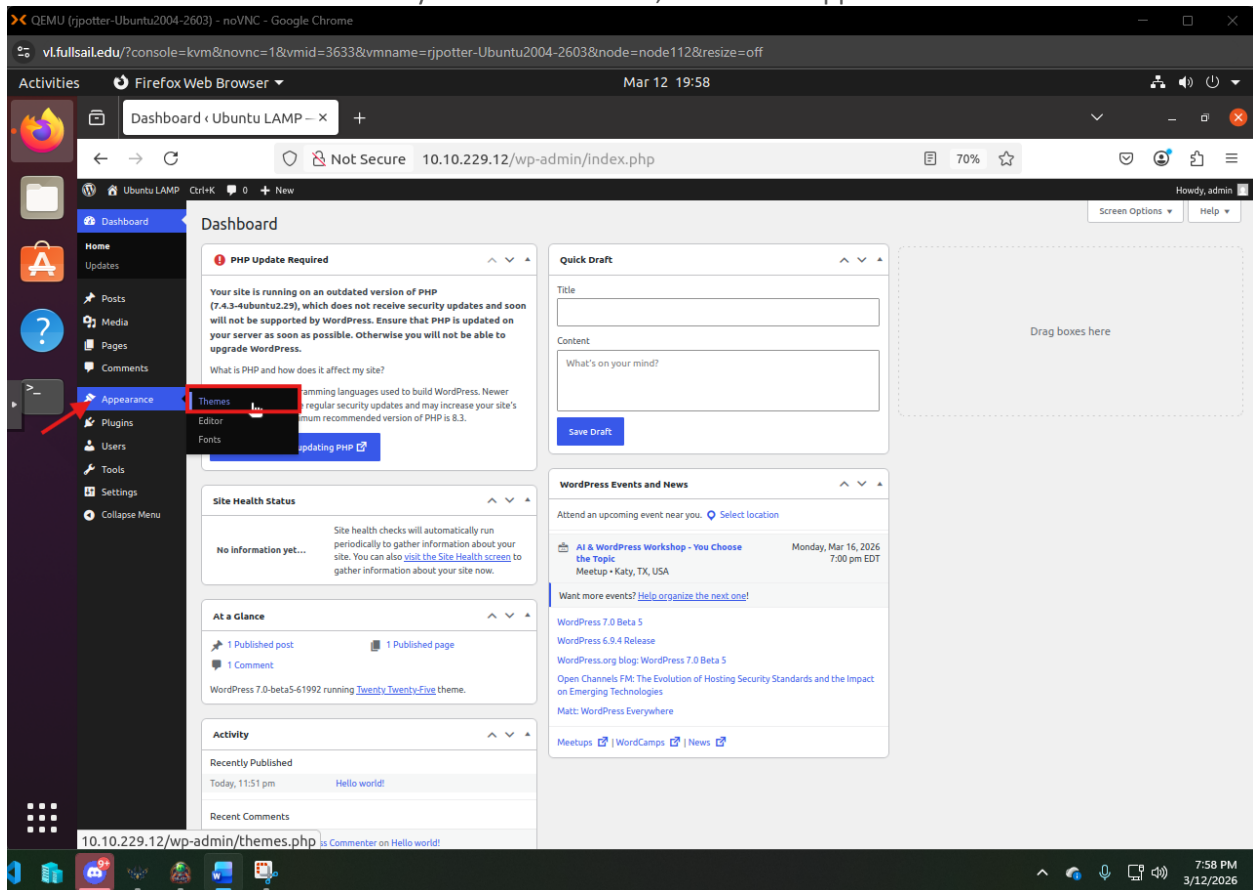


WordPress Site Selections

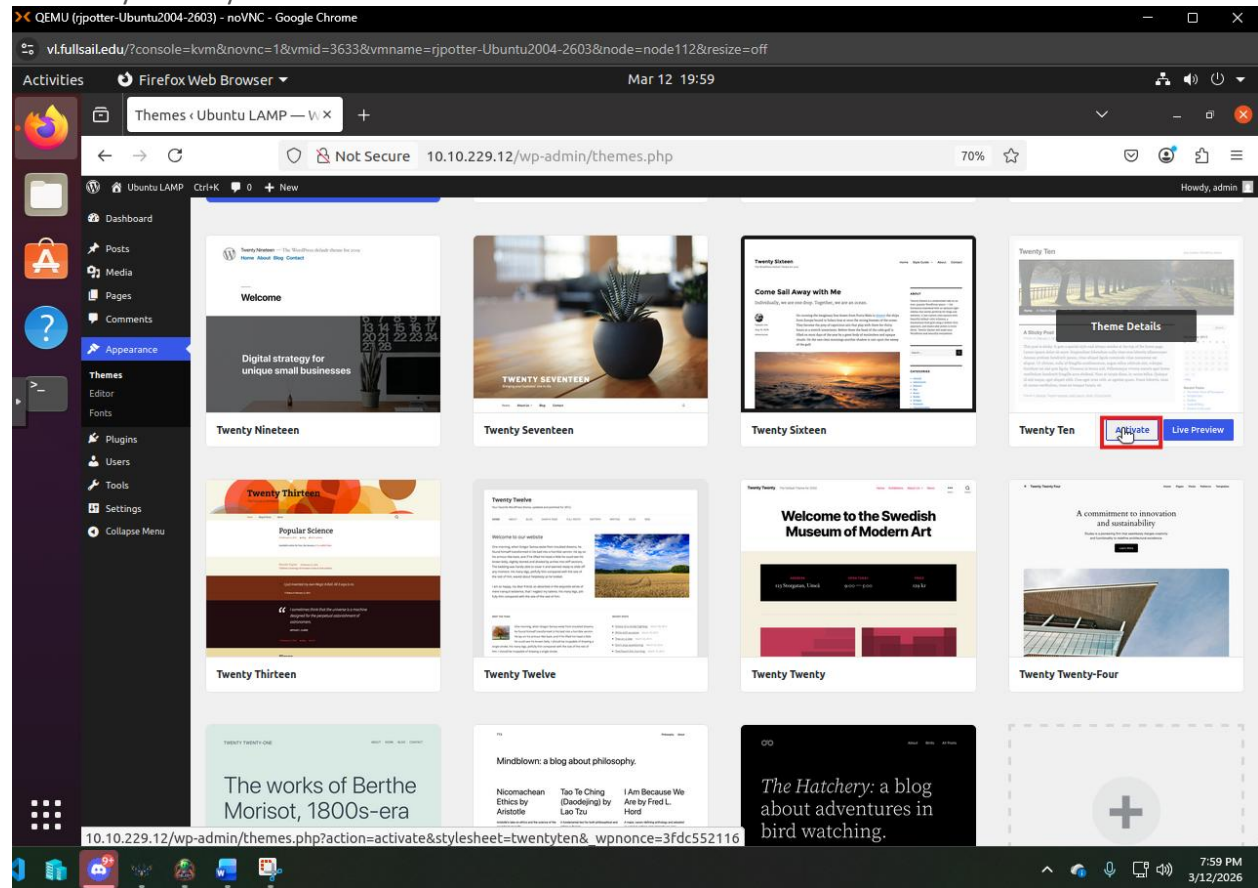
Our first step to select our theme to one of you choosing is to login using the admin account we just created:



We can now add a New Theme to your WordPress site, hover over Appearance and Click Themes:



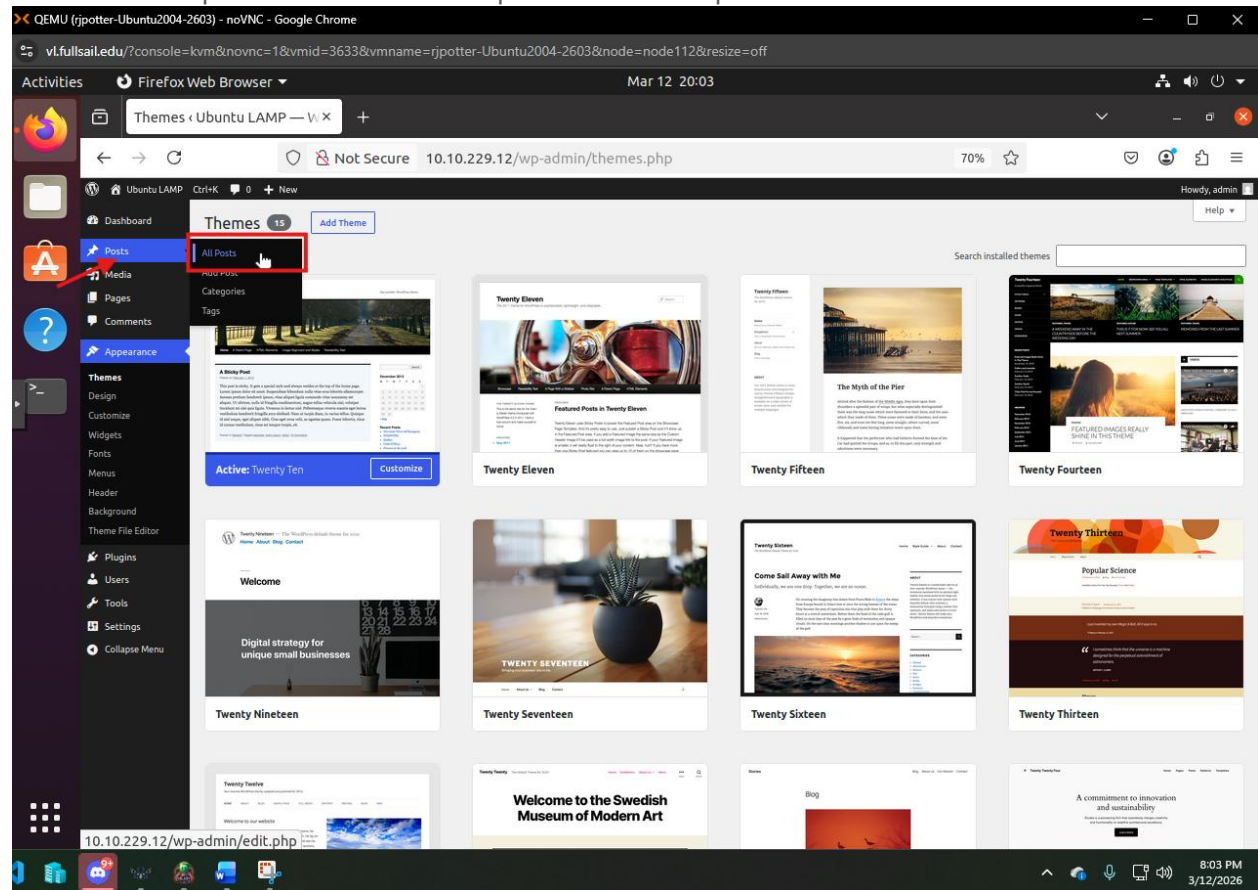
Select any theme you like and Click Activate:



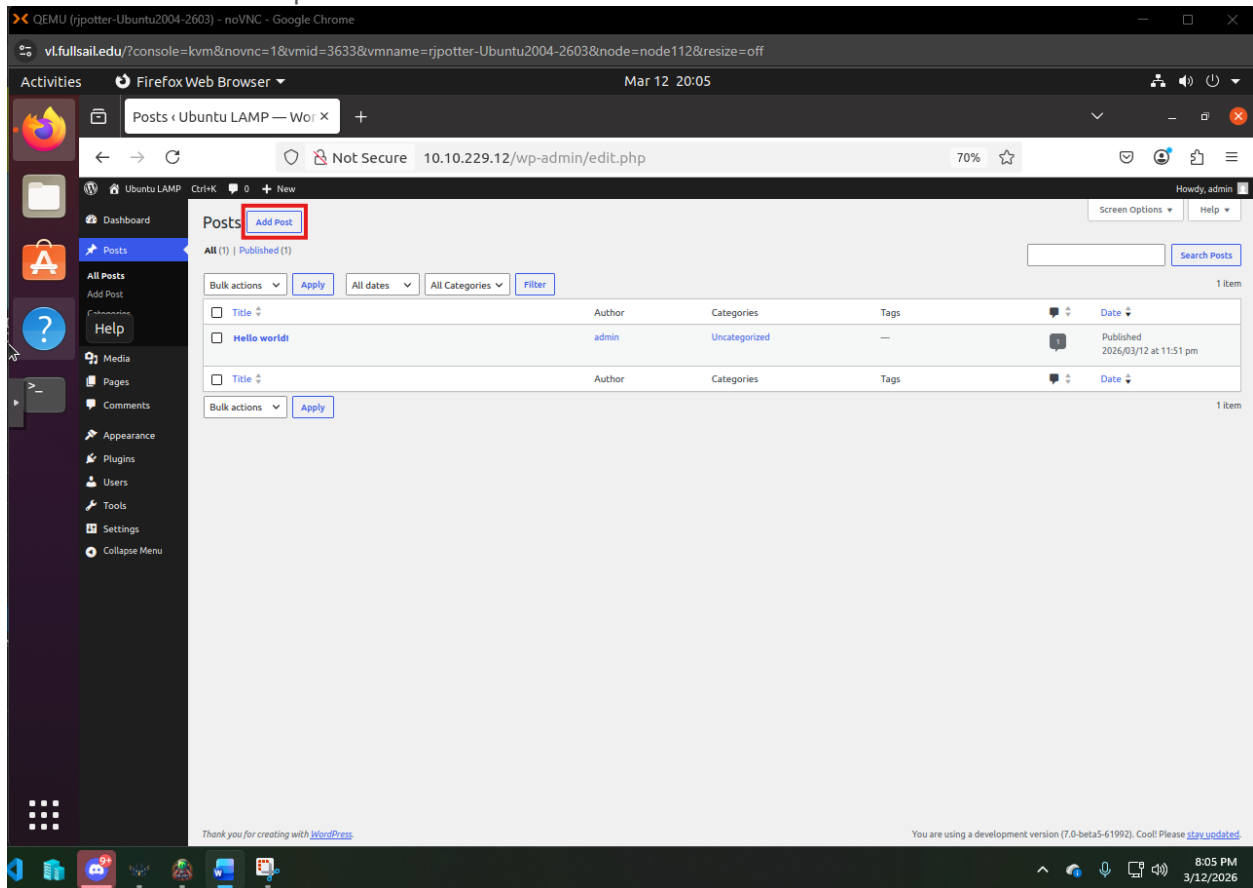
Congratulations! We Set the Theme for the site. Now lets test the site by creating a test post.

Test WordPress Website

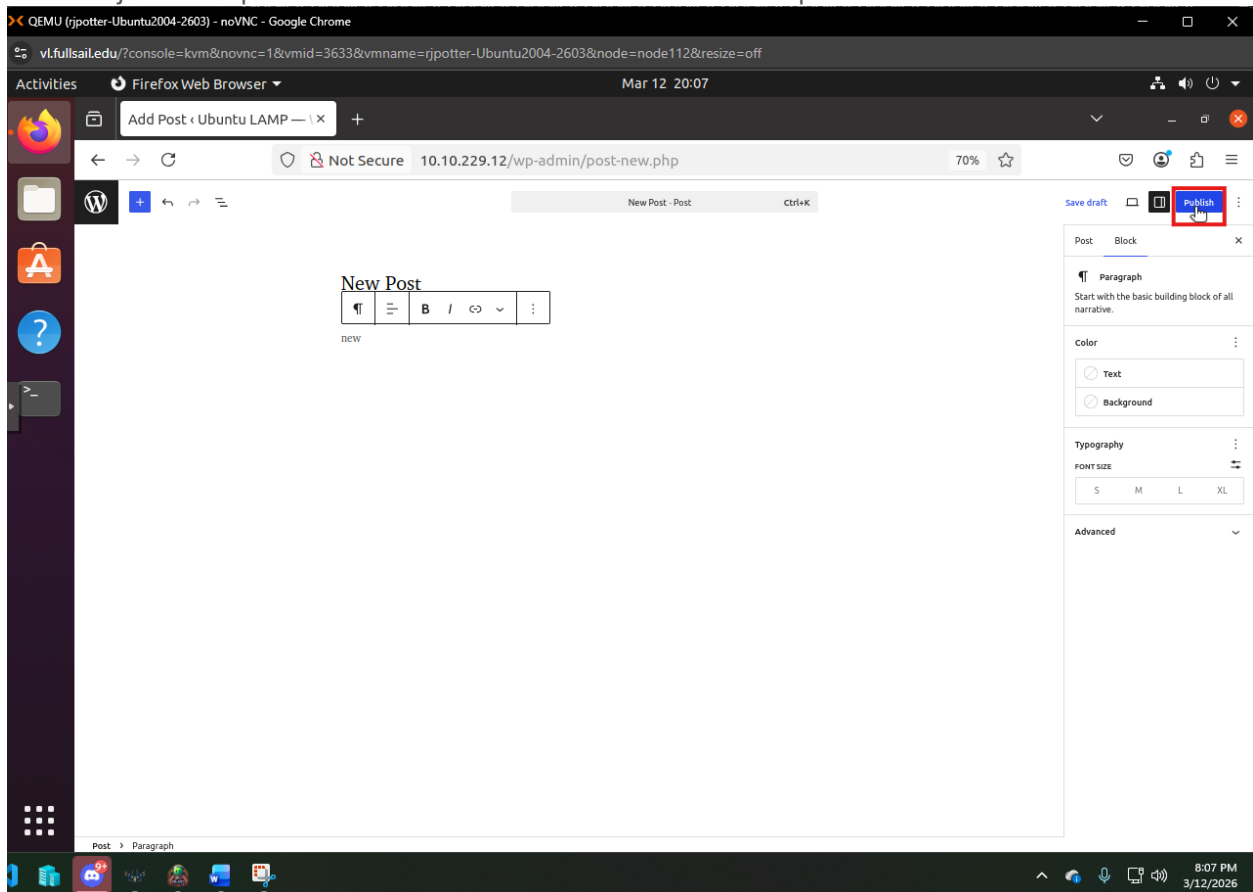
In the left hand panel We want to select posts and then all posts:



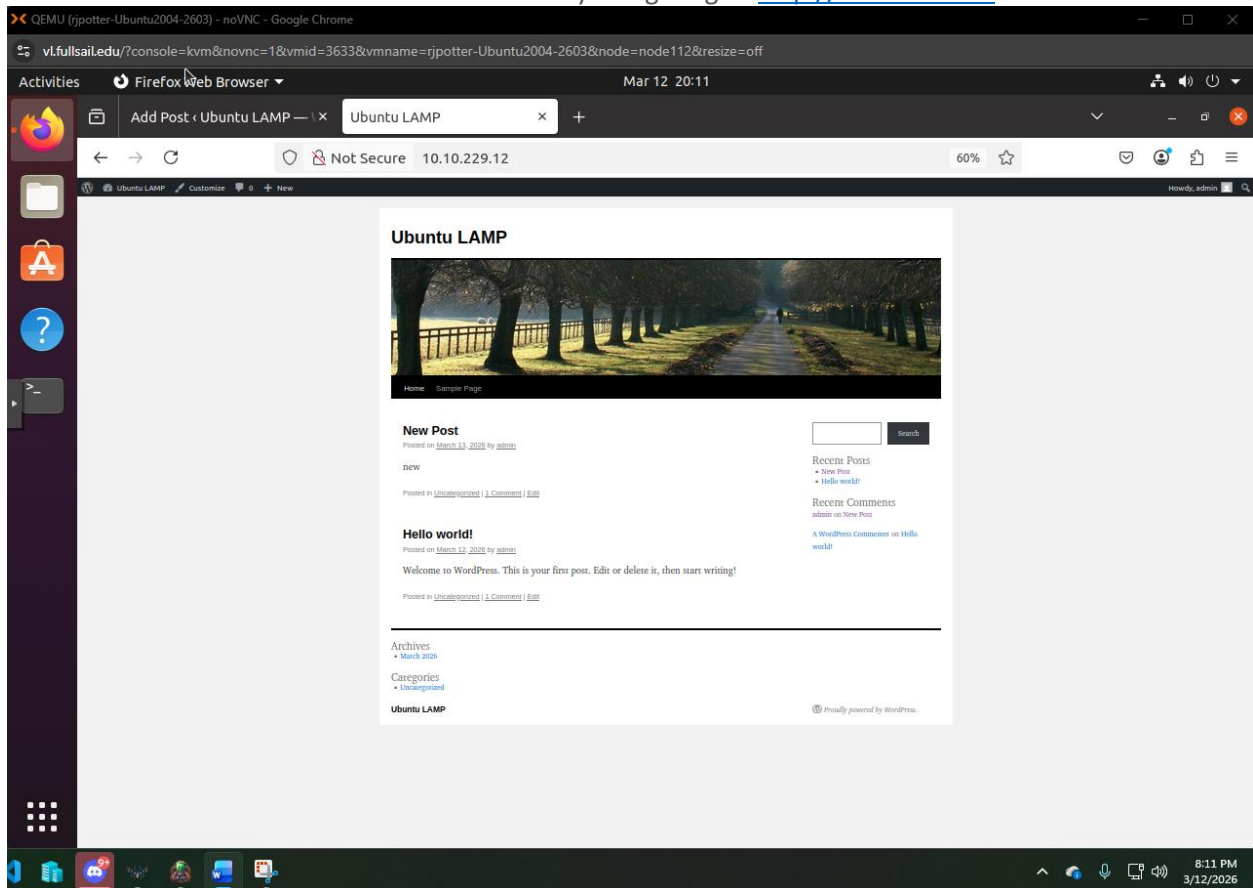
Then to create our new post we will click on the Add Post Button:



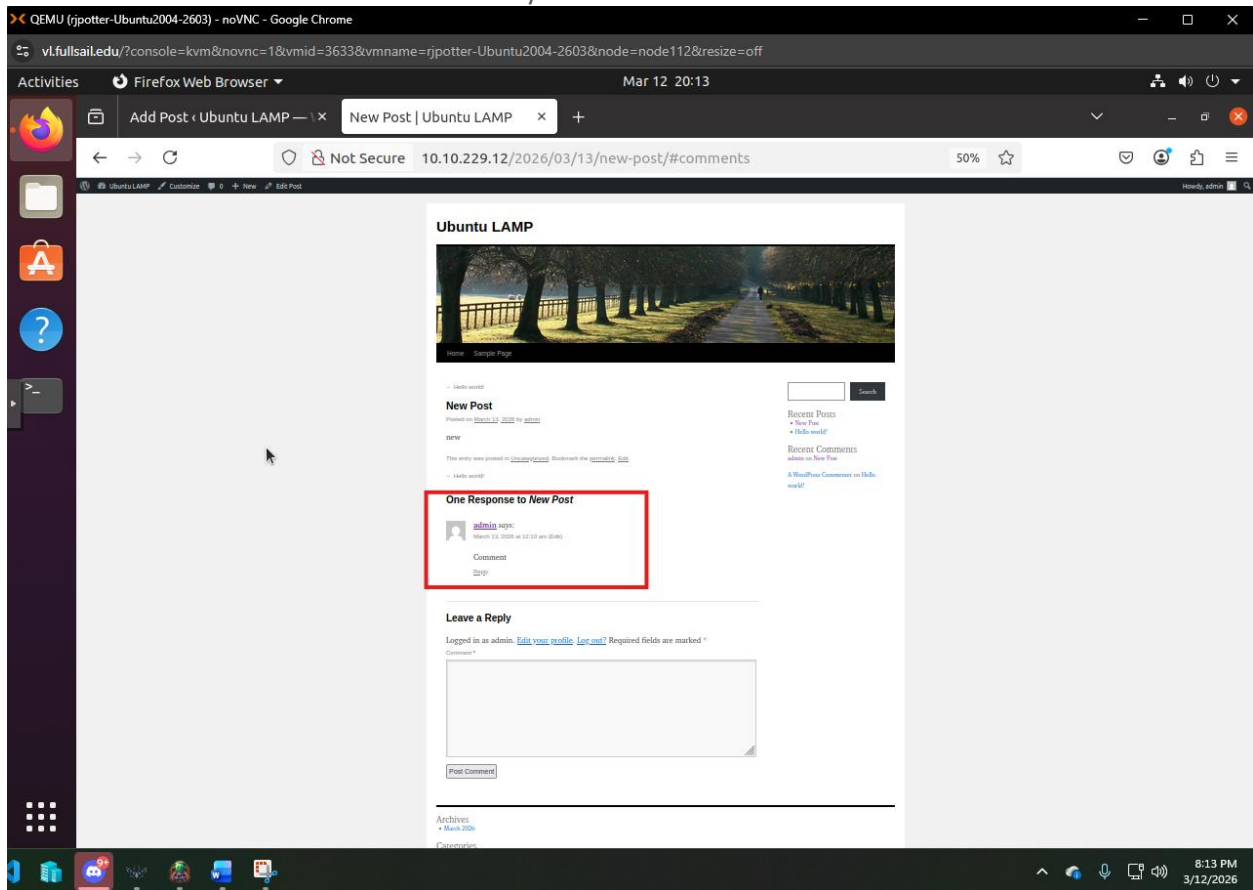
As this is just a test post we will create one that looks like this and press **Publish** and **Confirm**:



Now let have a look at what our site looks like by navigating to <http://10.10.229.12>:



And if we click on comment we can view any comments that have been left for us:



We now Have a fully operational WordPress Site.

WordPress Security Settings and Configurations

With the initial setup out of the way, we can now begin focusing on securing our website. It is important to consider security from the start, as a weak security posture can lead to several negative impacts on the business. These risks include reputational damage, financial loss, and operational disruption.

We will be implementing three key changes to help harden the environment and establish a baseline level of defense in depth. That said, no security solution is ever 100% perfect. The goal is to make the website a more difficult target, as most attackers will move on if easier opportunities are available.

The first change will be adjusting default file permissions to ensure that only the system, WordPress, or authorized users can make modifications, preventing unauthorized changes. The second change will be relocating and securing the wp-config.php file, which contains sensitive configuration data. The third and final change will be implementing an application-layer firewall to provide additional protection for the website and its data.

Defence In Depth:

Defense in depth is the concept of using multiple layers of security instead of relying on just one control to protect a system. The idea is simple: if one layer fails, there are still other layers in place to stop or slow down an attacker. A good way to think about this is like securing your home. You might have a lock on your front door, but you also have windows, maybe a fence, and possibly a security system. Even if someone gets past one of those, they still have to deal with the others. This layered approach makes it much harder for someone to successfully break in.

When applying this to a website, you don't depend on a single security measure. Instead, you combine different types of controls that each protect a specific part of the system. For example, setting proper file permissions helps prevent unauthorized changes to your files. Securing the wp-config.php file protects sensitive information like database credentials. Adding an application layer firewall helps block malicious traffic before it can reach your website. Each of these controls works together as part of a larger security strategy.

It is also important to understand that no security setup is ever 100% perfect. Attackers are constantly finding new ways to get around defenses. Because of this, the goal of defense in depth is not to be unbreakable, but to make your system harder to attack and less attractive as a target. Most attackers are looking for the easiest way in, so when they encounter multiple layers of security, they are more likely to move on to something else.

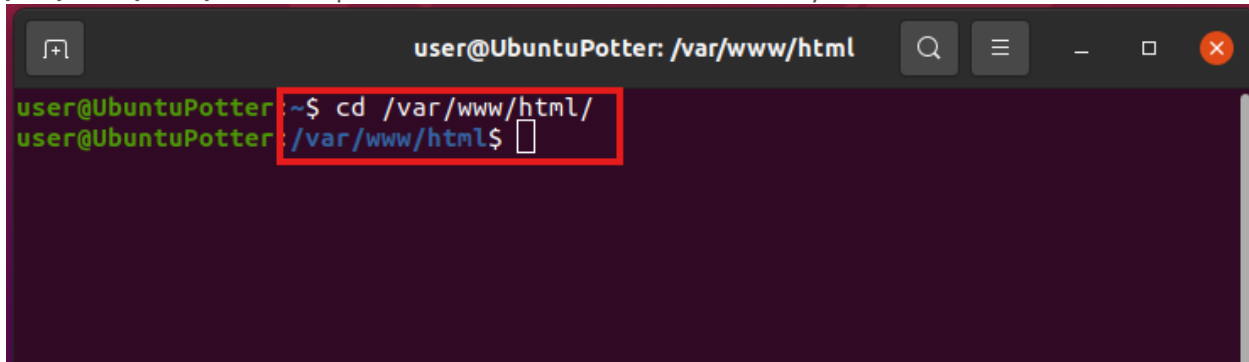
If you would like to read more you can Visit:

<https://www.fortinet.com/resources/cyberglossary/defense-in-depth>

File Permissions

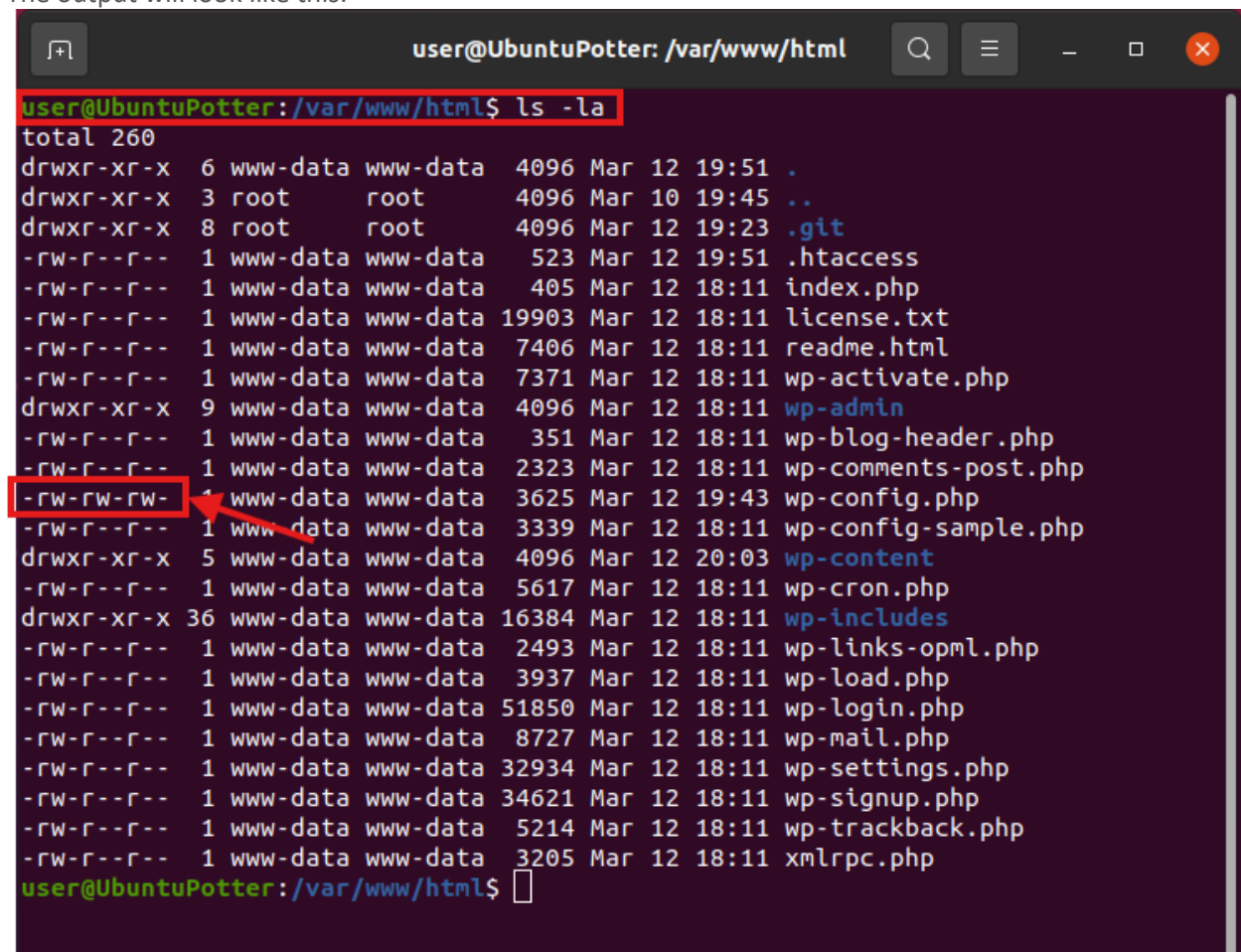
Vulnerability

So here we can see that all of the permissions that our current configurations has. This can be viewed by first navigating to the folder that contains our wordpress site. The command to do this is “`cd /var/www/html/`”. The output will look like this notice the directory next to the username:

A terminal window titled 'user@UbuntuPotter: /var/www/html'. The prompt is 'user@UbuntuPotter: ~\$'. The user enters 'cd /var/www/html/' and the prompt changes to 'user@UbuntuPotter: /var/www/html\$'.

```
user@UbuntuPotter: ~$ cd /var/www/html/
user@UbuntuPotter: /var/www/html$
```

This will put us in the directory and we can view the current configurations. With this command “`ls -la`”. The output will look like this:

A terminal window titled 'user@UbuntuPotter: /var/www/html'. The prompt is 'user@UbuntuPotter: /var/www/html\$'. The user enters 'ls -la'. The output shows a list of files and directories. The permissions for 'wp-config.php' are highlighted with a red box and an arrow pointing to it.

```
user@UbuntuPotter: /var/www/html$ ls -la
total 260
drwxr-xr-x  6 www-data www-data  4096 Mar 12 19:51 .
drwxr-xr-x  3 root     root     4096 Mar 10 19:45 ..
drwxr-xr-x  8 root     root     4096 Mar 12 19:23 .git
-rw-r--r--  1 www-data www-data   523 Mar 12 19:51 .htaccess
-rw-r--r--  1 www-data www-data   405 Mar 12 18:11 index.php
-rw-r--r--  1 www-data www-data 19903 Mar 12 18:11 license.txt
-rw-r--r--  1 www-data www-data  7406 Mar 12 18:11 readme.html
-rw-r--r--  1 www-data www-data  7371 Mar 12 18:11 wp-activate.php
drwxr-xr-x  9 www-data www-data  4096 Mar 12 18:11 wp-admin
-rw-r--r--  1 www-data www-data   351 Mar 12 18:11 wp-blog-header.php
-rw-r--r--  1 www-data www-data  2323 Mar 12 18:11 wp-comments-post.php
-rw-rw-rw-  1 www-data www-data  3625 Mar 12 19:43 wp-config.php
-rw-r--r--  1 www-data www-data  3339 Mar 12 18:11 wp-config-sample.php
drwxr-xr-x  5 www-data www-data  4096 Mar 12 20:03 wp-content
-rw-r--r--  1 www-data www-data  5617 Mar 12 18:11 wp-cron.php
drwxr-xr-x 36 www-data www-data 16384 Mar 12 18:11 wp-includes
-rw-r--r--  1 www-data www-data  2493 Mar 12 18:11 wp-links-opml.php
-rw-r--r--  1 www-data www-data  3937 Mar 12 18:11 wp-load.php
-rw-r--r--  1 www-data www-data 51850 Mar 12 18:11 wp-login.php
-rw-r--r--  1 www-data www-data  8727 Mar 12 18:11 wp-mail.php
-rw-r--r--  1 www-data www-data 32934 Mar 12 18:11 wp-settings.php
-rw-r--r--  1 www-data www-data 34621 Mar 12 18:11 wp-signup.php
-rw-r--r--  1 www-data www-data  5214 Mar 12 18:11 wp-trackback.php
-rw-r--r--  1 www-data www-data  3205 Mar 12 18:11 xmlrpc.php
user@UbuntuPotter: /var/www/html$
```

Now here we have a problem with the current configuration ANYONE Can read and Write in our WP-Config file. Let me Explain a little about what we are looking at.

Linux file permissions are how the system controls who can access or change files and folders. Every file has three groups of permissions: the owner (the person who created it), the group (a set of users), and everyone else. Each of these groups can be given three types of access: read, write, and execute. Read means you can view the file, write means you can change it, and execute means you can run it like a program.

You can see these permissions by using the `ls -la` command. This command lists all files, including hidden ones, and shows details about them. At the beginning of each line, you will see a string of letters and dashes that represent the permissions.

For example, you might see something like this: `-rwxr-xr--`. The first character tells you what type of file it is. A dash means it is a regular file, and a “**d**” means it is a directory. The next nine characters are split into three groups of three. The first group shows what the owner can do, the second shows what the group can do, and the third shows what everyone else can do.

In this example, the owner has full access (read, write, and execute), the group can read and execute but not make changes, and everyone else can only read the file. This setup helps make sure that only the right people can make changes, which is important for keeping a system secure and preventing unauthorized access.

Do you want to know more? Here is a good reference to learn more about it:

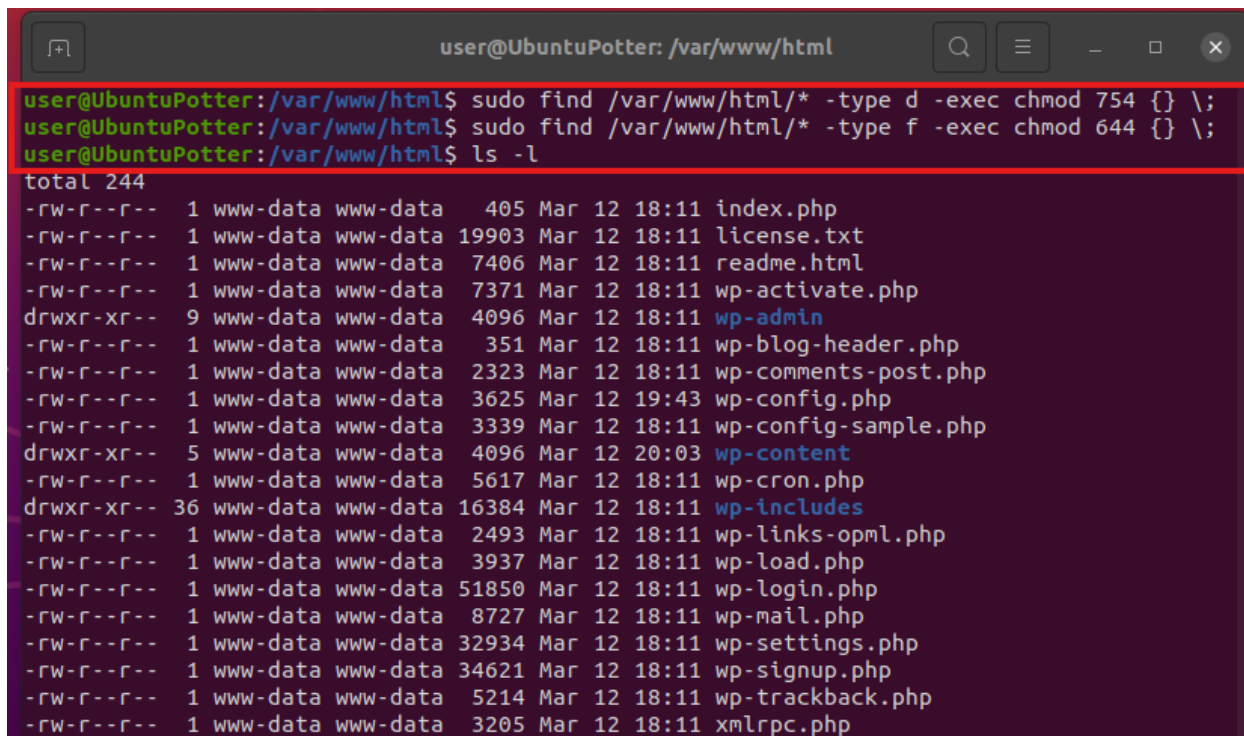
<https://www.redhat.com/en/blog/linux-file-permissions-explained>

Configuration

WordPress Makes the following recommendations for file permissions on files and folders. They recommend that Files should be set to 644 and what Folders should be set to 754. This will give users the minimal amount of privileges required and in order to maintain a functional website.

So first we will set the permissions of all the Folders. The command we will use here is “**sudo find . -type d -exec chmod 754 {} \;**” This will change the permissions of out folders it should look like this:

The next command will be to set the permissions of the files in the same directory. The command for this is “**sudo find . -type f -exec chmod 644 {} \;**” and then “**ls -l**” to see the new permissions. The output will look like this:



```
user@UbuntuPotter: /var/www/html
user@UbuntuPotter:/var/www/html$ sudo find /var/www/html/* -type d -exec chmod 754 {} \;
user@UbuntuPotter:/var/www/html$ sudo find /var/www/html/* -type f -exec chmod 644 {} \;
user@UbuntuPotter:/var/www/html$ ls -l
total 244
-rw-r--r-- 1 www-data www-data 405 Mar 12 18:11 index.php
-rw-r--r-- 1 www-data www-data 19903 Mar 12 18:11 license.txt
-rw-r--r-- 1 www-data www-data 7406 Mar 12 18:11 readme.html
-rw-r--r-- 1 www-data www-data 7371 Mar 12 18:11 wp-activate.php
drwxr-xr-- 9 www-data www-data 4096 Mar 12 18:11 wp-admin
-rw-r--r-- 1 www-data www-data 351 Mar 12 18:11 wp-blog-header.php
-rw-r--r-- 1 www-data www-data 2323 Mar 12 18:11 wp-comments-post.php
-rw-r--r-- 1 www-data www-data 3625 Mar 12 19:43 wp-config.php
-rw-r--r-- 1 www-data www-data 3339 Mar 12 18:11 wp-config-sample.php
drwxr-xr-- 5 www-data www-data 4096 Mar 12 20:03 wp-content
-rw-r--r-- 1 www-data www-data 5617 Mar 12 18:11 wp-cron.php
drwxr-xr-- 36 www-data www-data 16384 Mar 12 18:11 wp-includes
-rw-r--r-- 1 www-data www-data 2493 Mar 12 18:11 wp-links-opml.php
-rw-r--r-- 1 www-data www-data 3937 Mar 12 18:11 wp-load.php
-rw-r--r-- 1 www-data www-data 51850 Mar 12 18:11 wp-login.php
-rw-r--r-- 1 www-data www-data 8727 Mar 12 18:11 wp-mail.php
-rw-r--r-- 1 www-data www-data 32934 Mar 12 18:11 wp-settings.php
-rw-r--r-- 1 www-data www-data 34621 Mar 12 18:11 wp-signup.php
-rw-r--r-- 1 www-data www-data 5214 Mar 12 18:11 wp-trackback.php
-rw-r--r-- 1 www-data www-data 3205 Mar 12 18:11 xmlrpc.php
```

Validation

Now, Let out verify that the file permissions have been fixed. We will use the commands

"cd wp-admin/"

"cd wp-content/"

"cd wp-includes/"

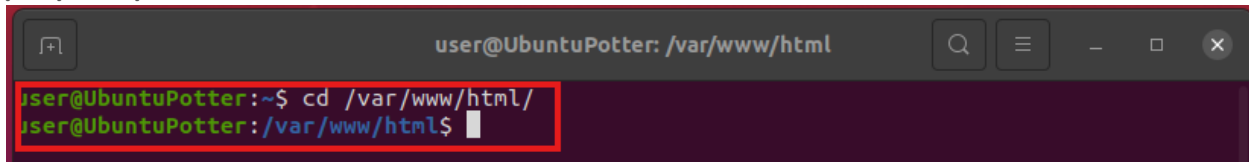
```
user@UbuntuPotter:/var/www/html$ cd wp-admin/
bash: cd: wp-admin/: Permission denied
user@UbuntuPotter:/var/www/html$ cd wp-content/
bash: cd: wp-content/: Permission denied
user@UbuntuPotter:/var/www/html$ cd wp-includes/
bash: cd: wp-includes/: Permission denied
user@UbuntuPotter:/var/www/html$
```

We as a normal user should now be denied access but wordpress will still be able to access these files and the site will continue to work because it is the owner and has full access.

Securing wp-config.php

Vulnerability

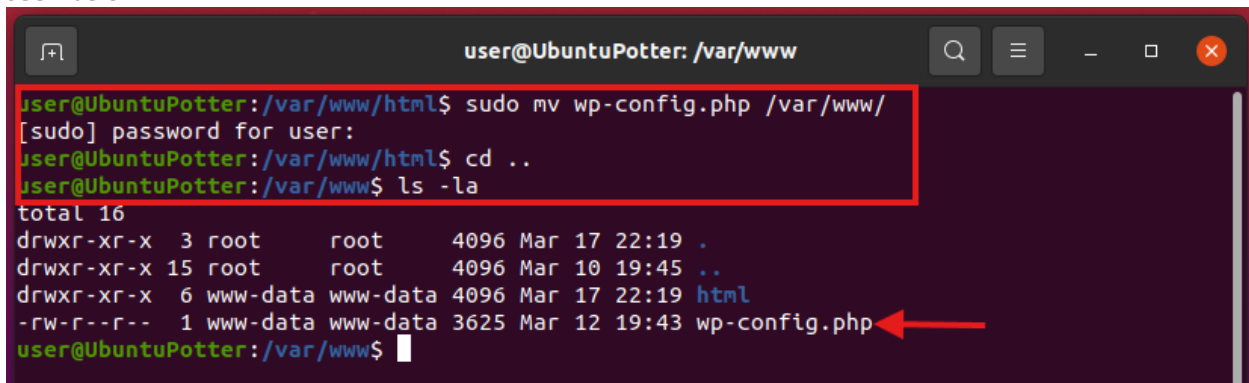
The Next step in hardening our WordPress is to move the configuration file Up one level to remove it from the web facing directory. This will make the file harder for a bad actor to find the config file to make changes to it. Our first step is to go back to the WordPress root directory. We will use the command “**cd /var/www/html**” It will look like this:

A terminal window titled 'user@UbuntuPotter: /var/www/html'. The prompt is 'user@UbuntuPotter:~\$'. The command 'cd /var/www/html/' is entered and executed. The prompt changes to 'user@UbuntuPotter:/var/www/html\$'.

```
user@UbuntuPotter:~$ cd /var/www/html/
user@UbuntuPotter:/var/www/html$
```

Configuration

Now to move the file to the level above this one we will use the command “**sudo mv wp-config.php /var/www/**” and to confirm it was moved we will use the “**cd ..**” command to go to /var/www/ directory and then use the command “**ls -la**” and we can see that the wp-config.php file is now in this directory. As seen below:

A terminal window titled 'user@UbuntuPotter: /var/www'. The prompt is 'user@UbuntuPotter:/var/www/html\$'. The command 'sudo mv wp-config.php /var/www/' is entered and executed. The prompt changes to 'user@UbuntuPotter:/var/www/html\$'. The command 'cd ..' is entered and executed. The prompt changes to 'user@UbuntuPotter:/var/www\$'. The command 'ls -la' is entered and executed. The output shows the directory listing for /var/www, with 'wp-config.php' listed as a file owned by www-data. A red arrow points to the 'wp-config.php' file in the listing.

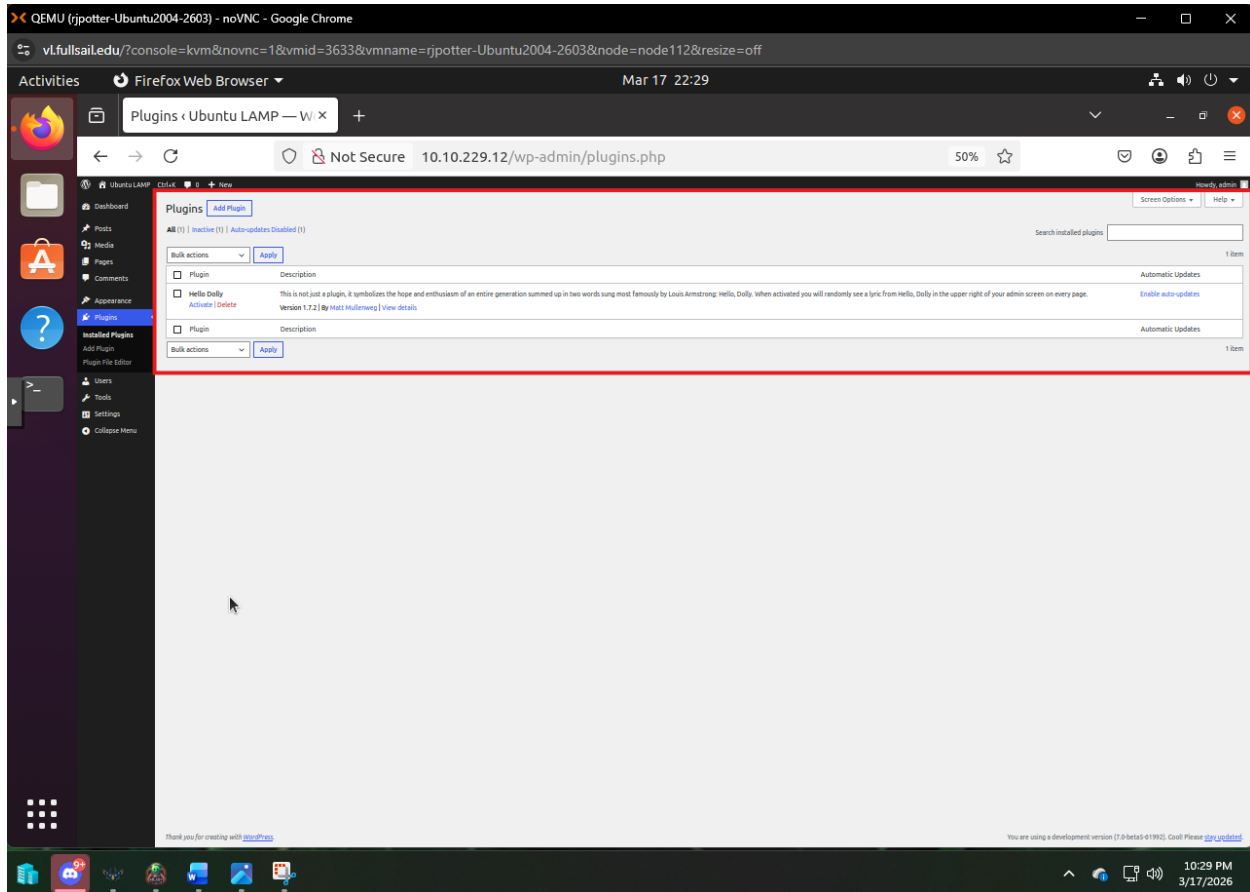
```
user@UbuntuPotter:/var/www/html$ sudo mv wp-config.php /var/www/
[sudo] password for user:
user@UbuntuPotter:/var/www/html$ cd ..
user@UbuntuPotter:/var/www$ ls -la
total 16
drwxr-xr-x  3 root    root    4096 Mar 17 22:19 .
drwxr-xr-x 15 root    root    4096 Mar 10 19:45 ..
drwxr-xr-x  6 www-data www-data 4096 Mar 17 22:19 html
-rw-r--r--  1 www-data www-data 3625 Mar 12 19:43 wp-config.php
user@UbuntuPotter:/var/www$
```

Validation

So as we can see the wp-config.php file is not in the /var/www directory. This will make it harder for an unskilled attacker to get access to it from the website and should deter bad actors.

Firewall (Shield)

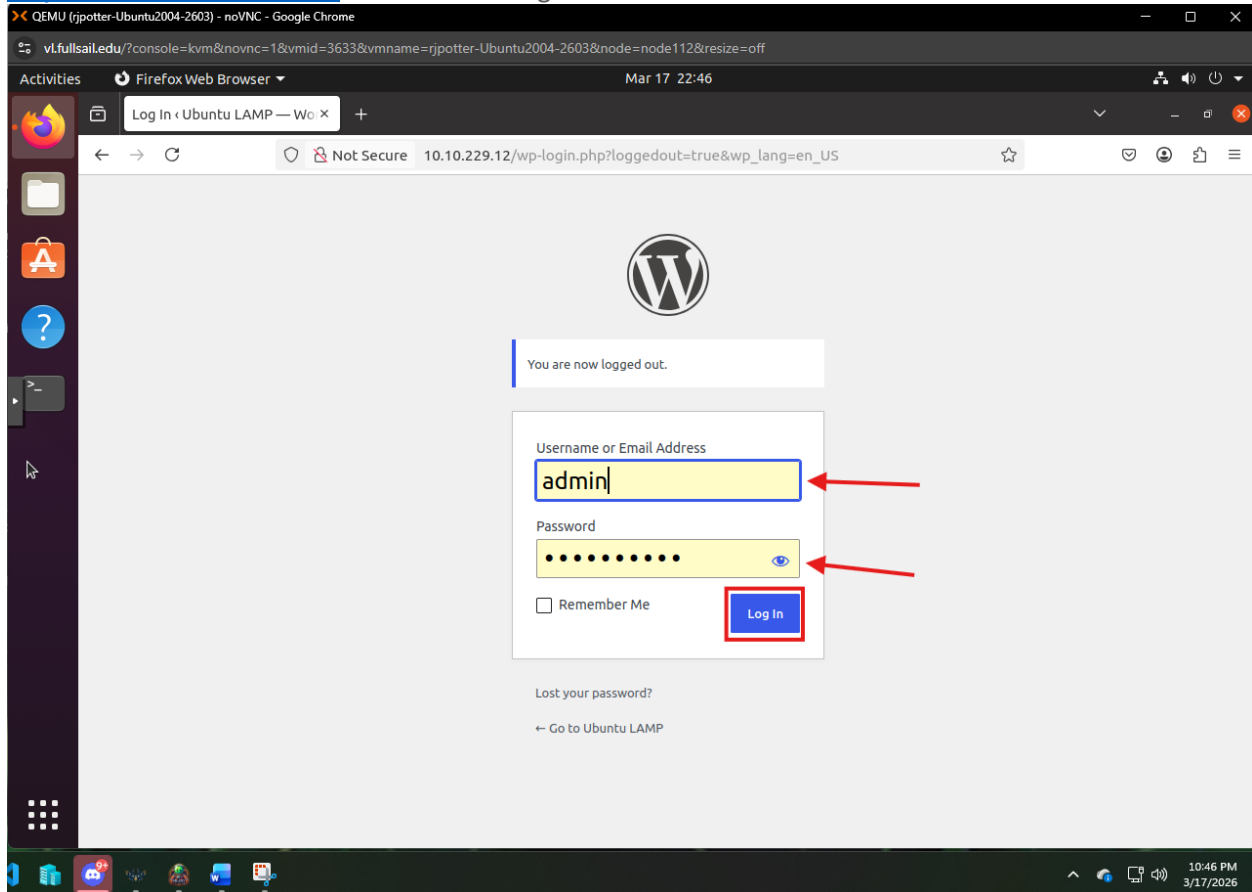
Vulnerability



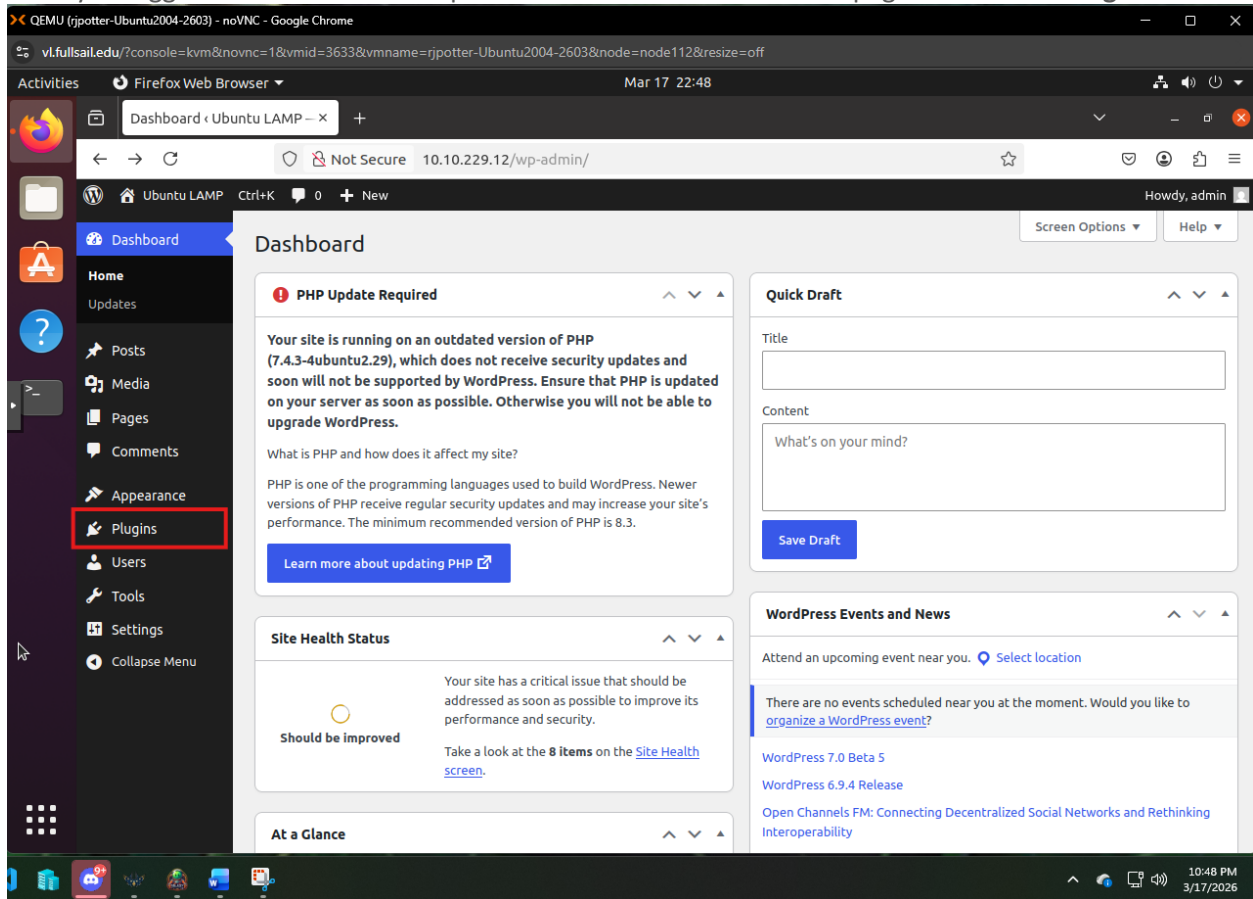
So currently on our WordPress site we have no firewall currently setup so we have no control over network traffic other than our existing hardware firewall. By adding WordPress firewall we can add an additional layer of protection for our setup.

Configuration

Our First step in the process is to access the admin login page. Open the Firefox browser and navigate to <http://10.10.229.12/admin> and enter the login credentials in this document.



Once you login in look at the admin panel on the left hand side of the page and click on **Plug-ins**



From here Click the “Add Plugins” button:

The screenshot shows a QEMU virtual machine running Ubuntu 20.04. The browser is displaying the WordPress admin interface at 10.10.229.12/wp-admin/plugins.php. The 'Add Plugin' button is highlighted with a red box. The left sidebar shows the 'Plugins' menu item selected. The main content area shows the 'Hello Dolly' plugin installed and active. The bottom status bar indicates the user is using a development version of WordPress (7.0-beta5-61992).

QEMU (rjpotter-Ubuntu2004-2603) - noVNC - Google Chrome

10.10.229.12/wp-admin/plugins.php

Plugins < Ubuntu LAMP — W x

Not Secure 10.10.229.12/wp-admin/plugins.php

Howdy, admin

Screen Options Help

Plugins

[Add Plugin](#)

All (1) | Inactive (1) | Auto-updates Disabled (1)

Search installed plugins

Bulk actions [Apply](#) 1 item

☐	Plugin	Description	Automatic Updates
☐	Hello Dolly Activate Delete	This is not just a plugin, it symbolizes the hope and enthusiasm of an entire generation summed up in two words sung most famously by Louis Armstrong: Hello, Dolly. When activated you will randomly see a lyric from Hello, Dolly in the upper right of your admin screen on every page. Version 1.7.2 By Matt Mullenweg View details	Enable auto-updates

Bulk actions [Apply](#) 1 item

10.10.229.12/wp-admin/plugin-install.php

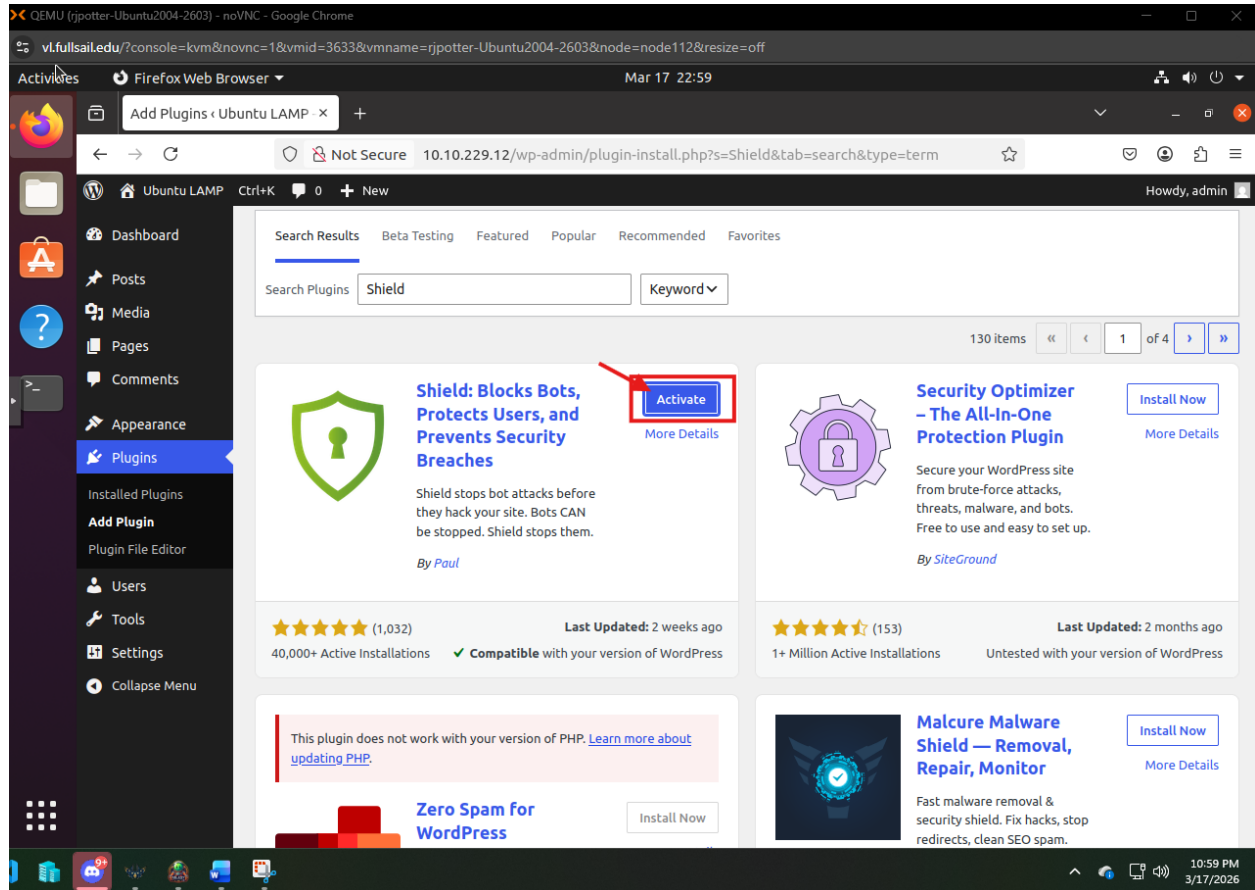
You are using a development version (7.0-beta5-61992). Cool! Please [stay updated](#).

10:49 PM 3/17/2026

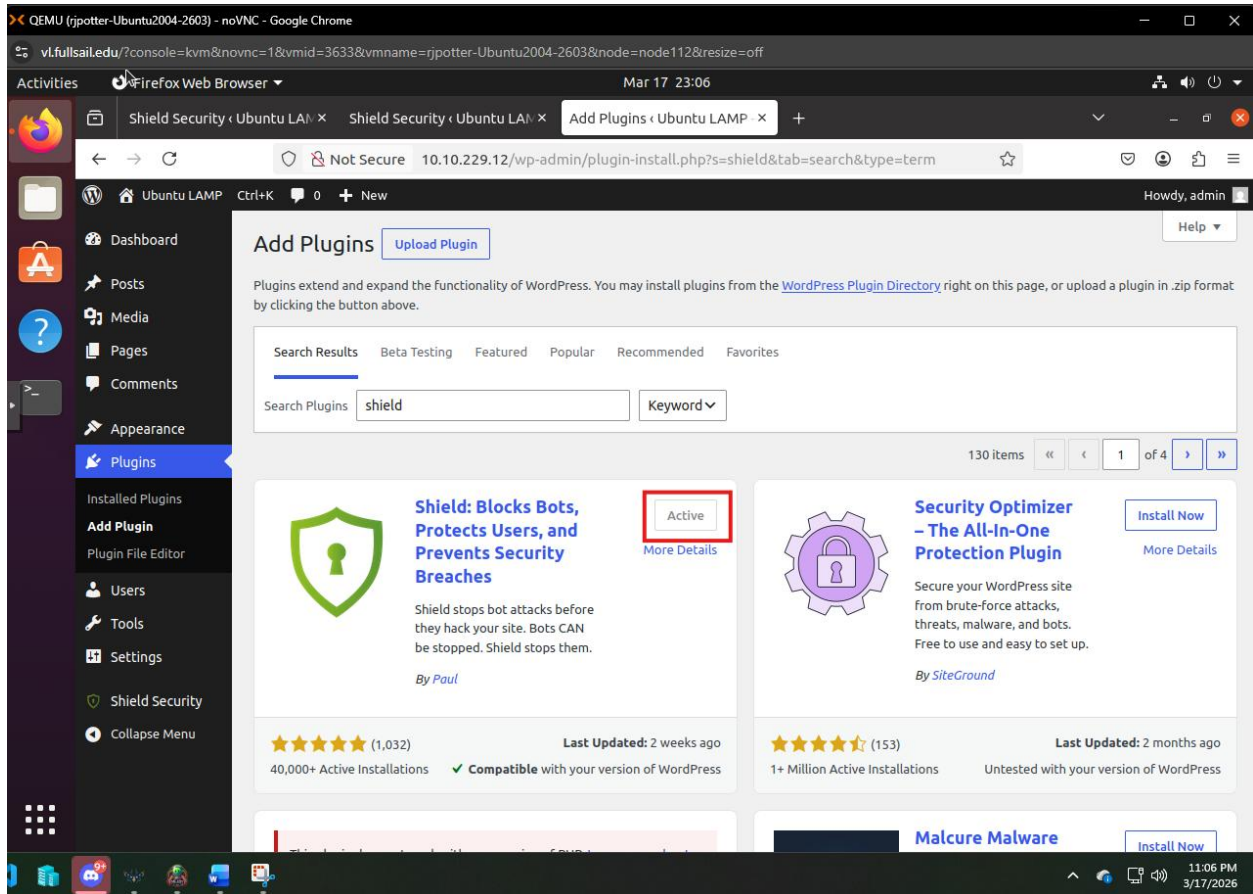
In the Search box Enter “Shield” and this is the Plugin we are looking for: “Press install”

The screenshot shows a web browser window displaying the WordPress plugin search results for 'Shield'. The browser's address bar shows the URL: `10.10.229.12/wp-admin/plugin-install.php?s=Shield&tab=search&type=term`. The search results page has a sidebar on the left with a 'Plugins' menu item highlighted. The main content area shows a search bar with 'Shield' entered. Below the search bar, there are three plugin cards. The first card, 'Shield: Blocks Bots, Protects Users, and Prevents Security Breaches', is highlighted with a red box. It features a green shield icon, a description, a 'By Paul' attribution, a 5-star rating with 1,032 reviews, 40,000+ active installations, and a 'Last Updated: 2 weeks ago' status. A red arrow points to the 'Install Now' button on this card. The second card, 'Security Optimizer - The All-In-One Protection Plugin', has a purple gear icon, a description, a 'By SiteGround' attribution, a 5-star rating with 153 reviews, 1+ million active installations, and a 'Last Updated: 2 months ago' status. The third card, 'Malware Shield - Removal, Repair, Monitor', has a blue shield icon, a description, a 'By SiteGround' attribution, a 5-star rating with 153 reviews, 1+ million active installations, and a 'Last Updated: 2 months ago' status. At the bottom of the page, there is a message: 'This plugin does not work with your version of PHP. [Learn more about updating PHP.](#)' and a 'Zero Spam for' section with an 'Install Now' button. The browser's status bar at the bottom shows the time as 10:55 PM on 3/17/2026.

Once its installed, Press “Activate”:

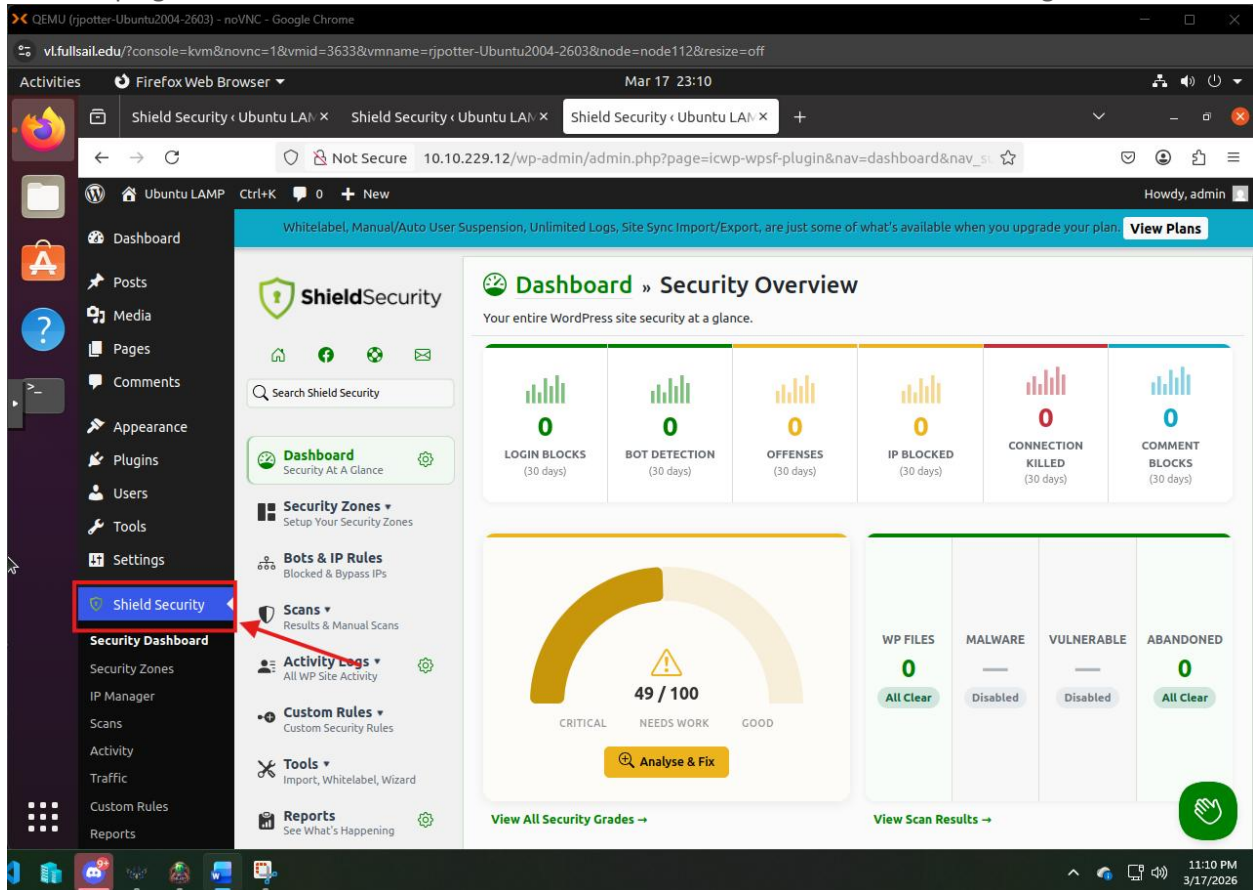


It should now show As active:



Validation

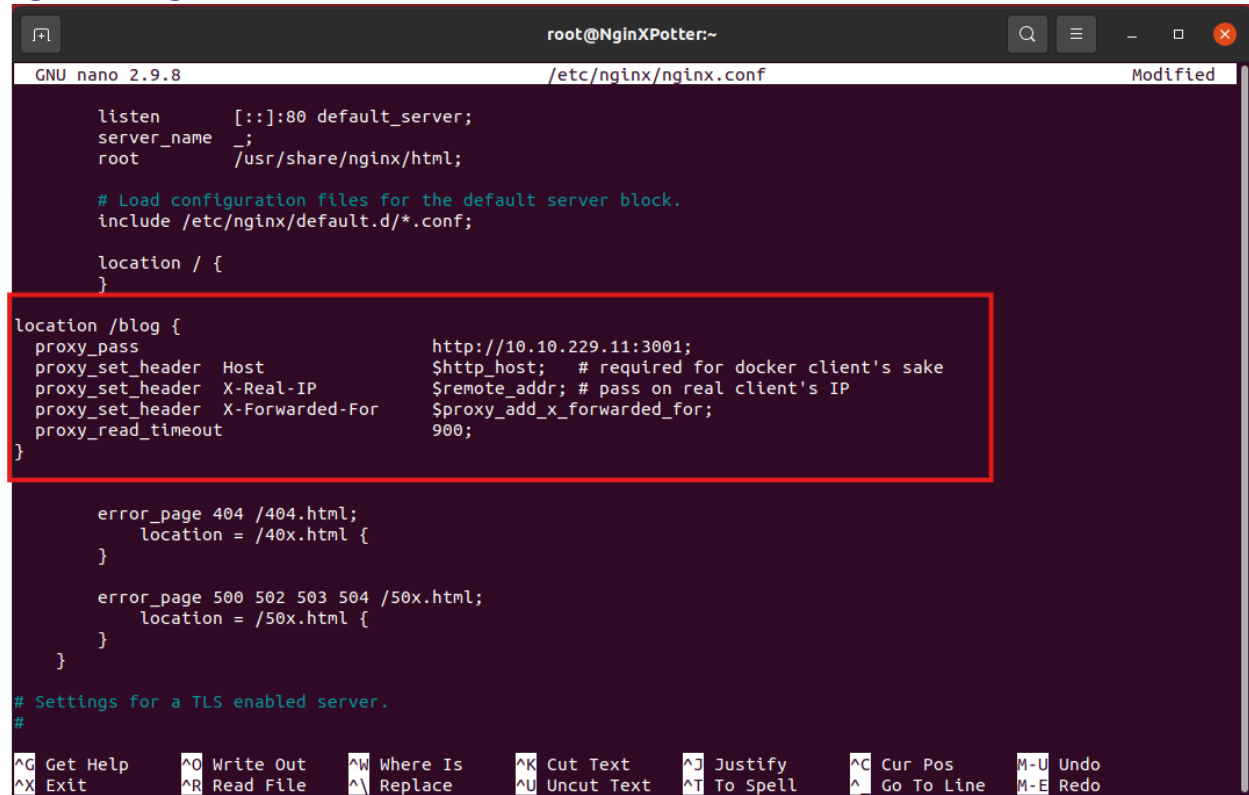
We can now View our dashboard and see that the security is no active and see any activity. This serves multiple purposes as it now protects us from bots as well as provides an additional source of logging traffic to our site so we know what kind of visitors we get and if there is any malicious activity toward our site. The plugin dashboard can be found in the lower left hand menu on the Admin Page:



t

Appendix A

NginX Config File



```
root@NginXPotter:~
GNU nano 2.9.8 /etc/nginx/nginx.conf Modified

listen      [::]:80 default_server;
server_name _;
root        /usr/share/nginx/html;

# Load configuration files for the default server block.
include /etc/nginx/default.d/*.conf;

location / {

location /blog {
    proxy_pass          http://10.10.229.11:3001;
    proxy_set_header    Host                $http_host; # required for docker client's sake
    proxy_set_header    X-Real-IP           $remote_addr; # pass on real client's IP
    proxy_set_header    X-Forwarded-For     $proxy_add_x_forwarded_for;
    proxy_read_timeout  900;
}

    error_page 404 /404.html;
        location = /40x.html {
    }

    error_page 500 502 503 504 /50x.html;
        location = /50x.html {
    }
}

# Settings for a TLS enabled server.
#

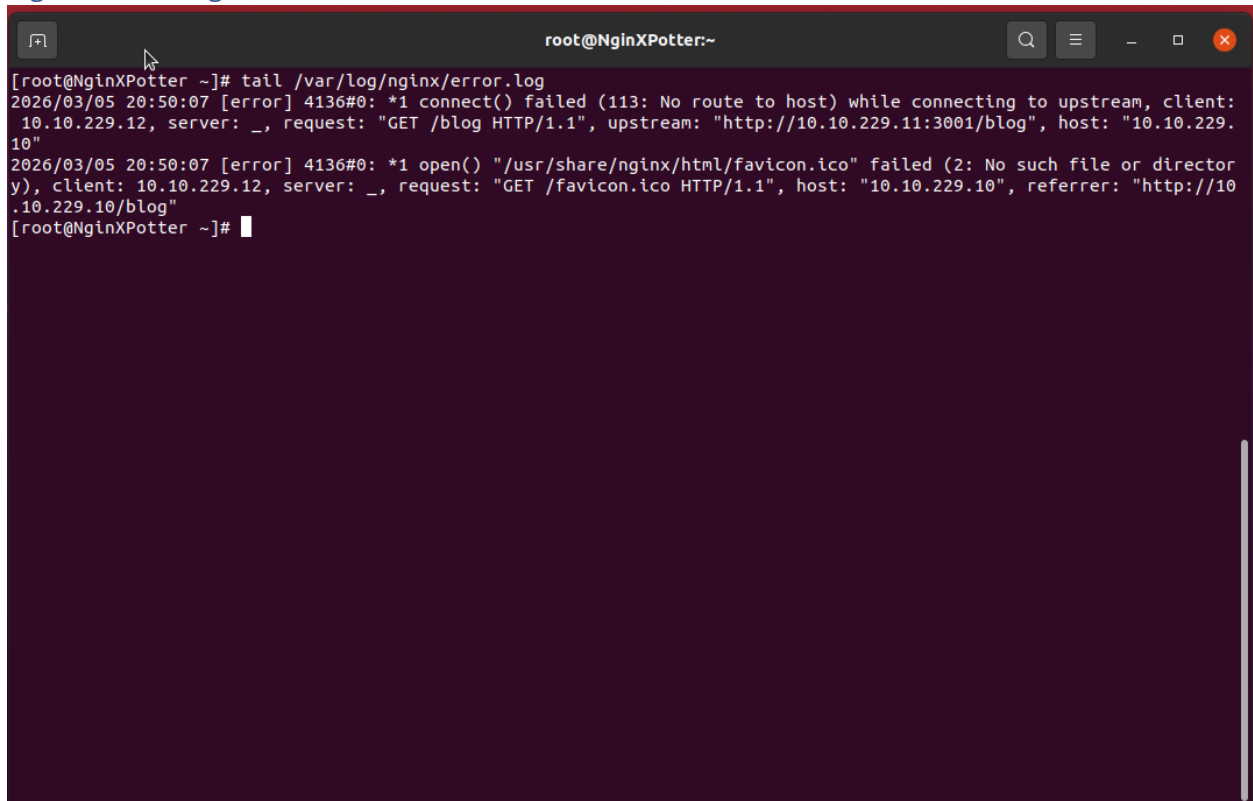
^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify     ^C Cur Pos     M-U Undo
^X Exit          ^R Read File    ^\ Replace      ^U Uncut Text   ^I To Spell    ^_ Go To Line   M-E Redo
```


Appendix B

NginX Access Log File

```
root@NginXPotter:~  
[root@DockerPotter ~]# docker run -d --name ghost -p 3001:2368 -e url=http://10.10.229.10/blog ghost  
8b41250f563bc145f3be664c7b0f065093700f923a0b3e96dc2eac926ba8272f  
[root@DockerPotter ~]# exit  
logout  
Connection to 10.10.229.11 closed.  
user@UbuntuPotter:~$ ssh root@10.10.229.10  
root@10.10.229.10's password:  
Activate the web console with: systemctl enable --now cockpit.socket  
  
Last login: Thu Mar  5 19:34:25 2026 from 10.10.229.12  
[root@NginXPotter ~]# tail /var/log/  
local/lock/ - log/  
[root@NginXPotter ~]# tail /var/log/nginx/access.log  
10.10.229.12 - - [05/Mar/2026:20:50:07 -0500] "GET /blog HTTP/1.1" 502 3404 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0" "-"  
10.10.229.12 - - [05/Mar/2026:20:50:07 -0500] "GET /nginx-logo.png HTTP/1.1" 200 368 "http://10.10.229.10/blog" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0" "-"  
10.10.229.12 - - [05/Mar/2026:20:50:07 -0500] "GET /poweredby.png HTTP/1.1" 200 1800 "http://10.10.229.10/blog" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0" "-"  
10.10.229.12 - - [05/Mar/2026:20:50:07 -0500] "GET /favicon.ico HTTP/1.1" 404 3332 "http://10.10.229.10/blog" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0" "-"  
[root@NginXPotter ~]#
```

NginX Error Log File

A terminal window titled 'root@NginXPotter:~' with search, menu, and window control icons in the title bar. The terminal shows the command 'tail /var/log/nginx/error.log' and its output, which contains two error messages from NginX. The first message is a connection failure to an upstream server, and the second is a file not found error for a favicon. The prompt returns to the root user after the log is displayed.

```
[root@NginXPotter ~]# tail /var/log/nginx/error.log
2026/03/05 20:50:07 [error] 4136#0: *1 connect() failed (113: No route to host) while connecting to upstream, client:
10.10.229.12, server: _, request: "GET /blog HTTP/1.1", upstream: "http://10.10.229.11:3001/blog", host: "10.10.229.
10"
2026/03/05 20:50:07 [error] 4136#0: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or director
y), client: 10.10.229.12, server: _, request: "GET /favicon.ico HTTP/1.1", host: "10.10.229.10", referer: "http://10
.10.229.10/blog"
[root@NginXPotter ~]#
```